SZENT ISTVÁN KATOLIKUS
TECHNIKUM ÉS GIMNÁZIUM

Albók Csaba, Varga-Ádám Róbert

MAISON DE PARFUM
SZOFTVERFEJLESZTŐ ÉS-TESZTELŐ VIZSGAREMEK

Sátoraljaújhely, 2026

Tartalom

1.TÉMAVÁLASZTÁS INDOKLÁSA 6
2.Felhasználói Dokumentáció 7
Üdvözöljük a Maison De Parfum felhasználói dokumentációjában! 7
Mire való a szoftver? 7
A szoftver funkciói 8
Regisztráció 8
Bejelentkezés 9
Kijelentkezés 10
Parfümök keresése 10
Parfüm kosárba helyezése 12
Parfüm törlése 13
Felhasználói Profil 14
Véleménykezelő rendszer 15
Rendszerkövetelmények 19
Hardverkövetelmények 19
Szoftverkövetelmények 21
3.Fejlesztői Dokumentáció 23
Alkalmazott fejlesztői eszközök 23
Backend fejlesztéshez használt eszközök: 23
Frontend fejlesztéshez használt eszközök: 23
Teszteléshez használt eszközök: 23
Dokumentáció készítéséhez használt eszközök: 23
Csapatmunkát segítő szoftverek: 24
A Projekt telepítése 24
Szükséges szoftverek és verziók 24
XAMPP Telepítési Útmutató 25
Visual Studio Code Telepítési Útmutató 25
Node.js Telepítési Útmutató 26
Alkalmazás futtatása a számítógépen 27
4.Kliens oldali megvalósítás és Architektúra (Frontend) 38
Fejlesztési környezet és konfiguráció 38
TypeScript konfiguráció 38

Adatkommunikációs Réteg (Services) ... 39
Hitelesítési Szolgáltatás (AuthService) ... 39
Reaktív Kosár Kezelés (CartService) ... 40
Felhasználói Felület Komponensei ... 42
Regisztrációs Modul (RegisterComponent) ... 42
Termékkatalógus és Szűrés (PerfumesComponent) ... 43
Modális Kosár (CartDialogComponent) ... 44
Adminisztrációs Felület és Védelem (AdminComponent & AdminGuard) ... 45
@Injectable({ providedIn: 'root' }) ... 45
Felhasználói Profil (ProfileComponent) ... 45
Stílus és Felhasználói Élmény (UX/UI) ... 46
Tesztelés és Minőségbiztosítás ... 46
Összegzés: ... 47
5.BACKEND ... 47
Node.js és a Futtatókörnyezet ... 47
Express.js Keretrendszer ... 47
Adatbázis és ORM: MySQL és Sequelize ... 48
Hitelesítés és Biztonság (JWT és Bcrypt) ... 48
Adatbázis Tervezés és Adatmodellek ... 48
Felhasználók (Users) Modell ... 48
Termékek (Perfumes) Modell ... 49
Kosár és Kosár Tartalom ... 50
Backend Architektúra és API Végpontok ... 51
A szerver inicializálása (server.js) ... 51
Hitelesítési Folyamat (Auth Controller) ... 52
Regisztráció ... 52
Bejelentkezés ... 52
Middleware: A Kapuőr ... 53
Funkcionális Modulok Részletezése ... 53
Kosár Kezelés (Cart Logic) ... 53
Adminisztrációs Funkciók és Seeding ... 54
Terméklogika és Adatstruktúra Kezelés ... 57
Termékek Listázása (GET /perfumes) ... 57
Profilkezelés és Adatbiztonság ... 58
Profil Frissítési Logika (Update Profile) ... 58
Válasz "Sanitization" (Adattisztítás) ... 58

Vélemények Kezelése és Jogosultsági Szintek ... 59
Kettős Jogosultsági Ellenőrzés ... 59
Kapcsolatok az adatbázisban ... 59
Rendszerkonfiguráció és Üzemeltetés ... 60
Környezeti Változók (.env) ... 60
CORS (Cross-Origin Resource Sharing) ... 60
Hibaelhárítási Útmutató (Troubleshooting) ... 60
Adatbázis Kapcsolódási Hibák ... 61
JWT Token Hibák ... 61
Seeding (Adatfeltöltés) Problémák ... 61
Részletes API Referencia és Dokumentáció ... 61
Hitelesítés (Auth) Végpontok ... 61
Termék (Perfumes) Végpontok ... 62
Kosár (Cart) Végpontok ... 63
Projekt Strukturális Összefoglalása (File System) ... 63
Jövőbeli Fejlesztési Irányok ... 65
Összefoglalás ... 65
6.Adatbázis Tervezés és Specifikáció ... 66
Entitás-Kapcsolat Modell (ERD) ... 66
Táblák Részletes Definíciója ... 67
Felhasználók tábla (felhasznalok) ... 67
Parfümök tábla (perfumes) ... 68
Kosár Fej tábla (kosar) ... 69
Kosár Tartalom tábla (kosar_tartalom) ... 69
Vélemények tábla (velemenyek) ... 70
Adatbázis Kapcsolatok és Integritás ... 70
Fizikai Megvalósítás (SQL) ... 71
Adatvédelmi és Biztonsági Megfontolások (GDPR) ... 71
Indexelési Stratégia és Teljesítmény ... 71
SQL Implementáció (DDL Szkriptek) ... 72
Üzleti Folyamatok Adatbázis Leképezése ... 73
A) Kosár Megtekintése (Összetett Lekérdezés) ... 73
Árváltozás Kezelése ... 73
Összegzés és Végszó ... 74
A projekt értékelése ... 74
Szakmai fejlődés ... 74

Továbbfejlesztési lehetőségek 74
7.Szoftvertesztelés 75
Bevezetés és Tesztelési Stratégia 75
Egységtesztelés (Unit Testing) 75
Frontend Egységtesztek (Angular) 75
Backend Egységtesztek (Node.js/Controller Logic) 76
Integrációs Tesztelés 76
API Kommunikáció és Adatfolyam 77
Autentikációs Middleware Tesztek 77
Fehér Dobozos Tesztelés (White-box Testing) 77
Kódlefedettség és Elágazások vizsgálata 77
Adatbázis Seederek és Migrációk 78
Biztonsági rések keresése a kódban 78
Fekete Dobozos Tesztelés (Black-box Testing) 78
UI/UX és Reszponzivitás Tesztelése 79
Kompatibilitási Tesztek 79
Összegzés és Következtetések 80
8.Irodalomjegyzék, források 81
EREDETISÉGI NYILATKOZAT 85

1.TÉMAVÁLASZTÁS INDOKLÁSA

A **Maison De Parfum** kizárólag parfümöket árusító webshop létrehozása több szempontból is indokolt és üzletileg megalapozott döntés lehet. A parfümök piaca egy jól körülhatárolható, niche szegmens, amely lehetővé teszi a könnyebb pozicionálást, a célcsoport pontosabb megszólítását és a hatékonyabb marketingstratégiák kialakítását. Egy ilyen specializált webshop kisebb versennyel néz szembe, miközben a keresőoptimalizálás is jelentősen egyszerűbbé válik, hiszen a kulcsszavak, termékkategóriák és tartalmak mind egy adott témára fókuszálhatnak.

A parfümök további előnye, hogy magas árréssel és egész éves, stabil kereslettel rendelkeznek. Ezek a termékek különösen jól fogynak ünnepi időszakokban, így kiszámítható bevételi forrást biztosítanak. A termékkínálat egyszerűsége miatt a készletkezelés is könnyen átlátható, hiszen a variációk leggyakrabban a kiszerelésre, a nemre és a márkára korlátozódnak. Ez a termékjellemző erősen csökkenti a backend fejlesztés komplexitását és szignifikánsan kevesebb hibalehetőséget eredményez.

Fejlesztői szempontból egy parfümwebshop ideális projekt: letisztult adatmodellel dolgozik, mégis alkalmas arra, hogy bemutassa egy teljes értékű e-kereskedelmi rendszer működését. A CRUD műveletek, a kategóriák, az admin felület, a kosár és a rendeléslogika mind megvalósíthatók anélkül, hogy túlzottan bonyolult termékstruktúrával kellene foglalkozni. Emellett látványos és esztétikus frontend készíthető, mivel a parfümök vizuálisan és érzelmileg is jól kommunikálható termékek.

Összességében a parfümwebshop egyszerre egyszerű és professzionális projektalap, amely üzleti és fejlesztői szempontból is jól kezelhető, indokolt választás. Elég összetett ahhoz, hogy valós rendszert modellezzen, ugyanakkor elég letisztult ahhoz, hogy a hangsúly a fejlesztésen, ne a terméklogika bonyolultságán legyen.

A **Maison De Parfum** tehát több szempontból is indokolt, tökéletes niche szegmens egyedileg kiemelkedik, megtalálható rajta olcsóbb, és drágább parfüm is, így kielégítve a vásárlók teljes igényét. Emellett a webshop specializált termékkínálata lehetővé teszi a könnyebb marketinget és célzott ajánlatok kialakítását, miközben a vásárlók számára egy átlátható, letisztult és esztétikus vásárlási élményt biztosít. A különleges illatok és a márkák széles választéka pedig hozzájárul ahhoz, hogy a **Maison De Parfum** hűséges és visszatérő ügyfélkört építsen.

2.Felhasználói Dokumentáció

Üdvözöljük a Maison De Parfum felhasználói dokumentációjában!

Ebben a részben bemutatjuk az alkalmazás működését, annak funkcióit, és külön részletesen mutatjuk be a hatékony, és egyszerű véleményküldési, illetve kezelési rendszerét! Az alkalmazás célja, hogy létrehozzunk egy olyan niche szegmens parfümökhöz kötődő webalkalmazást, ami mindenki számára könnyen kezelhető, letisztult, és szívesen vásárolna!

Mire való a szoftver?

Az MDP egy olyan platform, amely parfümöket árusít minden felhasználó számára. Az alkalmazás célja az, hogy minden felhasználó számára biztosítson egy olyan webshopot, ahonnan szívesen vásárolnának, illetve még cél, hogy megkönnyítse a vásárlást, azon felhasználók számára akik kifejezetten parfümöt szeretnének vásárolni, és nem kívánnak különböző féle drogériai termékek között válogatni!

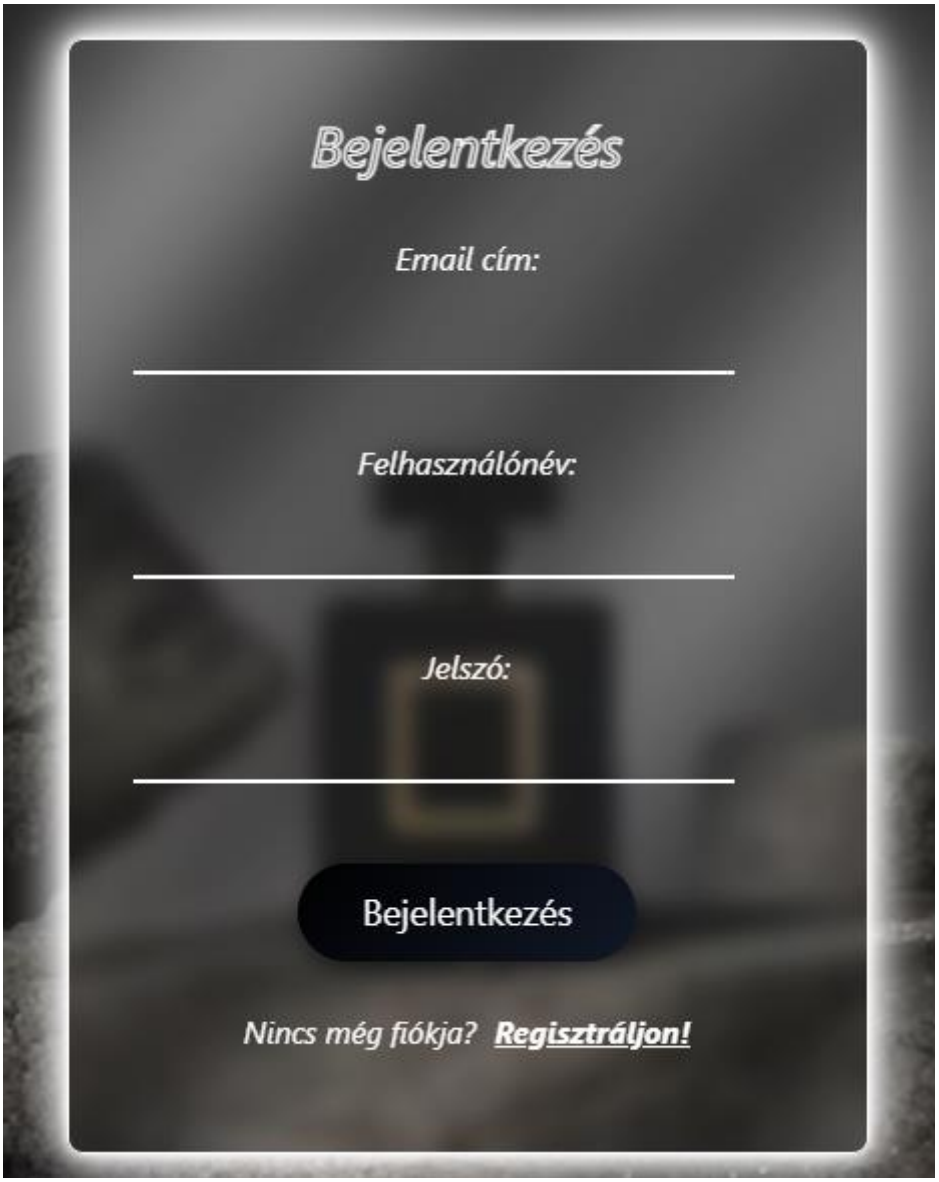
Érdemes említést tenni a szoftver funkcióról pontosabban. Az alkalmazás a következő funkciókkal rendelkezik: Regisztráció, Bejelentkezés, Kijelentkezés, Vélemény-küldés, Vélemény-módosítás, Vélemény Törlés, Parfümök keresése (search engine), Parfüm kosárba helyezése, Parfüm törlése a kosárból, Felhasználói profil kezelése.

A szoftver funkciói

Regisztráció

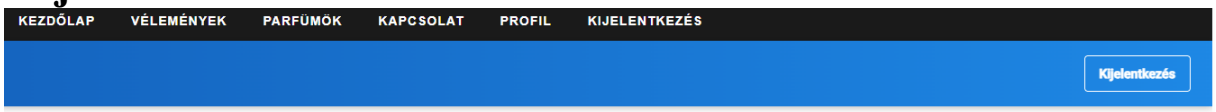
Ezen a felületen lehet regisztrálni az oldalra. Az oldal a következő adatokat kéri a felhasználótól: Teljes név, email cím, Felhasználónév, és jelszó. A regisztráció gombra kattintva a frontend küld egy POST kérést a backendek, a backend leellenőrzi, hogy minden mező ki lett-e töltve és minden megfelel-e az elvártaknak. Ha valami nincsen kitöltve, jelzi, hogy ki kell tölteni, ha már létezik a felhasználó, szintén hibát dob, ha minden tökéletes, és megfelel a felhasználó adatai mentésre kerülnek az adatbázisban. Ha már regisztrált a felhasználó a „Jelentkezz be!” gombra kattintva átirányításra kerül a bejelentkezés oldalra.

Bejelentkezés



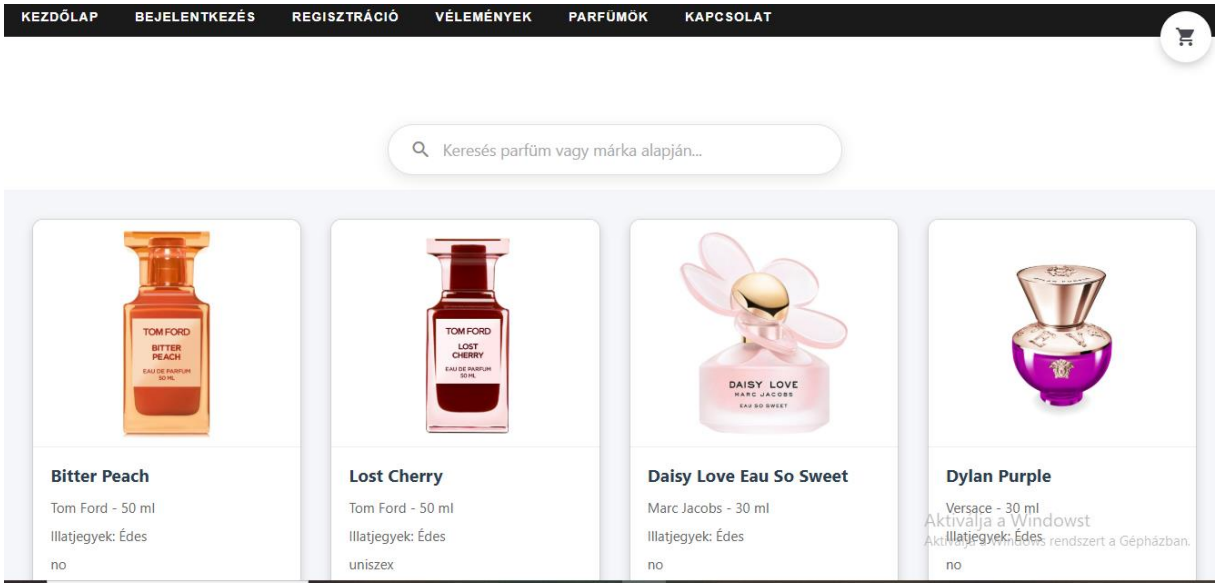
Ezen a felületen lehet bejelentkezni az oldalra. Az oldal a következő adatokat kéri a felhasználótól: email cím, Felhasználónév, és jelszó. A bejelentkezés gombra kattintva a frontend küld egy POST kérést a backendek, a backend le ellenőrzi, hogy minden mező ki lett e töltve, és minden megfelel e az elvártaknak. Ha valami nincsen kitöltve, jelzi, hogy ki kell tölteni. A backend ellenőrzi, hogy létező email cím lett e beírva, illetve a felhasználónév az a felhasználóhoz tartozó felhasználónév e, ha minden tökéletes generál egy JWT tokent, ami 1 óráig él, és átirányításra kerül a fő oldalra. Ha nincs még fiókja a „Regisztráljon!” gombra kattintva átirányításra kerül a regisztrációs oldalra!

Kijelentkezés

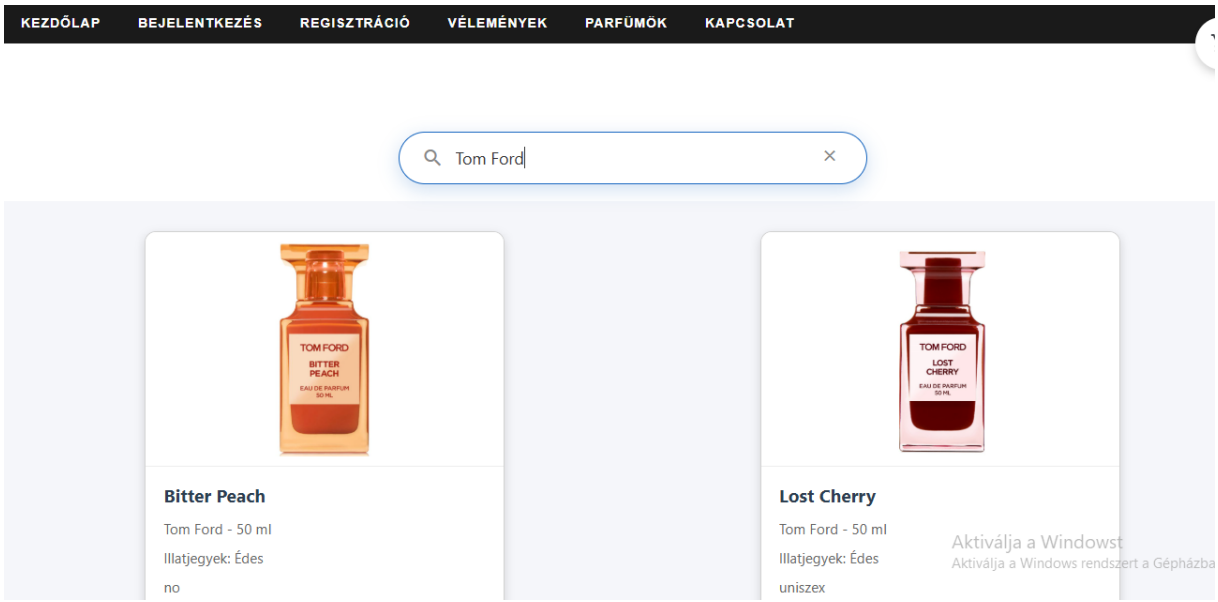


Az oldalon a kijelentkezés gomb kétszer jelenik meg. A vélemények oldalon felül található egy kijelentkezés gomb, illetve ha a felhasználó bevan jelentkezve a navbar-ból is ki tud jelentkezni a felhasználó! A kijelentkezés gombra kattinva, a frontend kérést küldd a backendnek, és ellenőrzi a middleware-ben, hogy a felhasználó jelenleg rendelkezik e JWT tokennel, ha igen a tokent beteszi a blackList-be, és a felhasználó sikeresen kijelentkezett. Ha nem rendelkezik JWT tokennel, akkor Visszaküldi, hogy a felhasználó nincsen bejelentkezve!

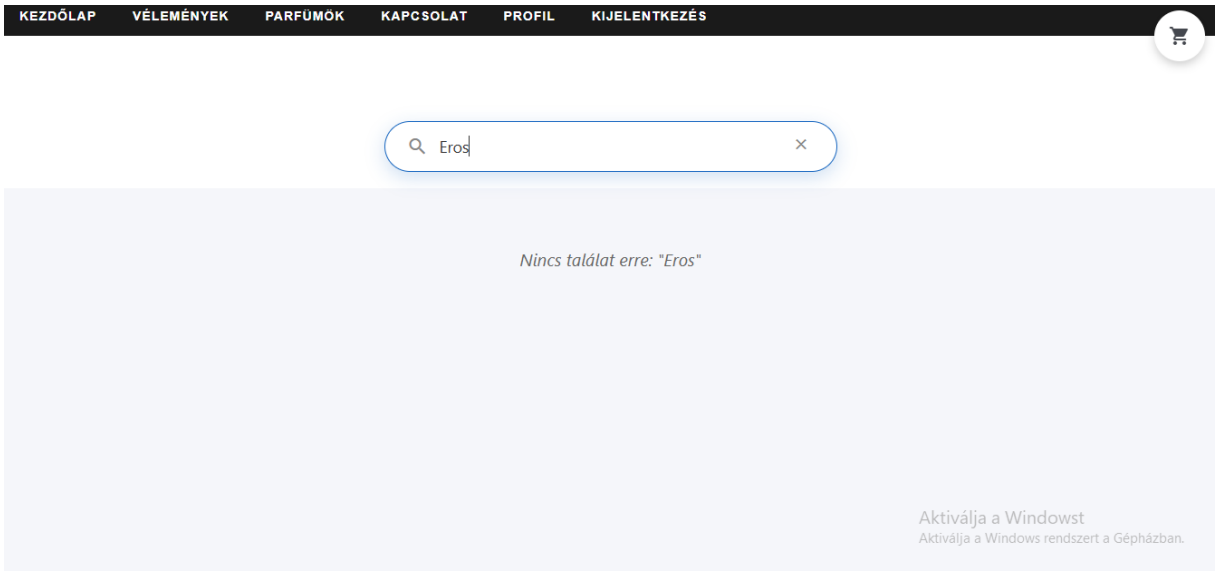
Parfümök keresése



Ahogy a felhasználó megnyitja a Parfümök oldalt ezt fogja látni maga előtt. Az oldalon jelenleg 30 parfüm érhető el, amik közül válogatni lehet, és kosárba lehet helyezni. Minden parfümről 5 adat jelenik meg: Kép a parfümről, a parfüm neve, a parfüm márkája –és űrtartalma, a parfüm jellemzője (édes vagy fűszeres), illetve, hogy női, férfi vagy uniszex.

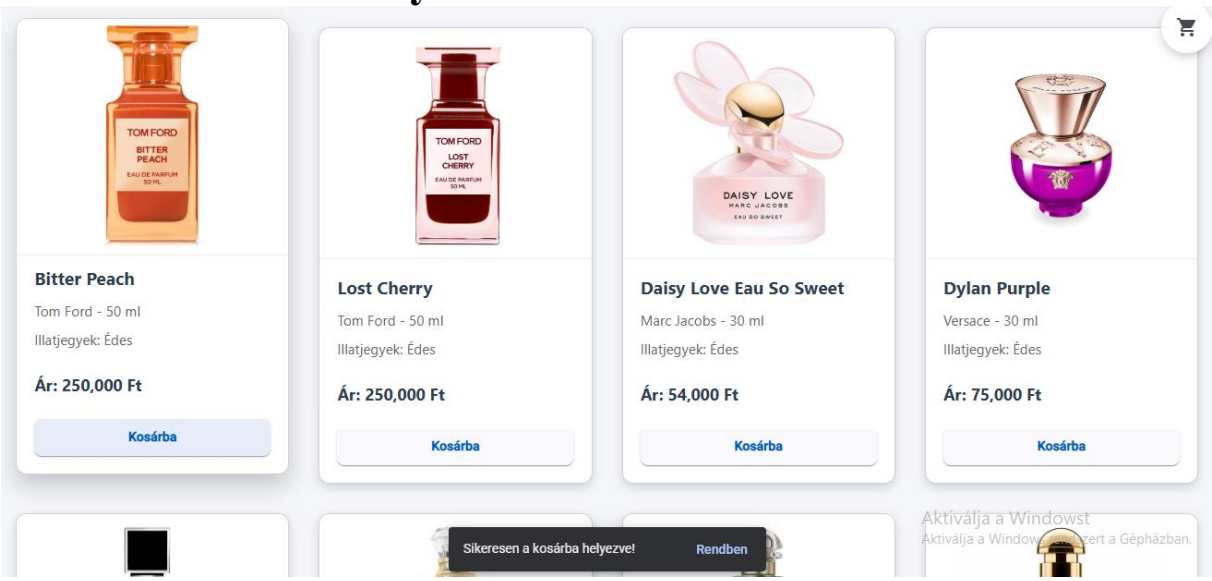


Érdemes említést tenni a keresőmotorról is! A keresősáv (motor) látványosan van még a parfümkártyák felett elhelyezve, és arra szolgál, hogy a felhasználó könnyebben érje el az általa megvételre kívánt parfümöt. Ha kíváncsi konkrétan egy parfümre, akkor beírja és ha van találat akkor megjelenik az a parfüm(vagy parfümök) amit keresett, a többi pedig addig nem látszik, és „eltűnik” az oldalról. Jelenleg a kép azt az esetet illusztrálja, amikor olyan parfüm kerül keresésre, ami megtalálható az oldalon.

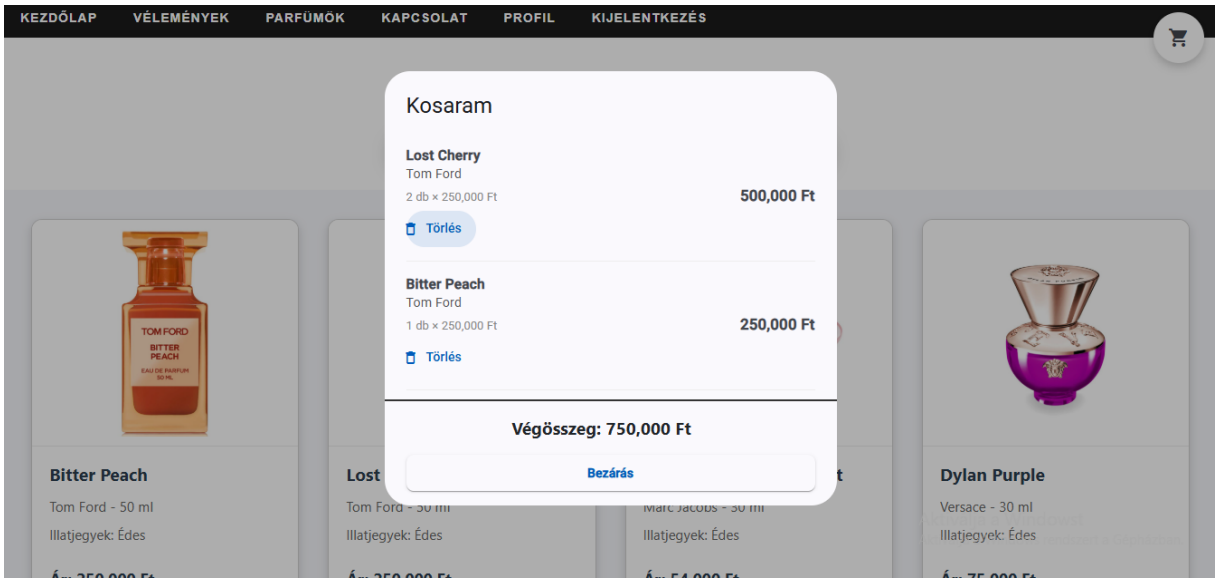


A keresőmotor figyelembe veszi, azt az esetet is amikor nincsen találat a keresett parfümre, ezt illusztálja a kép. Amikor olyan parfümre keres rá a felhasználó, ami nem érhető el, megjelenik a szöveg, és egy parfüm sem látható az oldalon. A kereső jobb közepén lévő x ikon szolgál arra, hogy újra „üres” legyen a keresőmező, és újra megjelenik minden parfüm!

Parfüm kosárba helyezése

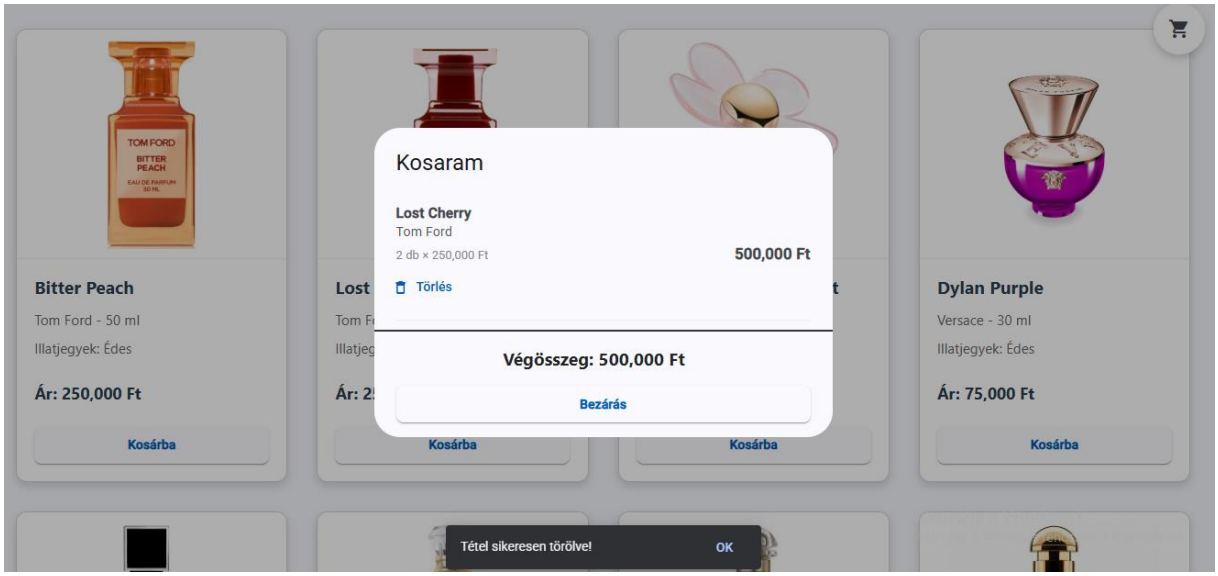


A „Kosárba” gombra kattintva 2 lehetőség jöhet fel. A képen a sikeres kosárba helyezés látszik, viszont ha a felhasználó nincsen bejelentkezve, akkor megkéri a felhasználót, hogy jelentkezzen be.



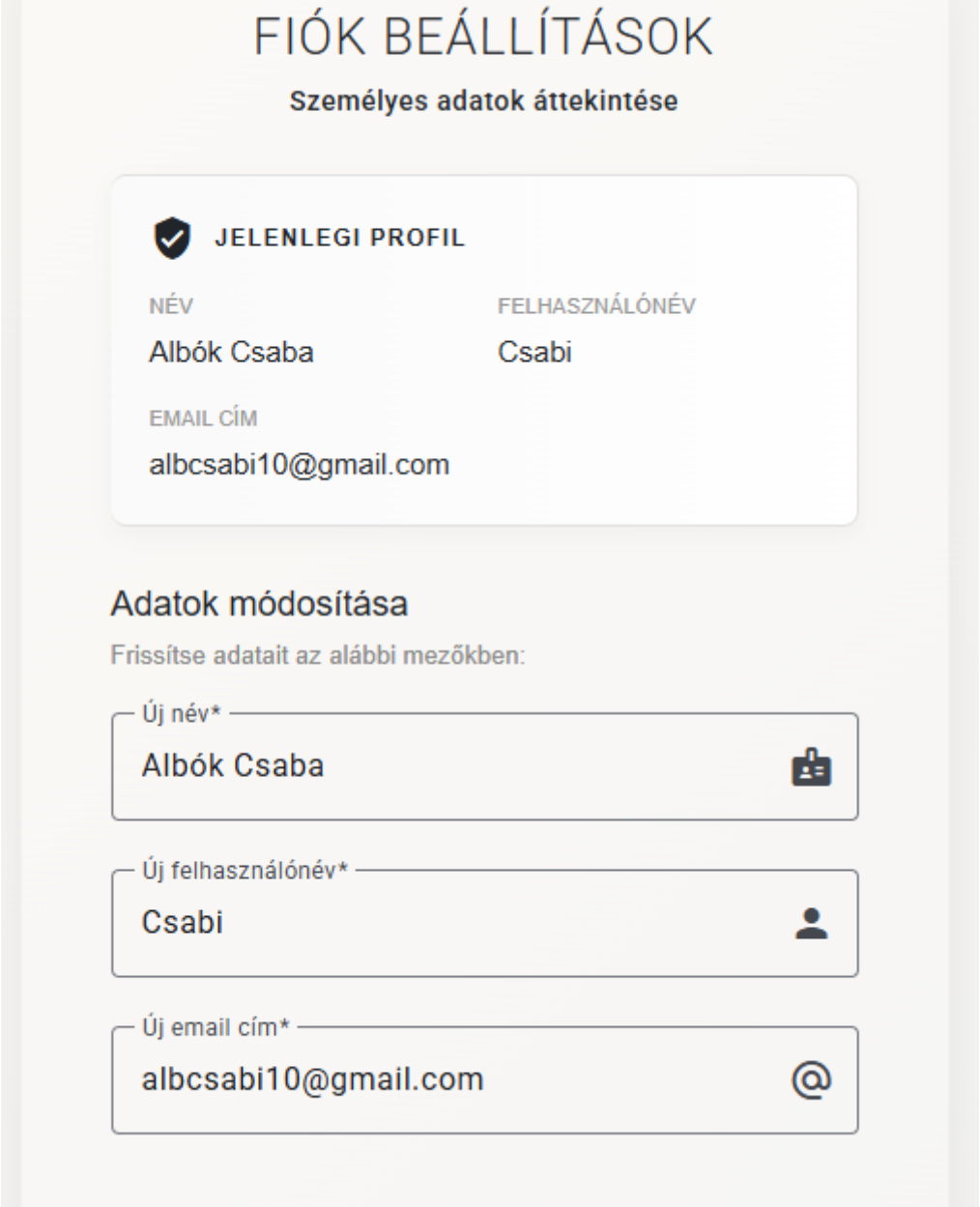
A kosárban megjelenik az összes parfüm, amit betettünk a kosárba, figyelembe van véve, hogy ha ugyanolyan parfümből teszünk bele többet, akkor nem jelenik újra, különállóként, hanem hozzáadja a már meglévő parfümhöz, növelve a darabszámát.

Parfüm törlése



A törlés gombra kattintva, a parfüm kitörlődik a felhasználó kosarából, illetve az adatbázisban tárolt kosár adatai közül is törlődik.

Felhasználói Profil



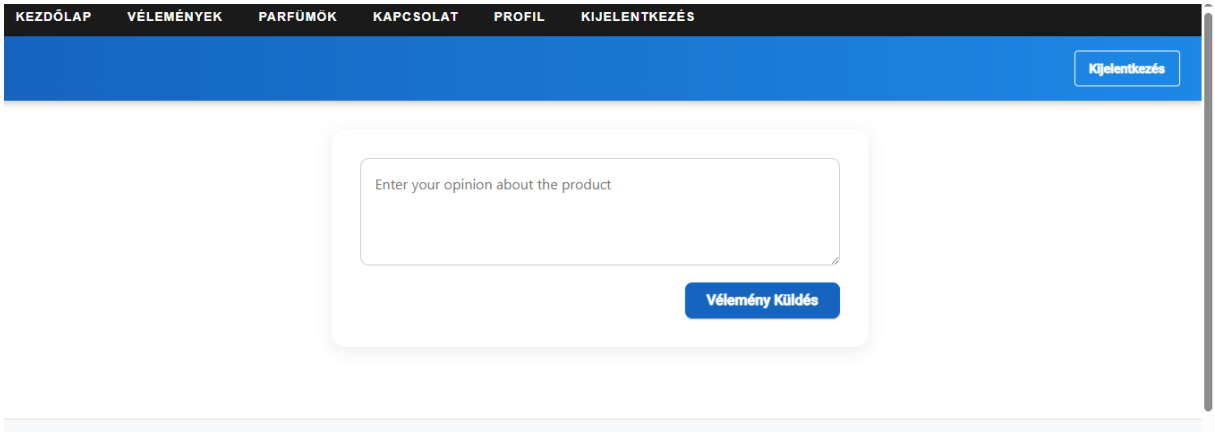
A felhasználói profil két részre bontható. A jelenlegi profil adatok, illetve módosítási lehetőség. A teljes név, email cím, illetve felhasználónév jelenik meg az oldalon, és ezeket lehet megváltoztatni. A backend figyelembe veszi a módosításnál, hogy az új adat nem lehet már meglévő email, vagy felhasználónév, ha minden tökéletes, akkor a módosítások mentésre kerülnek az adatbázisban, illetve dinamikusan már a frontenden is az új adatainkat fogjuk látni.

Véleménykezelő rendszer

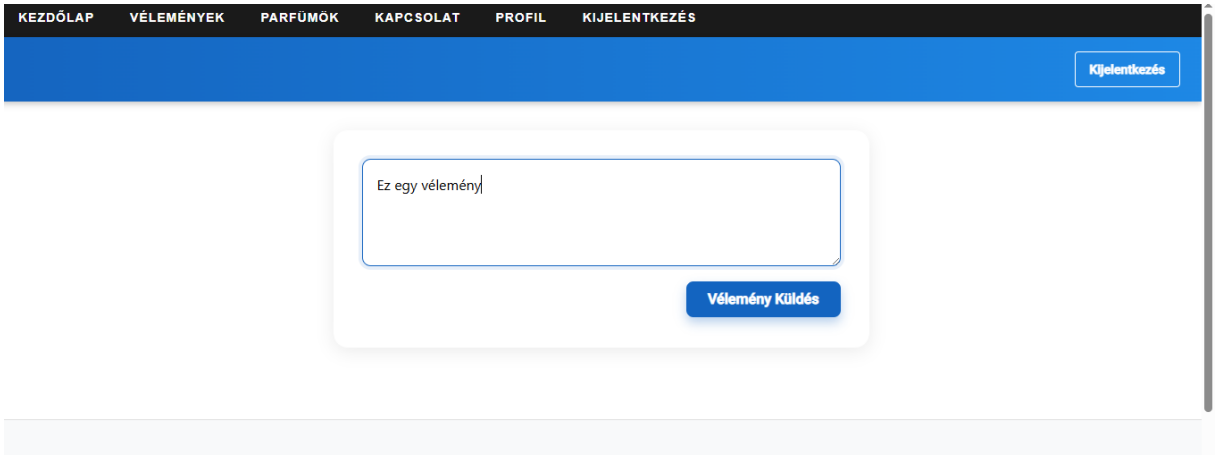
Ezek közül a legfontosabb funkció a teljes véleménykezelő rendszer. A véleménykezelés minden elemét tartalmazza a CRUD műveleteknek. A véleménykezelő rendszer 3 különböző verzió szerint lett feltöltve. Fontos megjegyezni, hogy annak érdekében, hogy a felhasználó tudjon véleményt küldeni szükséges volt egy regisztrált fiók. A vélemények 1.verziójában egy egyszerű felületen jelent meg a user felhasználóneve, és az általa beküldött vélemény. Itt még a user nem tudta kezelni a véleményét, nem tudta módosítani, illetve törölni, csak megtekinteni az oldalon.

A 2.verziójában bekerült, hogy a felhasználó tudja módosítani, illetve törölni a véleményét. Ez rögtön frissült az adatbázisban, és a frontenden is egyből látszott.

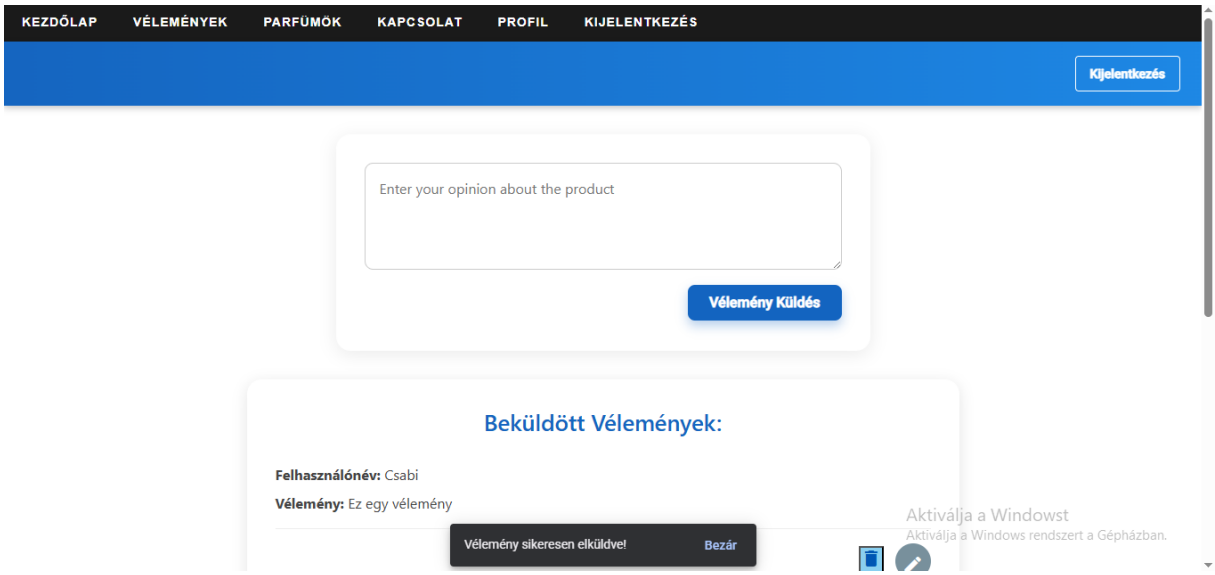
A 3. és egyben utolsó verziójában figyelembe vettük, hogy a felhasználó csak is kizárólag a saját véleményét tudja kezelni, és ne tudja más felhasználó véleményét törölni, vagy módosítani. Így a végső verzió úgy néz ki, hogy minden felhasználó tudja kezelni a saját véleményét, és egyedül az adminoknak van joga minden véleményhez hozzá férni biztonsági okokból!



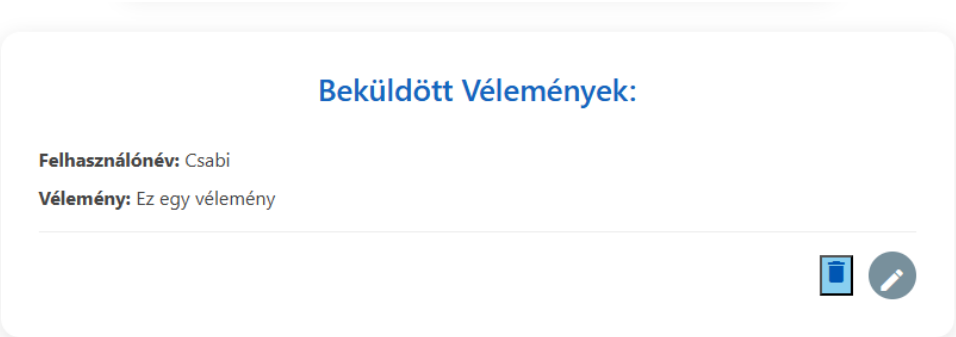
Ha az oldalon megnyitod a vélemények fület ezt látod magad előtt. Jelenleg egy olyan állapot, amikor nincsen rajta vélemény. Ahhoz, hogy a vélemények megjelenljenek, nem kell bejelentkezve lenni ez minden felhasználó számára elérhető. Ez az az állapot, amikor az oldal „üres” mivel az adatbázisban nem található vélemény.



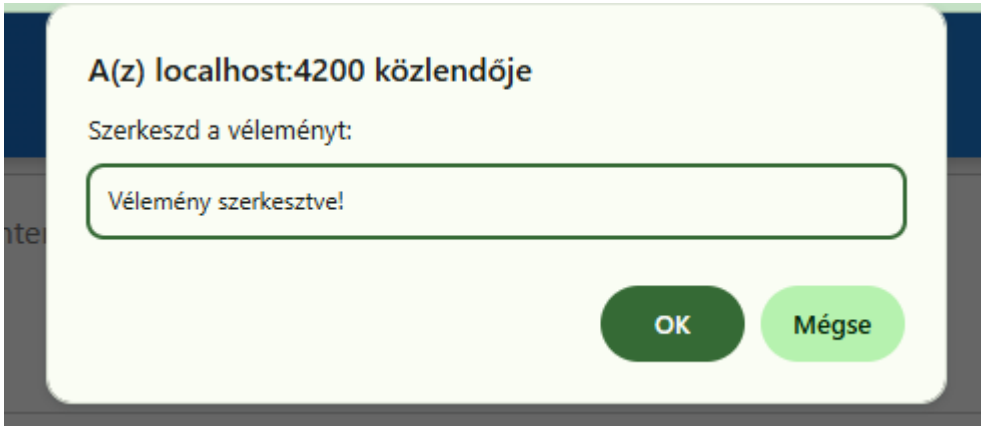
Nincsen megszabva, hogy milyen hosszú lehet a véleményed, amennyit szeretnél annyit írsz. Ahogy belekattintasz a mezőbe a textarea border-je kékre változik át te pedig nyugodtan elkezdheted gépelni amit szeretnél.



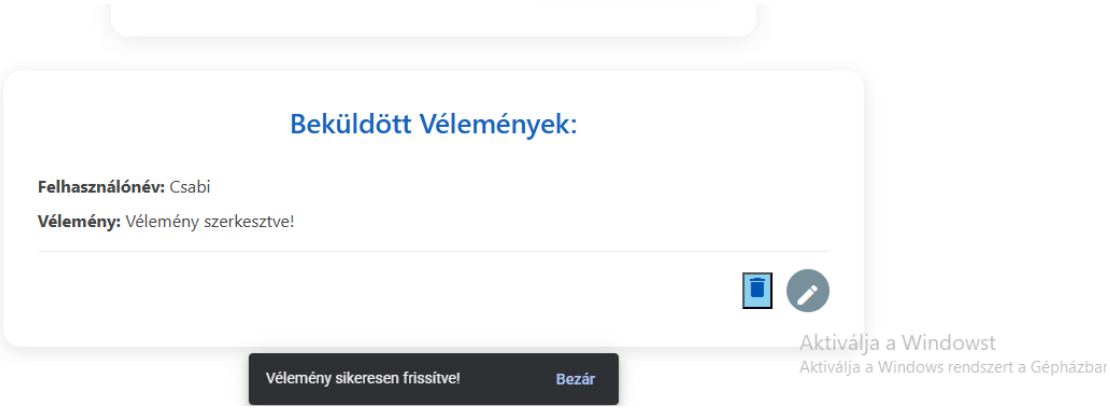
Ha rákattintasz a „Vélemény Küldés” gombra 2 opciód van. Az első amikor nem vagy bejelentkezve. Megjelenik egy SnackBar alul és megkér, hogy a vélemény küldéséhez jelentkezz be! Ha be vagy jelentkezve akkor kapsz egy „Vélemény sikeresen elküldve!” SnackBart az oldaltól. Mi Történt? Beírtad a véleményed az továábítva lát a backendnek, és mivel minden megfelelt a véleményed mentésre került az adatbázisba. Az adatbázisban a vélemények azonosítója hozzá tartozik a te felhasználói azonosítódhoz.



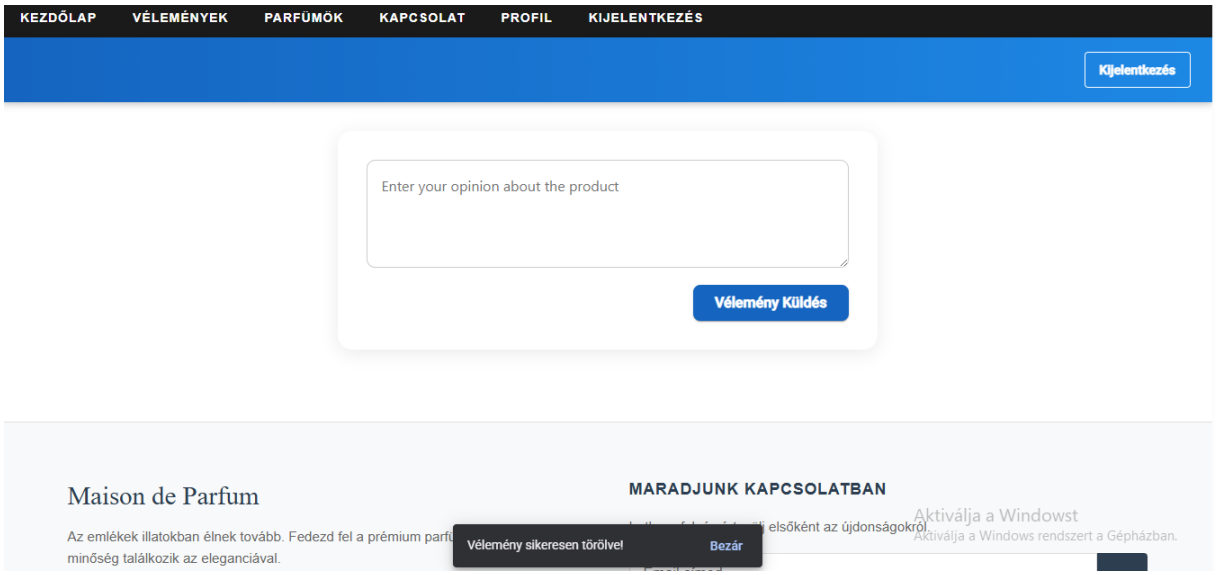
A vélemények az oldalon egy ilyen boxban jelennek meg egymás alatt. Látod a felhasználónevet, és az ahhoz tartozó véleményt. Mindenki csak a saját azonosítójához tartozó véleményt kezelheti!



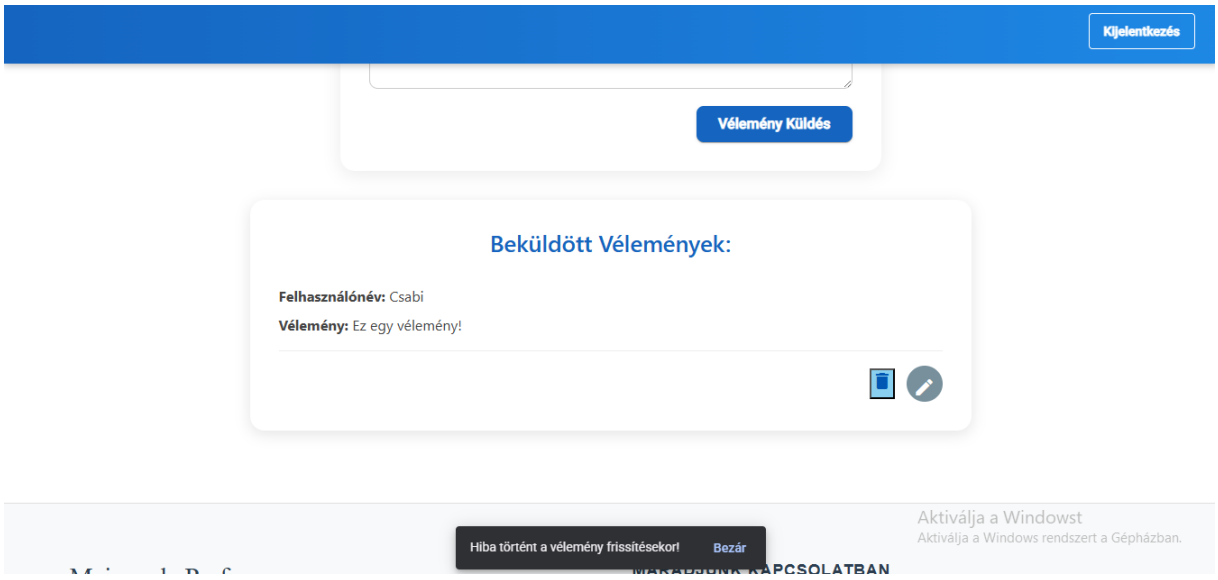
Az előző képen található jobb alsó sarokban lévő „ceruza ikonra” rákattintva jelenik meg ez a fül. Ez a vélemény szerkesztése. Itt először az a vélemény jelenik meg neked, amit beírtál és ezt nyugodtan arra írhatod át amire csak szeretnéd.



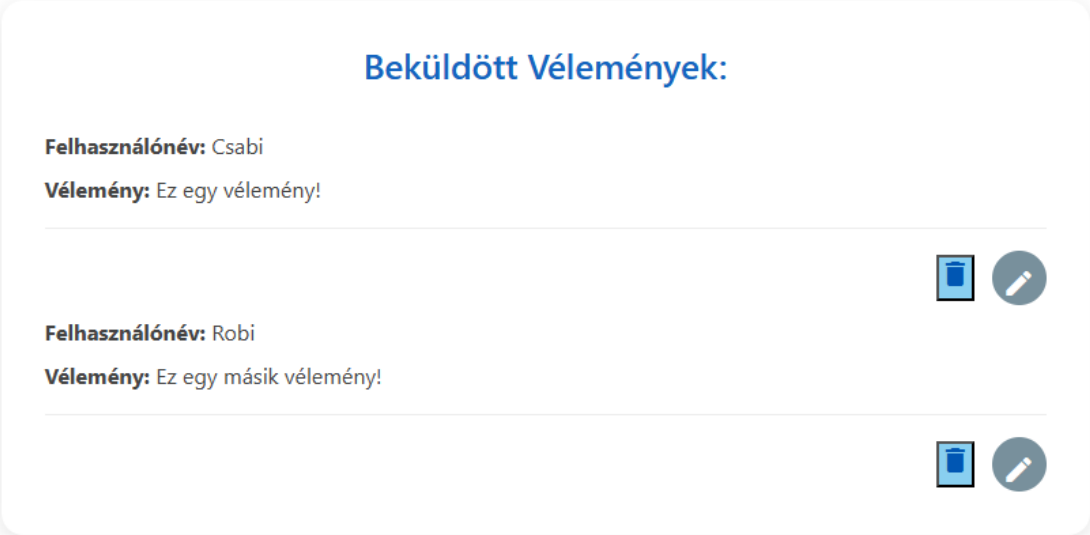
Az „OK” gombra kattintva, bezáródik az ablak, és kapsz egy szöveget mely szerint a véleményed sikeres frissült! Mi történt? A gomb megnyomásával az adatbázisban a véleményedhez tartozó szöveg arra frissült, amit beírtál és ennek eredményeképpen a frontenden egyből megváltozott a véleményed arra, amire frissítetted. A véleménykezelés dinamikusan működik az oldalon!



A „ceruza” ikon mellett található egy „kuka” ikon is. Ezzel tudod a véleményedet törölni az oldalról! A kuka gombra kattintva a véleményed egyből törlődni fog az adatbázisból, így a frontenden sem fogod már látni. Ez a művelet már nem visszavonható, azaz nem tudod meg nem történtté tenni!



Itt az van illusztrálva amikor nem a „Csabi” felhasználónévvel rendelkező user van bejelentkezve, hanem valaki más. Megpróbálja frissíteni azt a véleményt ami Csabihoz tartozik, majd az oldal közli vele, hogy hiba történt. Mi történt? Minden vélemény elvan látva egy hozzá tartozó azonosítóval. Tegyük fel Csabi az 1-es azonosítójú felhasználó. Beküld egy vélemény ennek az azonosítója szintén 1-es lesz. Belép egy Robi nevű felhasználó, aki Csabi véleményét szeretné módosítani, de ő már a 2-es azonosítóval rendelkezik, így ezt nem teheti meg. Mindenki csak a saját véleményét kezeéheti, frissítheti, vagy törölheti!



Ha több vélemény is van az oldalon, akkor azok a boxban így fognak egymás alatt megjelenni. Ez a megjelenési mód könnyen átláthatóvá teszi a felületen megjelenő véleményeket.

Rendszerkövetelmények

A Maison De Parfum egy egyszeű webshop, így a futtatásához nincsen szükség speciális helyi harvereszközökre. Az alábbiakban felsotoljuk a lehetséges variációkra való hardveres konfigurációkat:

Hardverkövetelmények

Alapfeltételek: a projekt Node.js-alapú backend-et (`express`, `sequelize`, `mysql2`) és Angular 19 alapú frontend-et tartalmaz. A backend MySQL-t használ, így a szerver memóriája és I/O teljesítménye fontos.

Fejlesztői gép (lokális - Minimum / Ajánlott)

Minimum

- CPU: 2 mag (pl. i3 vagy hasonló)
- RAM: 8 GB
- Tároló: 256 GB SSD
- OS: Windows 10/11 vagy Linux (WSL2 használható)
- Megjegyzés: kényelmesen fut `ng serve` + Node szerver, de egyszerre futó böngésző, IDE memóriát eszik.

Ajánlott

- CPU: 4 mag (i5 vagy jobb)
- RAM: 16 GB
- Tároló: 512 GB NVMe SSD
- Node: LTS verzió (ajánlott Node 18+), npm/yarn
- Előny: gyors build-ek (`ng build`) és több párhuzamos fejlesztői folyamat.

Teszt

- CPU: 2–4 mag
- RAM: 4–8 GB
- Tároló: 50–100 GB SSD
- DB: MySQL konténer vagy dedikált db instance (20–50 GB adat induláskor)

Éles üzem (kisebb forgalom - pl. hobby / prototípus)

Minimum

- CPU: 2 vCPU
- RAM: 4 GB
- Tároló: 50 GB SSD (adattároláshoz külön kötet: 20–50 GB)
- Hálózat: 1 Gbps hálózat javasolt
- További: Nginx reverse proxy, process manager (PM2 / systemd)

Ajánlott

- CPU: 4 vCPU
- RAM: 8–16 GB
- Tároló: 100+ GB NVMe SSD (sebesség miatt)
- DB: dedikált MySQL (pl. 2 vCPU, 4–8 GB RAM) vagy felügyelt DB-szolgáltatás

Éles üzem (közepes / növekvő forgalom)

- Frontend + API réteg: 4–8 vCPU, 8–16 GB RAM (skálázható, load balancer)
- DB: 4 vCPU, 16 GB RAM (InnoDB buffer pool méretezés)
- Tárhely: NVMe SSD, RAID vagy felhőbeli IOPS garancia
- Javaslat: horizontális skálázás backend rétegen, DB replikáció olvasási terheléshez

Nagy rendelkezésre állás / magas terhelés

- Több backend node + load balancer (Nginx/HAProxy)
- Replikált MySQL (master + read replicas), automatikus mentések
- Separate cache réteg (Redis) ~ memcached opció
- Monitoring és autoscaling konfiguráció

Részletes technikai javaslatok

- Tároló: NVMe/SSD kötelező éles környezetben I/O miatt. Logok és DB külön kötetre.
- RAM a MySQL-hez: InnoDB esetén állítsa be az nnodb_buffer_pool_size-t a memóriának megfelelően (pl. 8 GB RAM esetén ~4–6 GB).
- CPU: Node/Express CPU-bound feladatokat ritkán igényel, viszont sok egyidejű kérésnél több mag segít.
- Swap: legyen konfigurálva, de RAM hiánya esetén teljesítményromlás.
- Biztonság / üzemeltetés: Nginx reverse proxy + TLS, logrotate, rendszeres DB backup (naponta), monitoring (Prometheus/Grafana vagy felhős megoldás).
- Build folyamat: Angular buildnél nagy memóriát igényelhet a `ng build` — ajánlott a build szerveren 8–16 GB RAM.

Méretezési durva irányszámok (példa)

- ~100 egyidejű felhasználó (olvasás-domináns): 2–4 vCPU, 4–8 GB RAM
- ~1000 egyidejű felhasználó: 4–8 vCPU, 8–16 GB RAM, DB skálázás (read replicas)
- Adatmennyiség növekedése esetén: külön log/backup/archív tárolás, napi mentés

Rövid teendők / javaslatok a bevezetéshez

1. Fejlesztéshez: 16 GB RAM ajánlott (snappy dev experience).
2. Éles prototípus: 4 vCPU, 8 GB RAM + dedikált MySQL (4 GB RAM) és NVMe SSD.
3. Monitoring, automatizált mentés és TLS bevezetése azonnal az élesítés előtt.

Szoftverkövetelmények

- Operációs rendszer
  - Minimum: Windows 10 / Ubuntu 20.04 / macOS 11
  - Ajánlott: Windows 11 / Ubuntu 22.04 LTS / macOS 12+
- Böngészők (támogatott)

- Google Chrome (legfrissebb 2 verzió)
- Microsoft Edge (legfrissebb 2 verzió)
- Mozilla Firefox (legfrissebb 2 verzió)
- Safari (macOS / iOS): legfrissebb verziók (tesztelés szükséges Angular Material komponensekhez)

- Fejlesztői eszközök / futtatókörnyezet
  - Node.js: LTS (18+ vagy 20+ ajánlott)
  - npm: 8+ vagy Yarn
  - Angular CLI: 19.x
  - TypeScript: 5.6.x
  - Git, VS Code (ajánlott)
- Adatbázis és konténerek
  - MySQL 8.x (InnoDB, utf8mb4)
  - Docker (opcionális): MySQL konténer fejlesztéshez/staginghez ajánlott
- Futtatás / üzemeltetés
  - Process manager: PM2 vagy systemd
  - Reverse proxy: Nginx (TLS termináció)
  - HTTPS (TLS) kötelező éles környezetben

3.Fejlesztői Dokumentáció

Alkalmazott fejlesztői eszközök

A projekt elkészítése során különböző eszközöket alkalmaztunk a frontend, backend fejlesztéshez, dokumentáció elkészítéséhez, milyen szoftvereket használtunk a csapatmunkához, és a szoftverünk verziószám követéséhez, illetve miket alkalmaztunk a tesztelések végrehajtására. Az alábbiakban bemutatjuk ezeket az eszközöket

Backend fejlesztéshez használt eszközök:

- Visual Studio Code (VSC): fejlesztési környzete, ami segített a kódok megírásában, és kezelésében.
- XAMPP: Biztosította a helyi adatbázist, tartalmazza az Apache szervert, és a MySQL adatbáziskezelőt.
- JavaScript (JS): a szerveroldali logika megvalósításához használt fő programozási nyelv.
- MySQL: Az adatok tárolásához használt relációs adatbázis-kezelő rendszer.
- Express.js: A Node.js alapú szerveroldali alkalmazásfejlesztéshez használt keretrendszer.
- Node.js: Lehetővé teszi a JavaScript futtatását a szerveroldalon.

Frontend fejlesztéshez használt eszközök:

- Visual Studio Code (VSC): A kód fejlesztő környezete.
- Angular: A kliensoldali alkalmazás kialakításához használt keretrendszer.
- Bootstrap5: Reszponzív megjelenítéshez, és előre kész komponensek alkalmazásához.
- CSS: Az alkalmazás stílusának kialakításához.
- TypeScript: Az Angular fejlesztéséhez használt programozási nyelv.
- HTML: AZ alkalmazás szerkezetének megvalósítására.
- Opera: A tesztelés, és fejlesztés során használt böngésző.
- Angular Material: Az alkalmazásban található ikonok kialakításához.

Teszteléshez használt eszközök:

- Postman: API-k teszteléséhez, request-ek, response-ok validálásához.
- Jasmine: Egység tesztelés Angular projektekhez.
- Karma: Egység tesztelés automatizálása.

Dokumentáció készítéséhez használt eszközök:

- Microsoft Word: A fejlesztői, felhasználói dokumentáció, illetve egyéb szöveges anyagok szerkesztése és formázása.

Csapatmunkát segítő szoftverek:

- Github: A forráskód verzióihoz való hozzáférés.
- Trello: A projekt tervének, feladatok elkészítéséhez használt felvázolása.

A Projekt telepítése

A dokumentáció ezen részén bemutatjuk, hogyan tudja lokálisan saját számítógépére letelepíteni az alkalmazást.

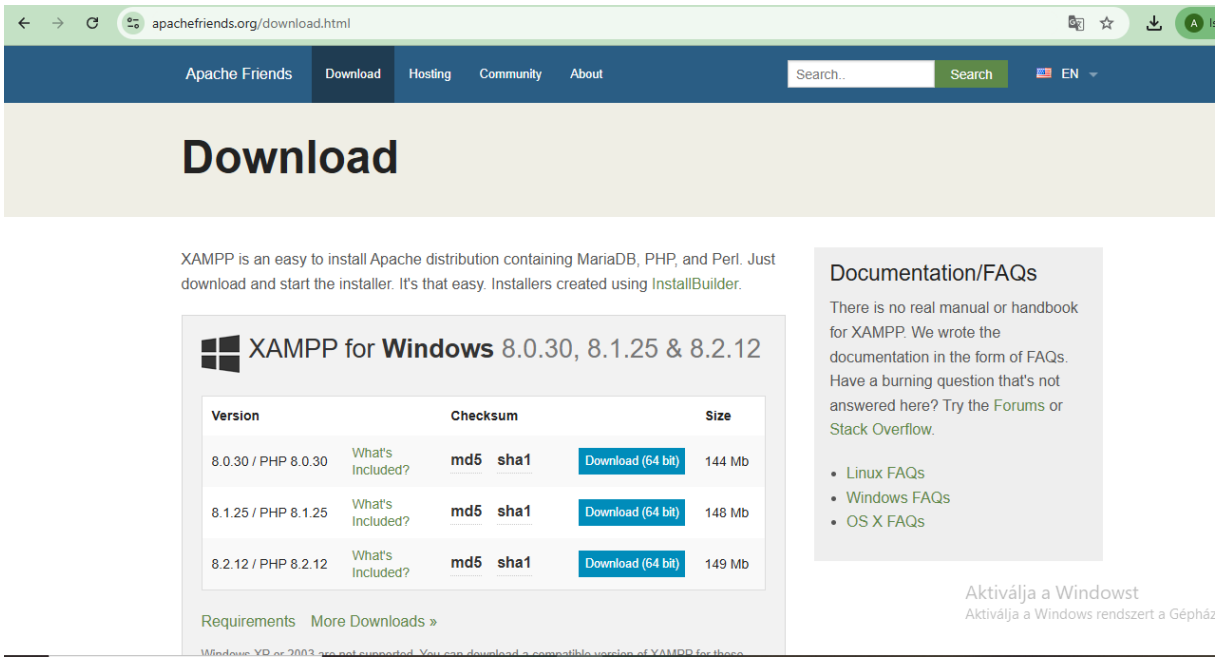
Kövesse az alábbi lépéseket ha saját számítógépén szeretné futtatni az alkalamzást!

Szükséges szoftverek és verziók

- Visual Studio Code (ajánlott verzió 1.100+)
- XAMPP Control Panel (ajánlott verzió: 8.2+)
- Böngésző (ajánlott: Opera, Google Chrome Mozilla Firefox, Microsoft Edge – frissített verzió)
- Node.js (ajánlott verzió: LTS)

Mielőtt bármit letöltene, vegye figyelembe az operációs rendszerének típusát! A parancssort megnyitva az alábbi kódok segítségével tudja lekérdezni:

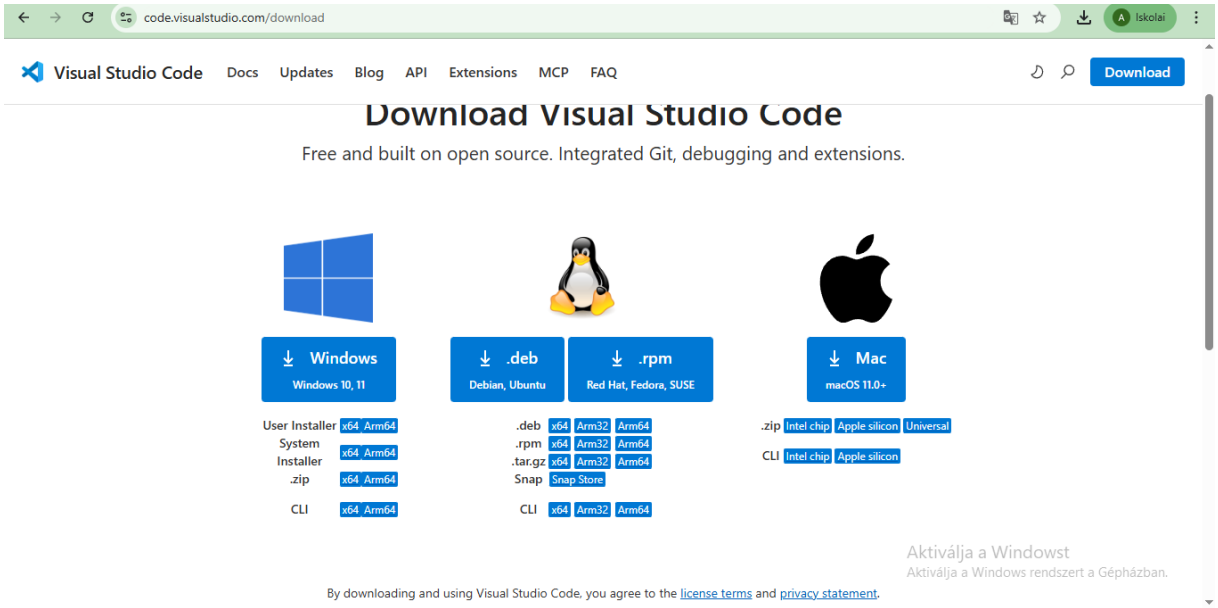
-Windows: wmic OS get OSArchitecture
-Linux: uname –m
-MacOS: uname –m



XAMPP Telepítési Útmutató

Kövesse az alábbi egyszerű lépéseket a környezet beállításához:

1. **Weboldal megnyitása:** Kattintson az alábbi linkre: https://www.apachefriends.org/download.html
2. **Verzió kiválasztása:** A számára megfelelő operációs rendszer alapján töltse le a legfrissebb verziót.
3. **Telepítő indítása:** Miután letöltődött a fájl, kattintson rá duplán az indításhoz.
   - *Megjegyzés:* Ha a Windows rákérdez, engedélyezze a futtatást ("Igen"). Ha kap egy figyelmeztetést (UAC), csak nyomjon az OK gombra.
4. **Telepítési folyamat:**
   - **Összetevők:** Hagyjon mindent kipipálva, és kattintson a *Next* gombra.
   - **Helyszín (Installation folder):** Javasoljuk, hogy hagyja az alapértelmezett `C:\xampp` mappát. **Ne** változtassa meg, hacsak nem profi felhasználó!
   - **Nyelv:** Válassza az *English* (angol) nyelvet (magyar sajnos nem elérhető), majd kattintson a *Next* gombokra a befejezésig.
5. **Befejezés:** A telepítés végén kattintson a *Finish* gombra. Ekkor megnyílik a XAMPP Control Panel, ahol az **Apache** és **MySQL** melletti **Start** gombokkal indíthatja el a szervert.



Visual Studio Code Telepítési Útmutató

Kövesse az alábbi lépéseket a program telepítéséhez:

1. **Weboldal megnyitása:** Kattintson az alábbi linkre: https://code.visualstudio.com/download

2. Operációs rendszer kiválasztása:

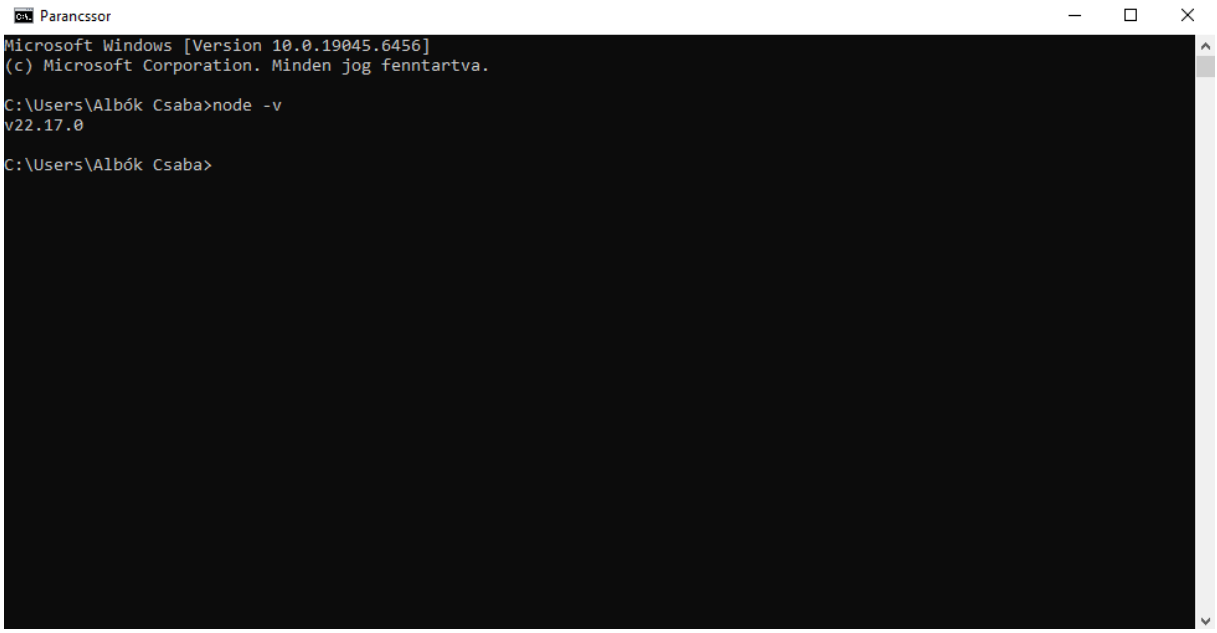
2. **Windows:** Kattintson a nagy kék **Windows** gombra (Windows 10, 11 felirattal).
3. **macOS:** Kattintson a nagy kék **Mac** gombra (macOS 11.0+ felirattal).
4. **Linux:** Válassza ki a rendszere szerinti formátumot a középső blokkból:
    - Kattintson a **.deb** gombra, ha Ubuntu, Debian, vagy Linux Mint rendszert használ.
    - Kattintson a **.rpm** gombra, ha Fedora, Red Hat, vagy SUSE rendszert használ.
5. **Telepítő indítása:** Miután a letöltés befejeződött, nyissa meg a fájlt.
6. **Telepítési folyamat (Windows esetén):**
    - **Szerződés:** Fogadja el a licencszerződést (*I accept the agreement*), majd kattintson a *Next* gombra.
    - **Telepítési hely:** Hagyja az alapértelmezett mappát, ne módosítsa.
    - **Extra beállítások:** Érdemes kipipálni az **"Add to PATH"** opciót és az **"Add 'Open with Code' action..."** lehetőségeket. Ez megkönnyíti a későbbi munkát. Kattintson a *Next*, majd az *Install* gombra.
7. **Nyelv beállítása:** A program alapértelmezés szerint angol.
    - Ha szeretné **magyarra** állítani: Indítás után a bal oldali sávban kattintson az **Extensions** ikonra (négyzet alakú ikon), írja be a keresőbe: `Hungarian`, és kattintson az *Install* gombra a magyar nyelvi csomagnál. Ezután indítsa újra a programot.
8. **Befejezés:** Kattintson a *Finish* gombra. A program készen áll a használatra!

Node.js Telepítési Útmutató

Kövesse az alábbi lépéseket a Node.js környezet beállításához:

1. **Weboldal megnyitása:** Kattintson az alábbi linkre: https://nodejs.org/en/download
2. **A megfelelő verzió kiválasztása:**
    - A weboldal tetején válassza az **LTS** fület (ez a stabil, hosszú távon támogatott verzió).

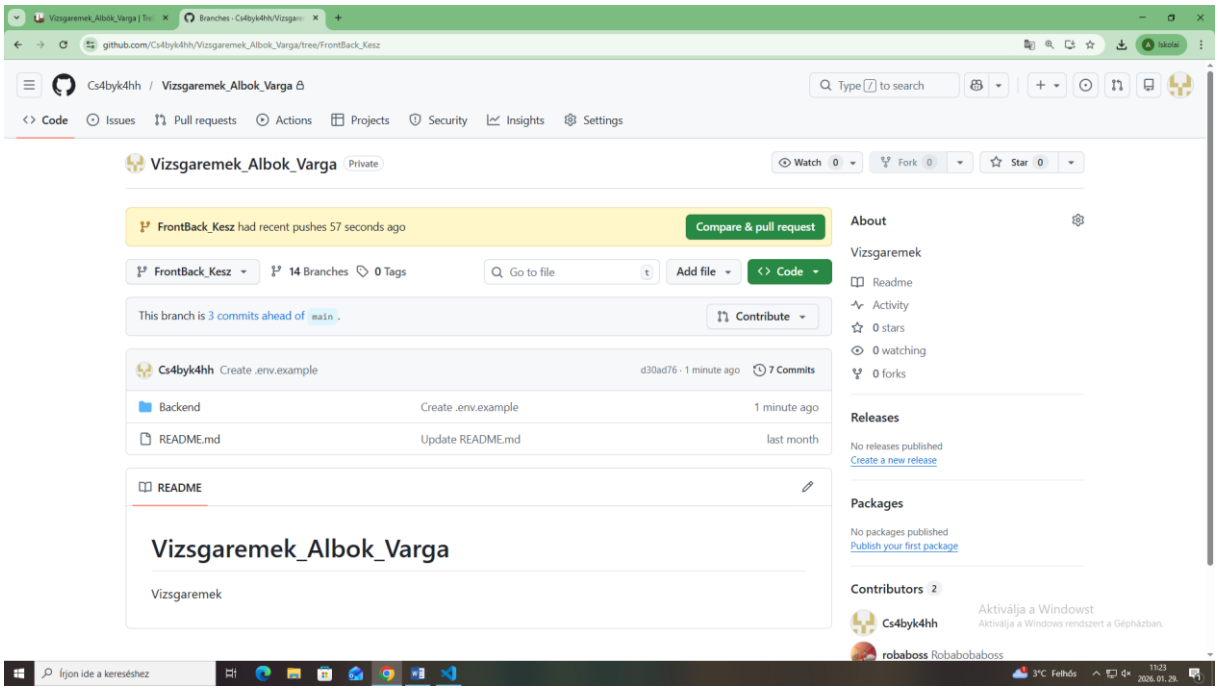
    - Győződjön meg róla, hogy az Ön operációs rendszere (Windows, macOS vagy Linux) van kiválasztva a legördülő menüben.
    - A telepítési módnál ("using") válassza az **Installer** (Telepítő) opciót a képernyőn látható *Docker* vagy *FnM* helyett, hogy egy egyszerűen futtatható `.msi` vagy `.pkg` fájlt kapjon.
3. **Telepítő indítása:** Miután a letöltés befejeződött, nyissa meg a fájlt.
4. **Telepítési folyamat:**
    - Kattintson a *Next* (Tovább) gombokra, és fogadja el a felhasználási feltételeket.
    - **Telepítési hely:** Hagyja az alapértelmezett mappát (`C:\Program Files\nodejs\`).
    - **Custom Setup:** Itt ne módosítson semmit, minden szükséges eszköz (pl. npm csomagkezelő) alapból ki van választva.
    - **Tools for Native Modules:** Megjelenhet egy ablak, ami felajánlja a kiegészítő eszközök (pl. Chocolatey) automatikus telepítését. Kezdőként ezt **nem kötelező** bepipálni, de ha később komolyabb fejlesztésre készül, hasznos lehet.
5. **Befejezés:** Kattintson az *Install*, majd a folyamat végén a *Finish* gombra.
6. **Ellenőrzés:** Ahhoz, hogy biztos legyen a sikerben, nyisson meg egy parancssort (írja be a Windows keresőbe: `cmd`), és gépelje be: `node -v`. Ha megjelenik egy verziószám, a telepítés sikeres volt!



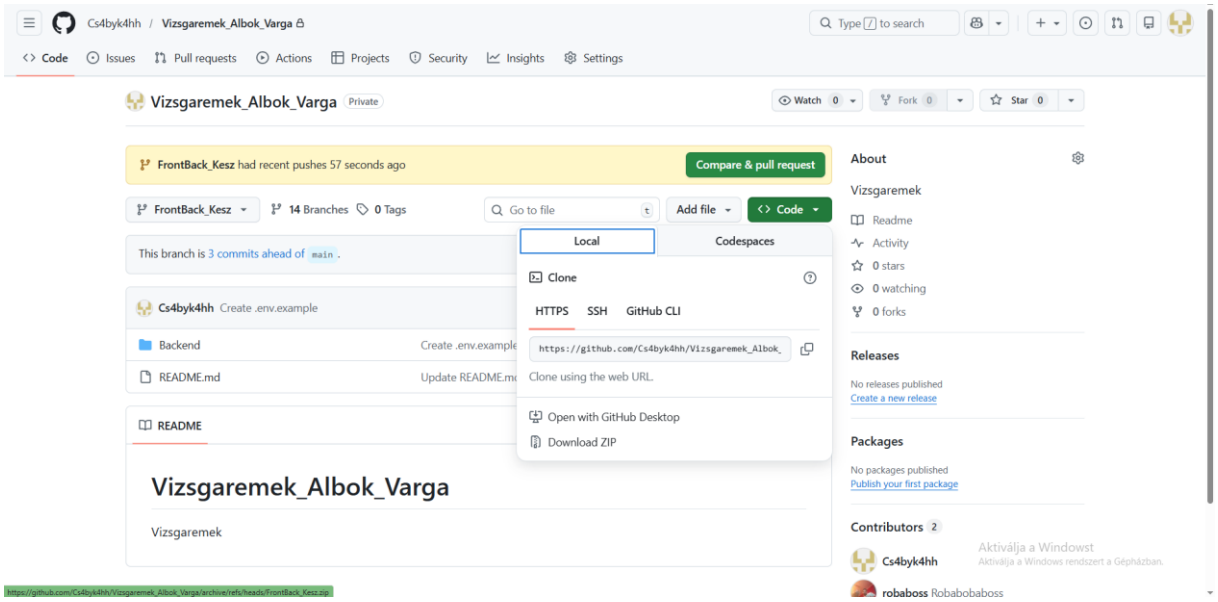
Alkalmazás futtatása a számítógépen

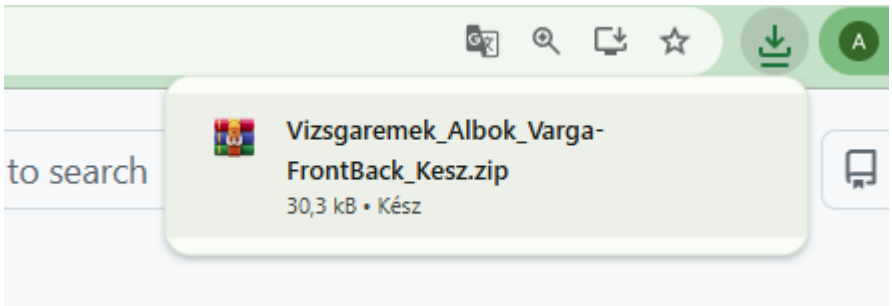
Ha minden szükséges szoftver rá lett telepítve a számítógépre, most bemutatjuk, hogy érje el, hogy az alkalmazás fusson a gépén!

https://github.com/Cs4byk4hh/Vizsgaremek_Albok_Varga.git -> A böngészőbe bemásolja ezt a linket látni fogja a Github repositorynkat.

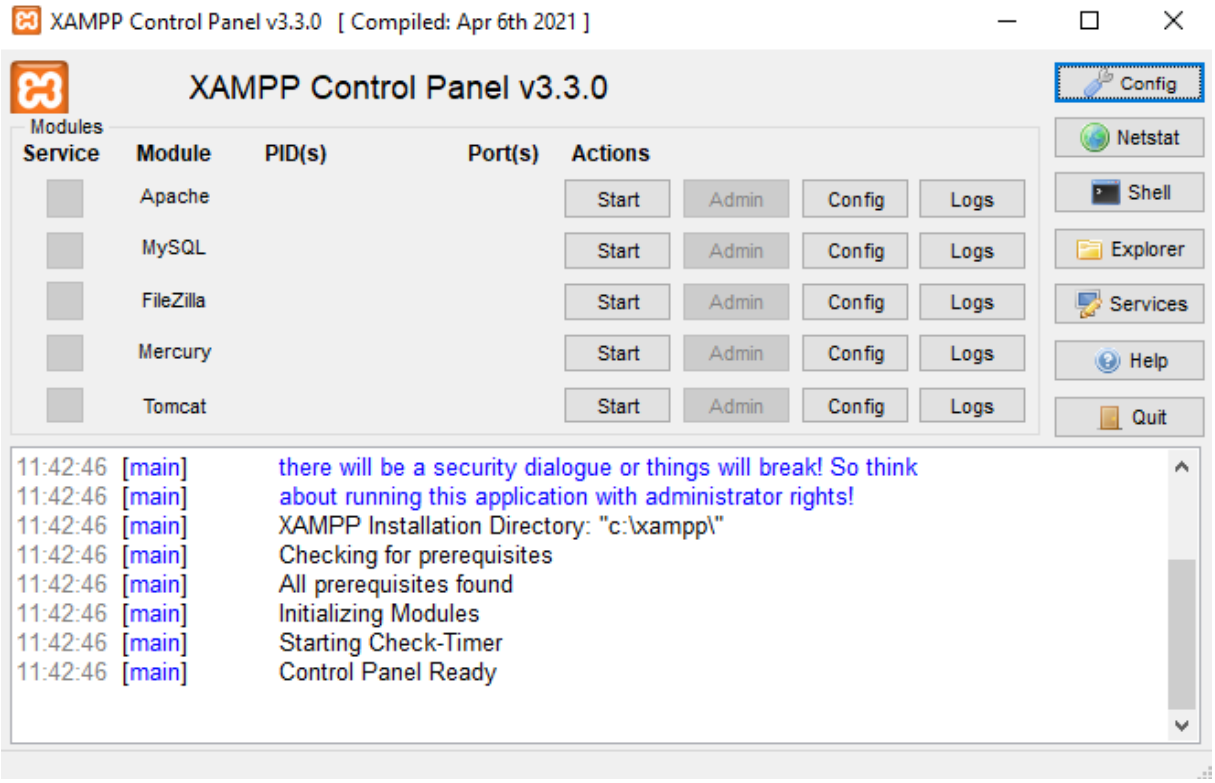


Ezt fogja maga előtt látni. Ahhoz hogy letöltse a projektet nyomjon a <> Code legördülő gombra, és válassza ki a Download ZIP lehetőséget!

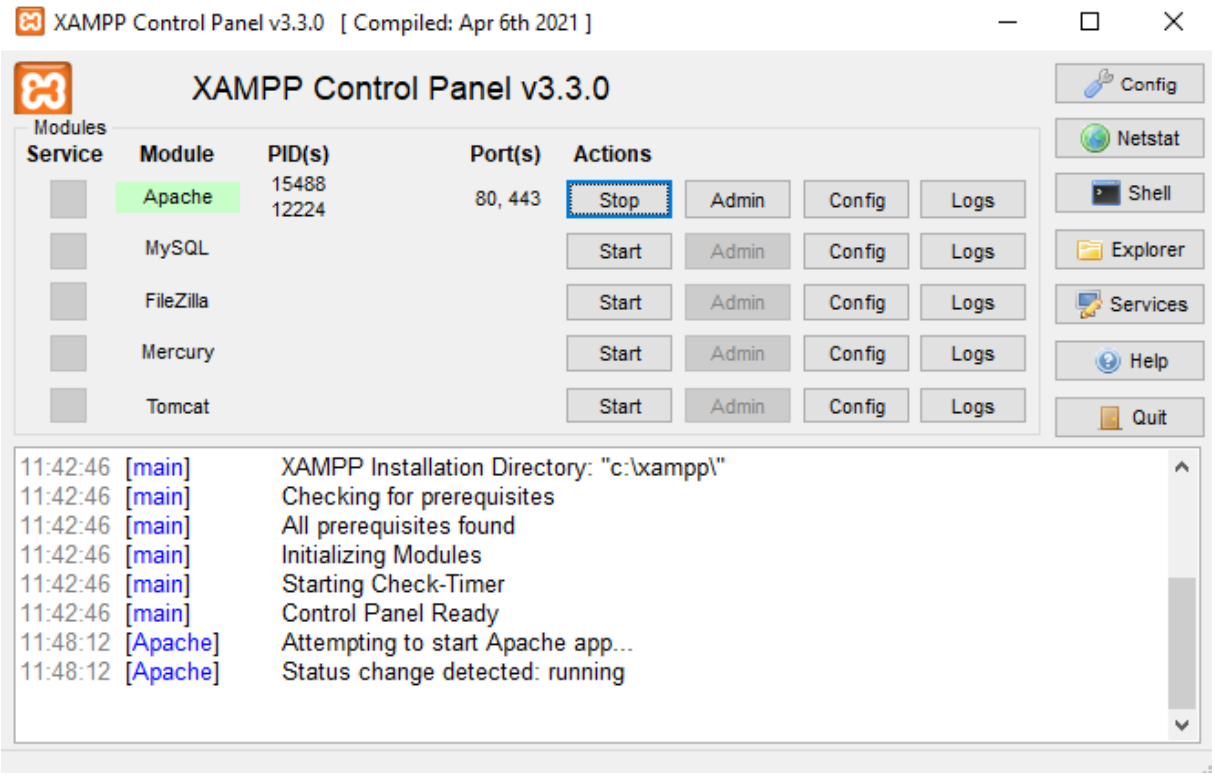




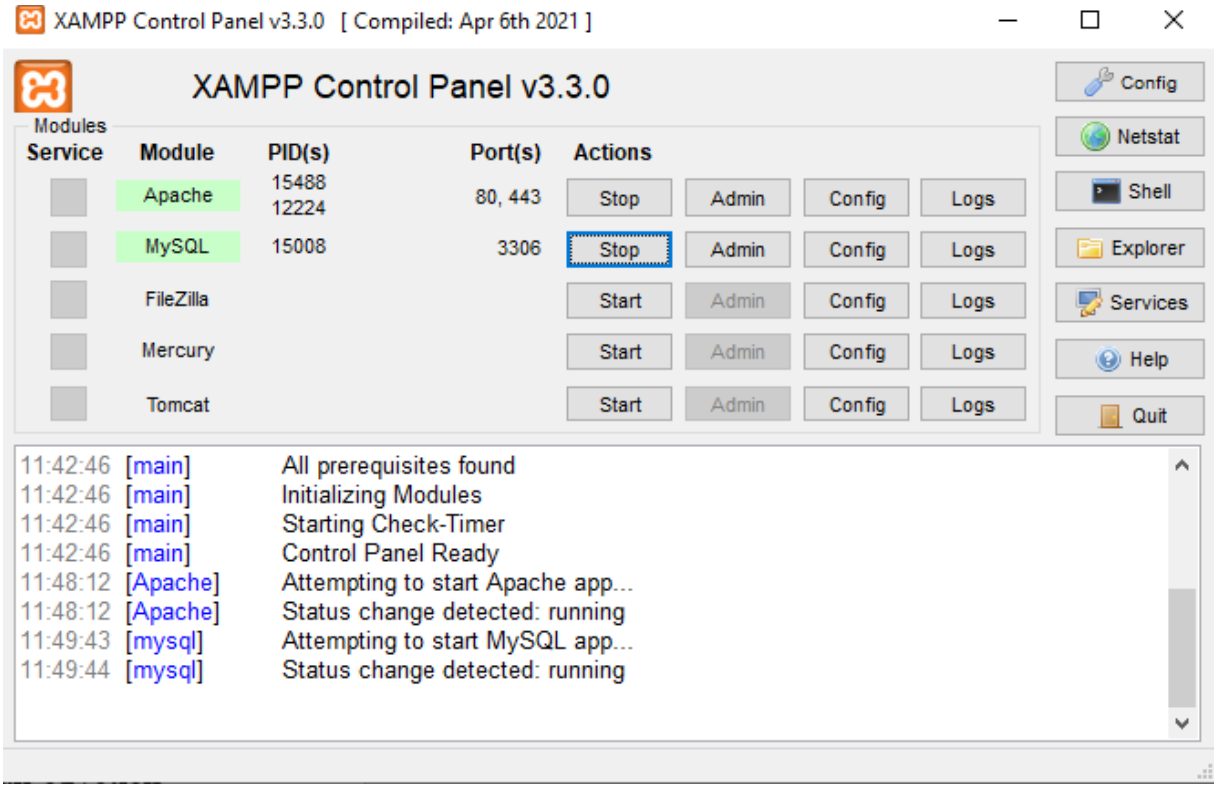
A Download ZIP-re kattintva a jobb felső sarokban ez fog megjelleni. Erre az egér bal gombjával kattintson rá. Utána a mappát húzza ki az asztalra, vagy ha úgy kényelmesebb tegye bele egy új ön által létrehozott mappába.



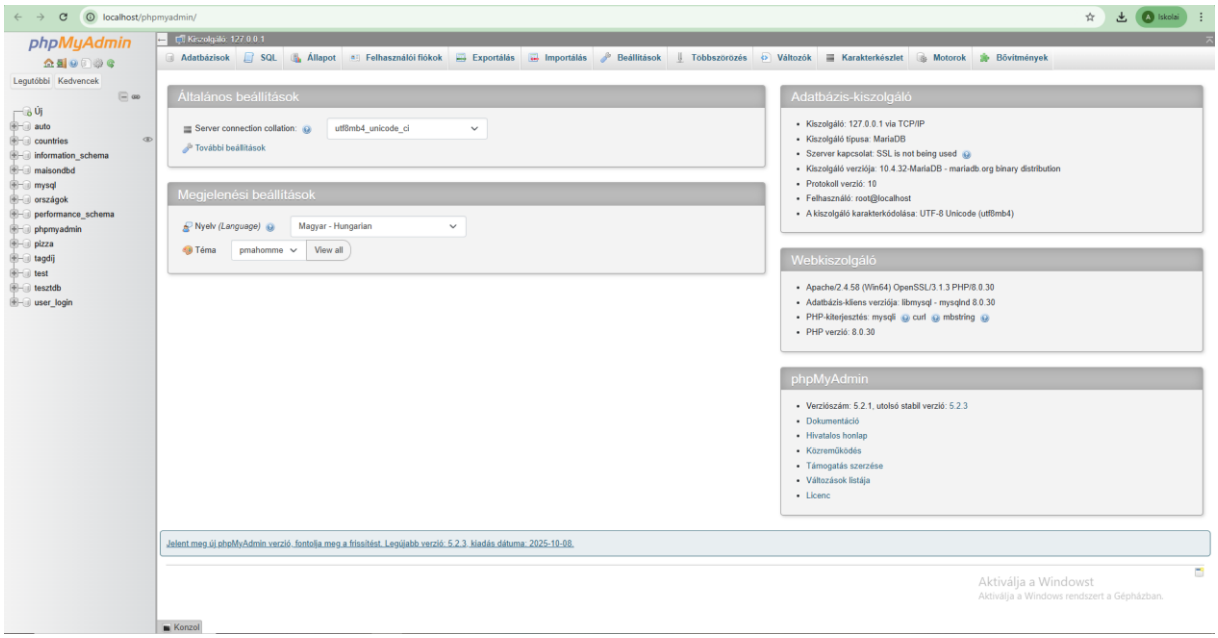
Ezután nyissa meg a XAMPP-ot! Ezt fogja látni maga előtt. Az Apache mellett lévő Start gombot nyomja meg. Ennek hatására az Admin gombot már meglehet nyomni, de itt nem kell! Várja meg, még az Apache háttere sárgáról, zöldre vált, így indul el az Apache.



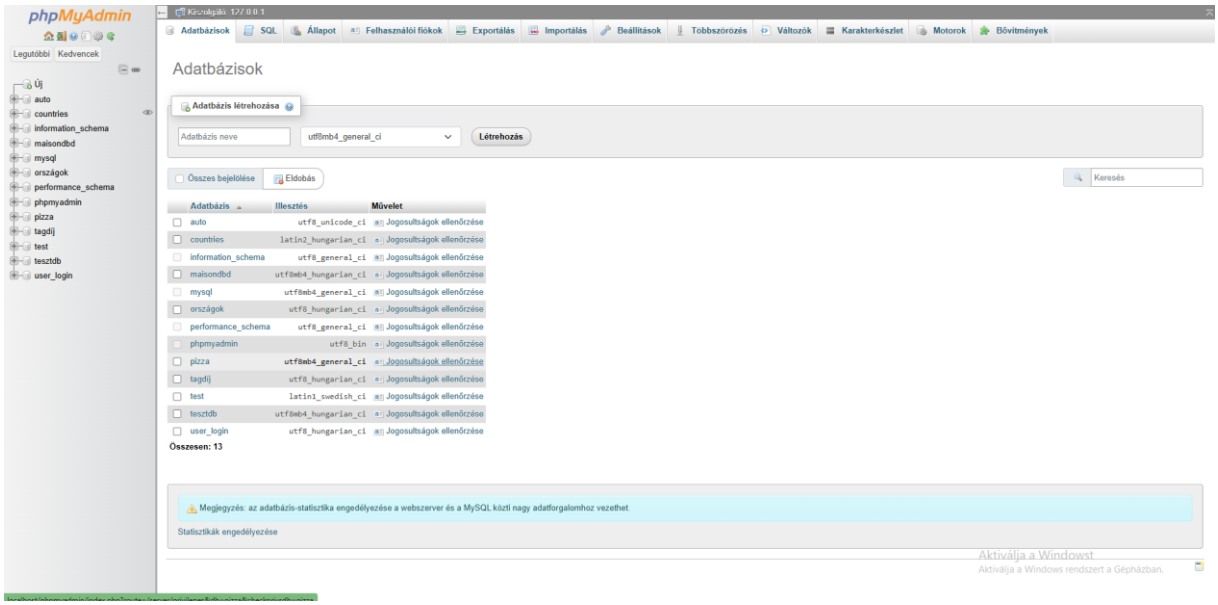
Ezután nyomja meg a MySQL mellett lévő Start gombot, és várja meg még a MySQL háttere zöldre vált!



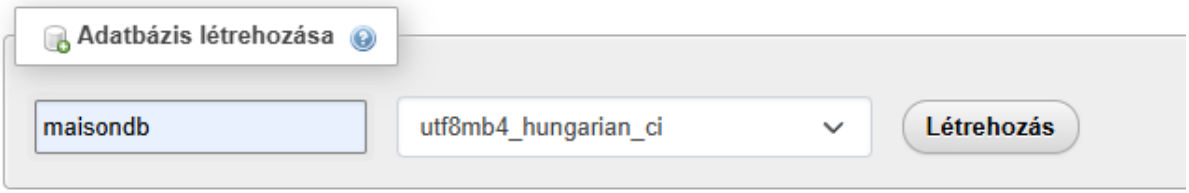
Ha ezt látja maga előtt, nyomja meg a MySQL-nél lévő Admin gombot!



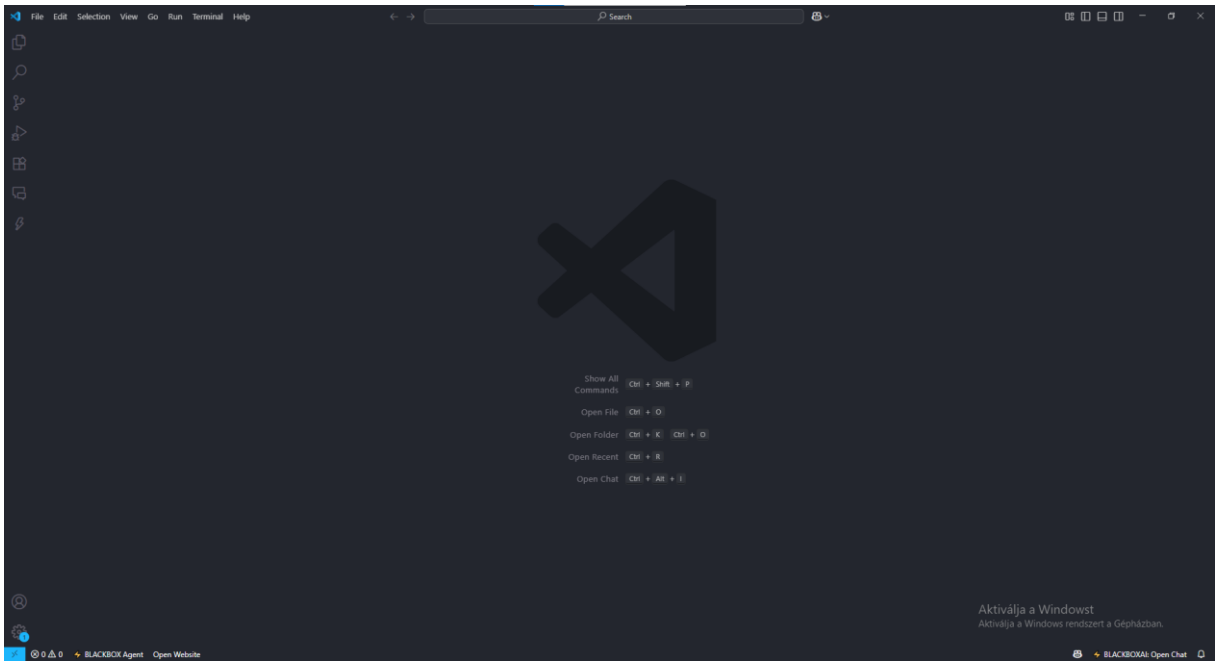
Ha mindent jól csinált ezt látja maga előtt. a phpMayAdmin felirat alatt látható egy „Új” feliratú gomb. Nyomja meg!



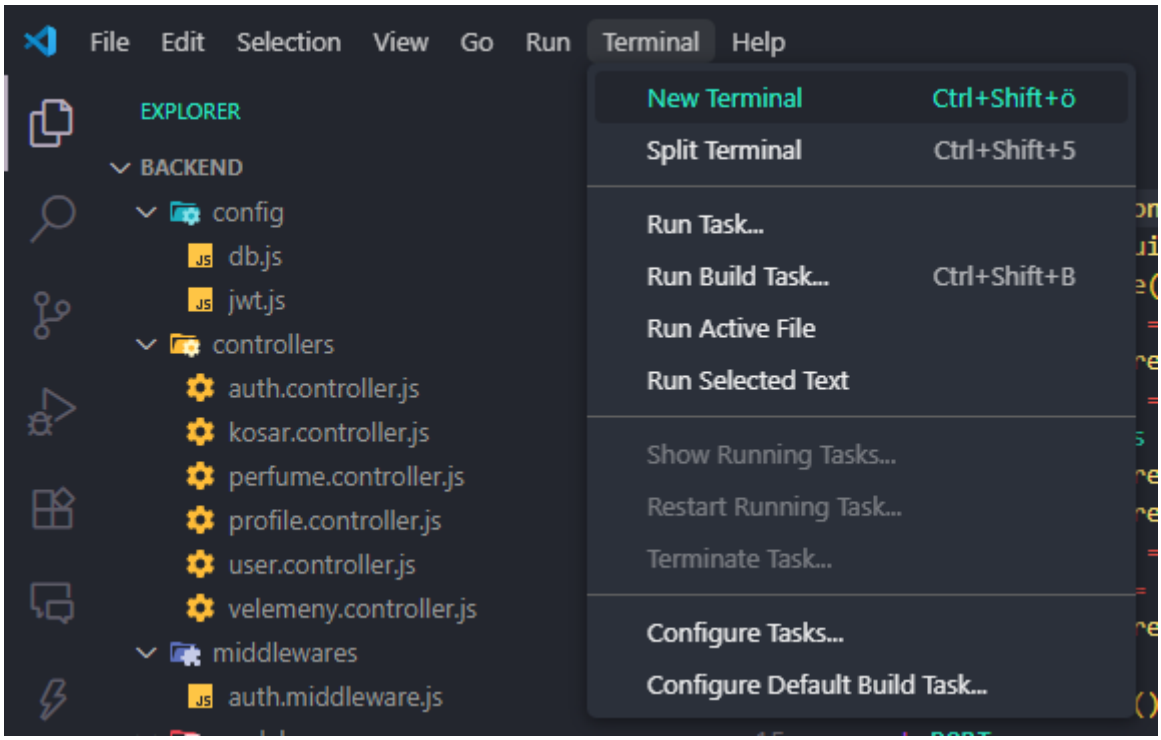
Most ezt látja maga előtt. Adatbázisnévnek írja be, hogy **maisondb,** a mellette lévő legördülő menübe, válassza ki az **utf8mb4_hungarian_ci**-t!



Nyomja meg a létrehozás gombot, és az adatbázis elkészült!



Most nyissa meg a Visual Studio Code-ot. A következő billenytűparancs megnyomása után válassza ki azt a mappát, amelyben elhelyezte a projektet! **Crtl + k + o.** Ezt a billentyűparancsot egyszerre nyomja le, és válassza ki a mappát!



A „New Terminal”-ra kattintva nyisd meg a terminál fület!

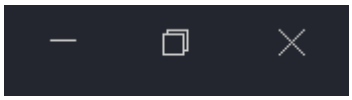
A terminálban ezt a parancsot futtasd le! Ha esetleg hibát dob ellenőrizd, hogy a backend mappában vagy e. Ha nem, ugyanide az **npm i** előtt írd be, hogy **cd Backend.**

```
Run `npm audit` for details.
PS C:\Users\Albók Csaba\Desktop\Backend> node server.js
```

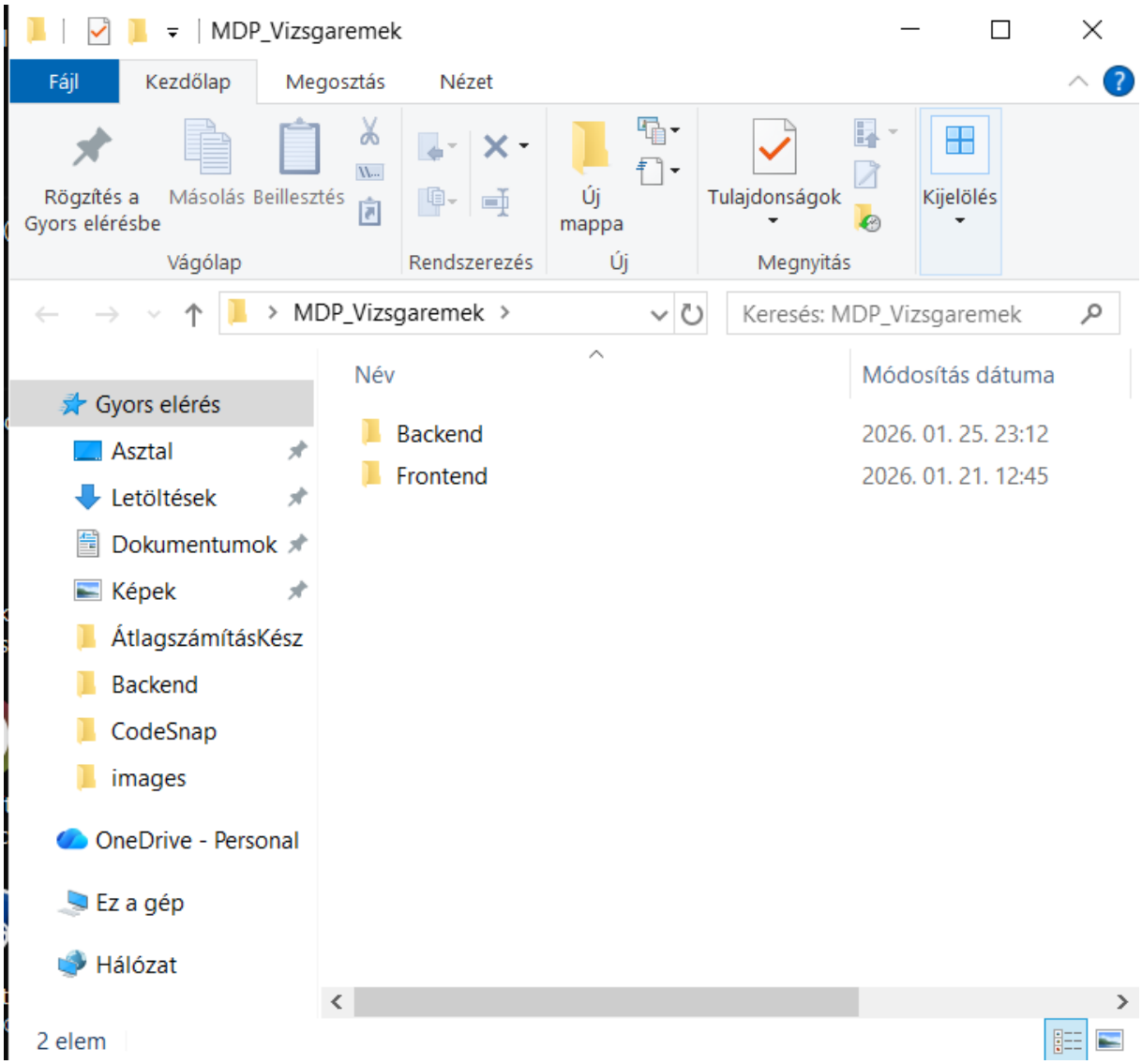
Ha lement **az npm i** parancs, utána írd be ezt, ezzel elindítod a backend szervert!

```
PS C:\Users\Albók Csaba\Desktop\Backend>
node server.js
[dotenv@17.2.3] injecting env (6) from .env -- tip: write to custom object with { processEnv: myObject }
[dotenv@17.2.3] injecting env (0) from .env -- tip: encrypt with Dotenvx: https://dotenvx.com
[dotenv@17.2.3] injecting env (0) from .env -- tip: enable debug logging with { debug: true }
Server is running on http://localhost:3000
Adatbázis szinkronizálva!
Parfümök hozzáadva a táblához!
Adminok létrehozva!
Adminok létrehozva!
```

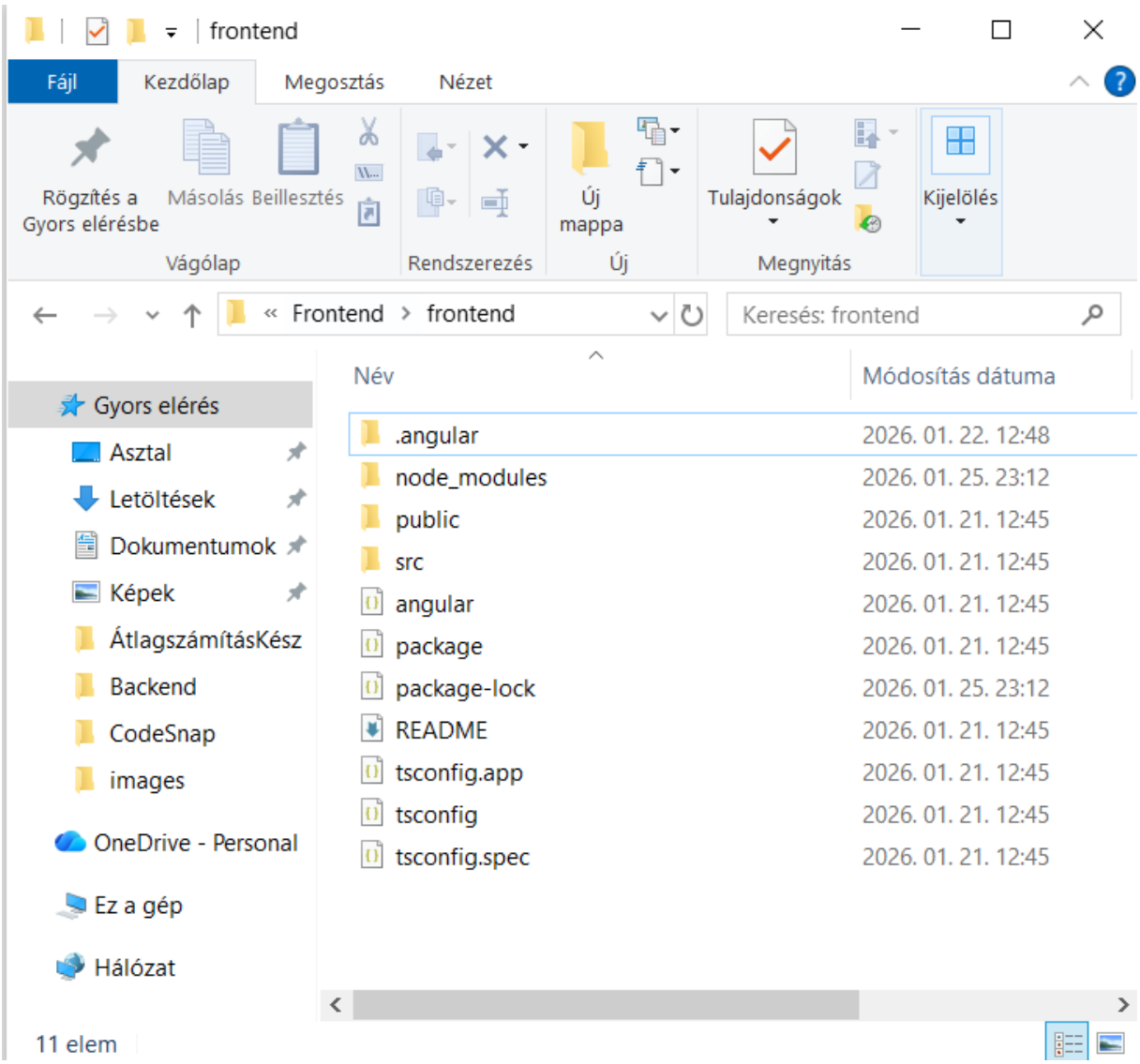
Ez jelenti azt, hogy a backend fut és létrejött a kapcsolat az adatbázissal, illetve a szükséges adatokat betöltötte az adatbázisba, mivel ez volt az első indítás!

Ezután a Visual Studio Code-ot ne zárja be! Tegye le kicsibe, nyomja meg  a bal szélen látható – ilyen ikont, a jobb felső sarokban! Ha bezárja a VsC-t akkor a backend szerver leáll, és nem fognak működni a funkciók az oldalon!

Lépjen be a letöltött mappába, itt maga előtt ezt fogja látni.

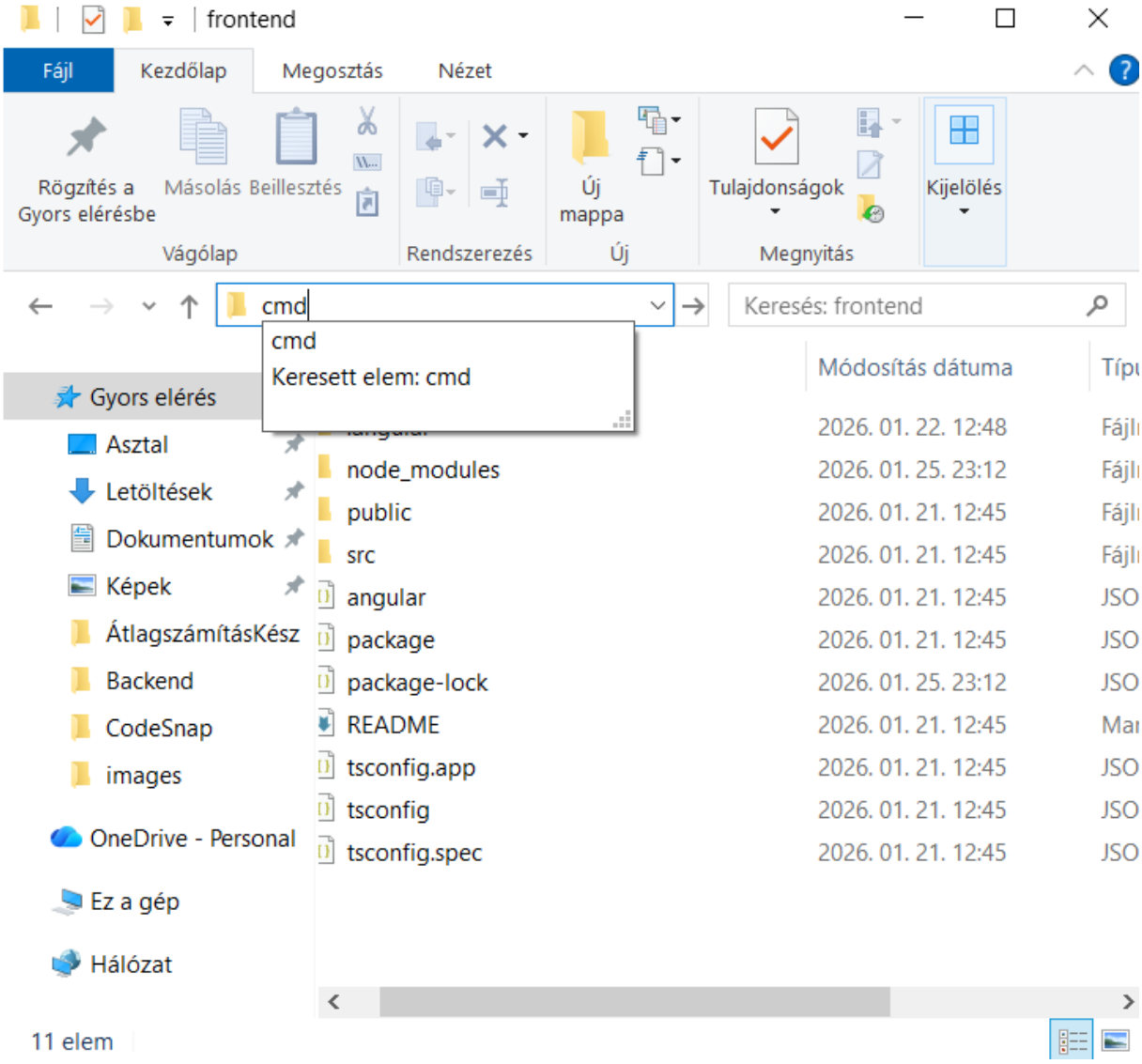


Egérrel kétszer bal klikk a frontend mappára.



Most ezt látja. Ahol látja fent a mappa ikon mellett, hogy << Frontend , ezt jelölje ki és

írja be, hogy cmd.



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. Minden jog fenntartva.

C:\Users\albcs\Desktop\MDP_Vizsgaremek\Frontend\frontend>npm i
```

A cmd-ben írja be az **npm i** parancsot, ezzel letölti a szükséges fűggöséket a projekt sikeres futtatásához!

```
ng serve
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. Minden jog fenntartva.

C:\Users\albcs\Desktop\MDP_Vizsgaremek\Frontend\frontend>ng serve
Component HMR has been enabled.
If you encounter application reload issues, you can manually reload the page to bypass HMR and/or disable this feature w
ith the `--no-hmr` command line option.
Please consider reporting any issues you encounter here: https://github.com/angular/angular-cli/issues

Initial chunk files | Names         |  Raw size
styles.css          | styles        | 284.63 kB |
main.js             | main          | 143.81 kB |
scripts.js          | scripts       | 107.69 kB |
polyfills.js        | polyfills     |  90.20 kB |

                    | Initial total | 626.34 kB

Application bundle generation complete. [4.750 seconds]

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
Re-optimizing dependencies because vite config has changed
Re-optimizing dependencies because vite config has changed (x2)
  ➜  Local:   http://localhost:4200/
  ➜  press h + enter to show help
```

Ha ez megvan és letöltötte a csomagokat, akkor írja be az **ng serve** parancsot és nyomjon entert!

Ha mindent jól csinált ezt fogja látni maga előtt!

Másolja ki a „Local”-nál található linket, majd a böngészőjébe amit használ, nyisson meg egy új ablakot és oda illessze be! Ha mindent a leírások alapján csinált akkor sikeresen működik a webalkalmazás az ön számítógépén!

4.Kliens oldali megvalósítás és Architektúra (Frontend)

A **Maison De Parfum** webáruház felhasználói felületét a **Google Angular** keretrendszerének legfrissebb, **19-es verziójával** valósítottuk meg. A választás azért esett erre a technológiára, mert az Angular egy teljes körű, nagyvállalati környezetben is bizonyított keretrendszer, amely szigorú struktúrát, típusbiztonságot (TypeScript) és kiváló skálázhatóságot kínál.

A fejlesztés során a **Single Page Application (SPA)** architektúrát követtük. Ennek lényege, hogy a szerver nem kész HTML oldalakat küld vissza minden kérésre, hanem csak egyetlen keretet (index.html), és az alkalmazás futása során a JavaScript (Angular) dinamikusan cseréli a tartalmat a böngészőben. Ez drasztikusan csökkenti a hálózati forgalmat és "applikáció-szerű", villámgyors felhasználói élményt nyújt.

Fejlesztési környezet és konfiguráció

A projekt alapjait a angular.json és a package.json fájlok határozzák meg. A fejlesztés során a következő kulcsfontosságú könyvtárakat integráltuk:

- **Core Framework (@angular/core v19.0.0):** A keretrendszer motorja.
- **RxJS (~7.8.0):** A reaktív programozás alapja, amely az aszinkron adatfolyamokat (pl. HTTP válaszok, felhasználói inputok) kezeli.
- **Angular Material (v19.2.19):** A Google hivatalos UI komponenskészlete, amelyet a modális ablakok (MatDialog), értesítések (MatSnackBar) és ikonok (MatIcon) megjelenítésére használtunk.
- **Bootstrap (v5.3.8):** A reszponzív rácsrendszer (Grid System) és a kártyák (card) formázásának alapja.

TypeScript konfiguráció

A projekt szigorú típusellenőrzést használ (strict: true a tsconfig.json-ben). Ez megköveteli, hogy minden változóhoz és visszatérési értékhez típust rendeljünk, ami fordítási időben (compile time) kiszűri a hibák 90%-át.

Példa a típusos definícióra a perfumes.model.ts fájlból:

TypeScript

```
export interface Perfume {
  id: number;
```

```
  name: string;
  brand: string;
  price: string;
  volume_ml: number;
  gender: string;
  season_spring: boolean; // Boolean flag a szűréshez
  season_summer: boolean;
  sweet: boolean;
  // ...
}
```

Ez az interfész garantálja, hogy ha bárhol a kódban egy parfüm objektummal dolgozunk, az biztosan rendelkezni fog ezekkel a tulajdonságokkal.

Adatkommunikációs Réteg (Services)

Az Angular architektúrájának egyik legfontosabb eleme a **Separation of Concerns (SoC)**, azaz a feladatok szétválasztása. A komponensek (Component) csak a megjelenítésért felelnek, míg az üzleti logikát és az adatkommunikációt a szolgáltatásokba (Services) szerveztük ki.

A szolgáltatások @Injectable({ providedIn: 'root' }) dekorátorral vannak ellátva, ami azt jelenti, hogy az alkalmazásban **Singleton**-ként (egyetlen példányban) léteznek.

Hitelesítési Szolgáltatás (AuthService)

Ez a szolgáltatás felelős a felhasználói identitás kezeléséért. A localStorage-ot használjuk a JWT token perzisztens tárolására, így oldalfrissítés után is bejelentkezve marad a felhasználó.

Kód elemzése (auth.service.ts):

TypeScript

```
logout(): void {
  const token = localStorage.getItem('token');
  if (!token) {
    this.router.navigate(['/login']);
    return;
  }

  // HTTP POST kérés a backend logout végpontjára
  this.http.post(`${this.apiUrl}/auth/logout`, {}, {
```

```
    headers: { Authorization: `Bearer ${token}` }
  }).subscribe({
    next: () => {
      this.snackBar.open('Sikeres kijelentkezés!', 'Bezár', { duration: 3000 });
      localStorage.removeItem('token'); // Token törlése a böngészőből
      this.router.navigate(['/login']); // Átirányítás
    },
    error: (error) => {
      console.error('Hiba:', error);
      // Hiba esetén is töröljük a tokent a biztonság kedvéért
      localStorage.removeItem('token');
      this.router.navigate(['/login']);
    }
  });
}
```

Magyarázat: A logout metódus nemcsak kliens oldalon "dobja el" a tokent, hanem egy aszinkron HTTP POST kérést is küld a szervernek. A .subscribe() metódussal iratkozunk fel a válaszra. Ez kritikus biztonsági lépés, mert így a szerver is tudomást szerez a munkamenet végéről (pl. feketelistára teheti a tokent). A MatSnackBar használata elegáns, nem tolakodó visszajelzést ad a felhasználónak.

Reaktív Kosár Kezelés (CartService)

A webáruház legkomplexebb kliens oldali logikája a kosár kezelése. Itt a **State Management** (állapotkezelés) mintát alkalmaztuk a BehaviorSubject segítségével.

Kód elemzése (cart.service.ts):

TypeScript

```
export class CartService {
  private cartItems: Perfume[] = [];
  // BehaviorSubject: Tárolja az aktuális értéket és értesíti a feliratkozókat
  cartItems$ = new BehaviorSubject<Perfume[]>([]);


  addToCart(perfume: Perfume) {
    // Megnézzük, benne van-e már a termék
    const existing = this.cartItems.find(item => item.id === perfume.id);
```

```
    if (existing) {
      // Ha igen, növeljük a mennyiséget (típus kényszerítés any-re a dinamikus mező miatt)
      (existing as any).quantity = ((existing as any).quantity || 1) + 1;
    } else {
      // Ha nincs, hozzáadjuk 1-es mennyiséggel
      (perfume as any).quantity = 1;
      this.cartItems.push(perfume);
    }

    // Értesítjük az összes komponenst a változásról
    this.cartItems$.next([...this.cartItems]);
  }

  getTotal() {
    return this.cartItems.reduce((acc, item: any) =>
      acc + item.price * (item.quantity || 1), 0);
  }
}
```

Szakmai elemzés:

- **BehaviorSubject:** Ez egy speciális Observable típus az RxJS könyvtárból. Két nagy előnye van itt:
  1. Mindig tárolja az *utolsó* állapotot (tehát ha egy komponens később töltődik be, azonnal megkapja a kosár tartalmát).
  2. A .next() metódussal új adatot pumpálhatunk a rendszerbe, amire a komponensek (pl. a fejlécben lévő kosár ikon) azonnal reagálnak ("Reaktív programozás").
- **Immutabilitás:** A [...this.cartItems] szintaxis (Spread operator) egy másolatot készít a tömbről. Ez azért fontos, hogy az Angular változásfigyelő mechanizmusa (Change Detection) biztosan észrevegye, hogy új referencia került a rendszerbe, és frissítse a nézetet.
- **reduce:** A végösszeg számításához a funkcionális programozásból ismert reduce függvényt használtuk, amely elegánsan, egy sorban összegzi a (ár * mennyiség) szorzatokat.

Felhasználói Felület Komponensei

A src/app/components mappában található komponensek alkotják az alkalmazás "látható" részét. Minden komponens három rétegből áll:

1. **HTML (.html):** A szerkezet (Template).
2. **CSS (.css):** A stílus (Style).
3. **TypeScript (.ts):** A vezérlő logika (Controller).

Regisztrációs Modul (RegisterComponent)

A regisztrációs űrlap a FormsModule-t használja a "Two-Way Data Binding" (kétirányú adatkötés) megvalósítására.

HTML Struktúra (Részlet):

HTML

```
<form (ngSubmit)="register()">
  <fieldset>
    <legend>Regisztráció</legend>
    <label for="username">Felhasználónév: </label>
    <input type="text" name="username" [(ngModel)]="username" required>


    <div class="center">
      <input type="submit" value="Regisztráció" [disabled]="isSubmitting">
    </div>
  </fieldset>
</form>
```

TypeScript Logika (register.component.ts):

TypeScript

```
register() {
  this.isSubmitting = true; // UX: Dupla kattintás elkerülése


  const regisztracio = {
    name: this.name,
    email: this.email,
    password: this.password,
    username: this.username
  };
```

```
  this.http.post('http://localhost:3000/auth/register', regisztracio)
    .subscribe({
      next: (res) => {
        this.snackBar.open('Sikeres regisztráció!', 'Bezár', { duration: 3000 });
        this.router.navigate(['/login']); // Automatikus átirányítás
      },
      error: (err) => {
        this.isSubmitting = false; // Hiba esetén újra engedélyezzük a gombot
        this.snackBar.open('Hiba történt!', 'Bezár');
      }
    });
}
```

A **kétirányú adatkötés** ([(ngModel)]) biztosítja, hogy amint a felhasználó gépel, a TypeScript változó (this.username) azonnal frissüljön, és fordítva. A isSubmitting flag használata egy apró, de fontos UX (User Experience) megoldás: megakadályozza, hogy a felhasználó türelmetlenségében többször elküldje a regisztrációs kérést, ami duplikációkhoz vagy szerverhibákhoz vezethetne.

Termékkatalógus és Szűrés (PerfumesComponent)

Ez a komponens a webáruház szíve. Feladata a termékek listázása, szűrése és a kosárba helyezés indítása.

Template Megvalósítás (*ngFor): Az Angular strukturális direktívája, az *ngFor segítségével járjuk be a termékek tömbjét, és generálunk minden elemhez egy HTML kártyát.

HTML

```
<div class="card-container">
  <div class="card" *ngFor="let perfume of filteredPerfumes">
    <img [src]="getImagePath(perfume.name)" alt="{{ perfume.name }}">
    <div class="card-body">
      <h5 class="card-title">{{ perfume.name }}</h5>
      <p class="card-text fw-bold">
        Ár: {{ (+perfume.price) * 1000 | number:'1.0-0'}} Ft
      </p>
      <button mat-raised-button color="primary" (click)="addToCart(perfume)">
```

```
      Kosárba
    </button>
  </div>
 </div>
</div>
```

Itt látható az Angular **Pipe**-ok használata is (number:'1.0-0'), amely a nyers számot (pl. 25) formázott pénznemmé alakítja (pl. 25 000), ezres elválasztókkal, tizedesjegyek nélkül.

Logika (perfumes.component.ts): A komponens inicializálásakor (ngOnInit) lekérjük az adatokat a szerverről. A keresőmező működését a filteredPerfumes tömbön keresztül valósítottuk meg. Amikor a felhasználó ír a keresőbe, nem indítunk új szerverhívást (ami lassú lenne), hanem a már letöltött listát szűrjük kliens oldalon. Ez azonnali visszajelzést ad.

Modális Kosár (CartDialogComponent)

A modern webdesign elveit követve a kosarat nem új oldalon, hanem egy felugró ablakban (MatDialog) jelenítjük meg. Ez lehetővé teszi a "Context Switching" elkerülését: a vásárló nem esik ki a böngészés folyamatából.

Adatbetöltés és szinkronizáció:

TypeScript

```
loadCart() {
  const token = localStorage.getItem('token');
  const headers = { Authorization: `Bearer ${token}` };


  this.http.get<any[]>('http://localhost:3000/cart/get', { headers })
    .subscribe({
      next: (res) => {
        this.cartItems = res; // A szerverről érkező friss adatok
      },
      error: (err) => console.error('Hiba a kosár lekérdezésekor:', err)
    });
}
```

Fontos megjegyezni, hogy bár van kliens oldali állapot (CartService), a CartDialog megnyitásakor mindig frissítünk a backendről (/cart/get). Ez biztosítja, hogy ha a felhasználó időközben másik eszközön módosította a kosarát, itt is a valós állapotot lássa. A törlés (removeFromCart) esetén a szerver válasza után a helyi tömböt (this.cartItems) a .filter() metódussal frissítjük, így a törölt elem azonnal eltűnik a listából.

Adminisztrációs Felület és Védelem (AdminComponent & AdminGuard)

A biztonság a frontend oldalon is kritikus. Bár a valódi védelem a backenden van, a frontend felelőssége, hogy az átlagfelhasználó elől elrejtse az adminisztrációs felületeket.

Útvonalvédelem (admin.guard.ts):

TypeScript

@Injectable({ providedIn: 'root' })

```
export class AdminGuard implements CanActivate {
  constructor(private router: Router) {}


  canActivate(): boolean {
    const role = localStorage.getItem('role');


    if (role === 'admin') {
      return true; // Engedélyezzük a belépést
    } else {
      alert('Csak adminoknak elérhető oldal!');
      this.router.navigate(['/login']); // Visszadobjuk a loginra
      return false;
    }
  }
}
```

Ez az AdminGuard az app-routing.module.ts-ben van bekötve: canActivate: [AdminGuard]. Mielőtt az Angular betöltené az Admin komponenst, lefuttatja ezt a logikát. Ha a localStorage-ban tárolt szerepkör nem 'admin', a navigáció megszakad.

Az AdminComponent maga egy egyszerű, de hatékony táblázatot tartalmaz, ahol az adminisztrátor láthatja a regisztrált felhasználókat és törölheti őket. A táblázatban egy vizuális feltétel ([disabled]="user.role === 'admin'") gondoskodik arról, hogy az admin ne törölhesse véletlenül saját magát vagy más adminisztrátorokat.

Felhasználói Profil (ProfileComponent)

A profil szerkesztése (PUT kérés) demonstrálja a teljes űrlapkitöltési folyamatot.

1. **Betöltés (GET):** Az űrlapmezőket inicializáláskor feltöltjük a jelenlegi adatokkal.
2. **Klónozás:** Az editUser = { ...data } szintaxissal másolatot készítünk az adatokról. Ez azért fontos, mert ha közvetlenül a megjelenített objektumot szerkesztenénk,

akkor a "Mégsem" gomb megnyomásakor nehéz lenne visszaállítani az eredeti állapotot.

3. **Mentés (PUT):** A szerkesztett objektumot küldjük vissza a szervernek.

Stílus és Felhasználói Élmény (UX/UI)

A megjelenésért a CSS3 és az Angular Material felel. Kiemelt figyelmet fordítottunk a modern trendekre, például a **Glassmorphism** (üveghatás) alkalmazására a bejelentkezési űrlapoknál.

CSS Megvalósítás (register.component.css):

CSS

```
fieldset {
  backdrop-filter: blur(0.4rem); /* A háttér elmosása */
  -webkit-backdrop-filter: blur(0.4rem);
  background-color: rgba(255, 255, 255, 0.1); /* Félig átlátszó fehér */
  border: 1px solid #ffffff;
  box-shadow: 0 0 15px #ffffff; /* Ragyogás effekt */
}
```

Ez a stílus modern, légies megjelenést kölcsönöz az alkalmazásnak. A háttérkép (background-attachment: fixed) rögzített, így görgetéskor parallax-szerű hatást kelt.

A reszponzivitást a **Flexbox** és a **CSS Grid** kombinációjával értük el. A terméklistánál (grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));) a böngésző automatikusan annyi kártyát helyez el egy sorban, amennyi kifér, így mobilon 1, tableten 2-3, monitoron 4+ oszlop jelenik meg média lekérdezések (@media) manuális írása nélkül is.

Tesztelés és Minőségbiztosítás

Bár a projekt fókuszában a funkcionalitás állt, minden komponenshez létrejöttek az alapvető egységteszt fájlok (.spec.ts). A tesztkörnyezet a **Jasmine** keretrendszert és a **Karma** tesztfuttatót használja.

Példa egy alapvető tesztre (app.component.spec.ts):

TypeScript

```
it('should create the app', () => {
  const fixture = TestBed.createComponent(AppComponent);
  const app = fixture.componentInstance;
  expect(app).toBeTruthy(); // Ellenőrzi, hogy a komponens létrejött-e
});
```

```
it(`should have as title 'reglog'`, () => {
  const fixture = TestBed.createComponent(AppComponent);
  const app = fixture.componentInstance;
  expect(app.title).toEqual('reglog'); // Ellenőrzi a változó értékét
});
```

Ezek a tesztek biztosítják, hogy az alkalmazás alapvető építőkockái (komponensek példányosítása, routing modul betöltése) hibátlanul működjenek.

Összegzés: A kliens oldali megvalósítás során egy robusztus, moduláris és biztonságos rendszert hoztunk létre. Az Angular fejlett eszköztárának (Guardok, Interceptorok, Service-ek, Reaktív Formok) használata biztosítja, hogy a kód karbantartható és továbbfejleszthető legyen, míg a Material Design és a Bootstrap integrációja a felhasználói elégedettséget szolgálja.

5.BACKEND

A Maison De Parfum backend rendszere modern, skálázható és széles körben támogatott technológiákra épül. A választott stack biztosítja a gyors fejlesztést, a biztonságos adatkezelést és a könnyű karbantarthatóságot. Az alábbiakban részletesen bemutatásra kerülnek a felhasznált főbb technológiai komponensek.

Node.js és a Futtatókörnyezet

A szerveroldali logika futtatásához a **Node.js** környezetet választottuk. A Node.js egy nyílt forráskódú, platformfüggetlen futtatókörnyezet, amely a Chrome V8 JavaScript motorjára épül. Kiemelkedő előnye az eseményvezérelt, aszinkron I/O modell, amely különösen alkalmassá teszi a valós idejű alkalmazások backendjének kiszolgálására.

Express.js Keretrendszer

A HTTP kérések kezelésére, az útválasztásra (routing) és a middleware-ek (köztes szoftverek) kezelésére az **Express.js** keretrendszert alkalmaztuk. Az Express a Node.js legnépszerűbb webes keretrendszere, amely minimalista, mégis rugalmas struktúrát biztosít. A package.json fájlunk tanúsága szerint az express verziója ^5.2.1, ami a keretrendszer legfrissebb, modern megoldásait kínálja.

Az Express használata lehetővé teszi, hogy a REST API végpontokat logikusan, értelmesen építsük fel. A rendszerben különválasztottuk az üzleti logikát (Controllers) és az útvonalak definícióját (Routes), amelyet az Express Router funkciója támogat.

Adatbázis és ORM: MySQL és Sequelize

Az adatok tárolásáért egy relációs adatbázis-kezelő, a **MySQL** felel. A választás azért esett a relációs modellre, mert egy webshop esetében kritikus fontosságú az adatok konzisztenciája, a táblák közötti kapcsolatok (pl. felhasználó és kosár, termék és vélemény) szigorú kezelése.

Az adatbázis és a Node.js alkalmazás közötti kommunikációt a **Sequelize ORM** (Object-Relational Mapping) könyvtár valósítja meg. A Sequelize lehetővé teszi, hogy SQL lekérdezések kézi írása helyett JavaScript objektumokkal és metódusokkal dolgozzunk, ami jelentősen felgyorsítja a fejlesztést és csökkenti az SQL Injection típusú támadások kockázatát.

A projektben használt főbb adatbázis-könyvtárak:

- mysql2: A MySQL adatbázis driver, amely gyorsabb és biztonságosabb, mint az elődje.
- sequelize: Maga az ORM rendszer.

Hitelesítés és Biztonság (JWT és Bcrypt)

A biztonság kiemelt szempont volt a tervezéskor. A jelszavak tárolása sosem történik nyílt szöveges formátumban. Erre a célra a **bcrypt** könyvtárat használjuk, amely "salt" (sózás) mechanizmussal és többszörös hasheléssel teszi visszafejthetetlenné a jelszavakat.

A felhasználók azonosítása nem munkamenet (session) alapú, hanem **JSON Web Token (JWT)** alapú. Ez a modern megközelítés (Stateless authentication) ideális Single Page Application (SPA) frontendek kiszolgálásához. Bejelentkezéskor a szerver aláír egy tokent, amelyet a kliens minden későbbi kérésnél elküld a fejlécben.

Adatbázis Tervezés és Adatmodellek

A rendszer alapját a jól strukturált adatbázis képezi. A Sequelize ORM segítségével definiáltuk a modelleket, amelyek közvetlenül leképeződnek az adatbázis tábláira. Az alábbiakban részletesen elemezzük az egyes entitásokat és azok kódját.

Felhasználók (Users) Modell

A felhasznalok tábla tárolja a regisztrált ügyfelek és az adminisztrátorok adatait. A modell definíciója a models/felhasznalok.js fájlban található.

Kód részlet a models/felhasznalok.js fájlból:

JavaScript

```
const {DataTypes} = require('sequelize');
const sequelize = require('../config/db');
```

```
const felhasznalok = sequelize.define('felhasznalok', {
   id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
      primaryKey: true
   },
   name: {
      type: DataTypes.STRING,
      allowNull: false
   },
   email: {
      type: DataTypes.STRING,
      allowNull: false,
      unique: true
   },
   username: {
      type: DataTypes.STRING,
      allowNull: false,
      unique: true
   },
   password: {
      type: DataTypes.STRING,
      allowNull: false
   },
    role: {
      type: DataTypes.STRING,
      allowNull: false,
      defaultValue: 'user'
   }
});

module.exports = felhasznalok;
```

Magyarázat: A modell biztosítja, hogy minden felhasználónak legyen egyedi azonosítója (id), neve, email címe és felhasználóneve. A unique: true kitétel garantálja, hogy ne lehessen ugyanazzal az email címmel vagy felhasználónévvel kétszer regisztrálni, ami alapvető üzleti logikai követelmény. A role mező alapértelmezett értéke 'user', de itt tároljuk az 'admin' jogosultságot is, amely később a védett végpontok elérésénél játszik szerepet.

Termékek (Perfumes) Modell

A webshop központi eleme a termékállomány. A perfumes tábla nemcsak az alapvető adatokat (név, ár, márka) tárolja, hanem specifikus jellemzőket is, amelyek segítik a szűrést (pl. évszakok, illatjegyek).

Kód részlet a models/perfumes.js fájlból:

JavaScript

```
const {DataTypes} = require('sequelize');
const sequelize = require('../config/db');

const perfumes = sequelize.define('perfumes', {
   id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
```

```
        primaryKey: true
    },
    name: {
        type: DataTypes.STRING,
        allowNull: false
    },
    brand: {
        type: DataTypes.STRING,
        allowNull: false
    },
    price: {
        type: DataTypes.DECIMAL(10, 2),
        allowNull: false
    },
    volume_ml: {
        type: DataTypes.INTEGER,
        allowNull: false
    },
    gender: {
        type: DataTypes.ENUM('ferfi', 'no', 'uniszex'),
        allowNull: false
    },
    season_spring: {
        type: DataTypes.BOOLEAN,
        defaultValue: false
    },
    // ... további évszakok (summer, autumn, winter) ...
    sweet: {
        type: DataTypes.BOOLEAN,
        defaultValue: false
    },
    spicy: {
        type: DataTypes.BOOLEAN,
        defaultValue: false
    }
});
module.exports = perfumes;
```

Elemzés: Különlegesség a BOOLEAN típusú mezők (season_spring, sweet, spicy stb.) használata. Ez a tervezési minta (Design Pattern) rendkívül hatékonnyá teszi a frontend oldali szűrést. Nem szöveges leírást kell elemezni, hanem egyszerű igaz/hamis értékek alapján lehet lekérdezni például az összes "tavaszi" vagy "fűszeres" parfümöt. A price mező DECIMAL(10, 2) típusa biztosítja a pénzügyi számítások pontosságát, elkerülve a lebegőpontos kerekítési hibákat.

Kosár és Kosár Tartalom

A kosár funkcionalitást két táblára bontottuk a normalizálás elvei szerint: kosar és kosar_tartalom.

A kosar tábla (models/kosar.js) köti össze a felhasználót a virtuális kosarával. Egy felhasználónak egy időben egy aktív kosara lehet. A kosar_tartalom (models/kosar_tartalom.js) pedig a kapcsolótábla, amely megmondja, hogy melyik kosárban, melyik parfümből hány darab van.

Kapcsolatok definiálása (models/index.js): Az adatbázis integritását a models/index.js fájlban definiált kapcsolatok (asszociációk) biztosítják.

JavaScript

```
const kosar_tartalom = require('./kosar_tartalom');
const kosar = require('./kosar');
const perfumes = require('./perfumes');
// ...
kosar_tartalom.belongsTo(perfumes, { foreignKey: 'parfum_id' });
```

Ez a sor teszi lehetővé, hogy amikor lekérjük a kosár tartalmát, a Sequelize automatikusan "join"-olja (hozzárendelje) a parfüm adatait (nevét, árát), így nem kell külön lekérdezéseket futtatni minden egyes tételhez.

Backend Architektúra és API Végpontok

A rendszer a klasszikus MVC (Model-View-Controller) mintát követi, de mivel ez egy API backend, a "View" réteg hiányzik, helyette JSON válaszokat küldünk. A kód három fő rétegre tagolódik:

1. **Routes (Útvonalak):** Definálják az URL-eket és a HTTP metódusokat.
2. **Controllers (Vezérlők):** Tartalmazzák az üzleti logikát.
3. **Models (Adatmodellek):** Kommunikálnak az adatbázissal.

A belépési pont a server.js, amely inicializálja az alkalmazást, beállítja a middleware-eket (pl. CORS) és elindítja a szervert.

A szerver inicializálása (server.js)

A server.js fájlban történik a rendszer teljes „összeállása”. Itt importáljuk az összes route definíciót és kötjük be őket az Express applikációba.

JavaScript

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
// ... route importok ...
const authRoutes = require('./routes/auth.routes');
const perfumeRoutes = require('./routes/perfume.routes');
const velemenyRoutes = require('./routes/velemeny.routes');
const cartRoutes = require('./routes/kosar.routes');
const userRoutes = require('./routes/user.routes');
const profileRoutes = require('./routes/profile.routes');
const app = express();

app.use(cors({
   origin: 'http://localhost:4200', // Angular frontend támogatása
   methods: ['GET', 'POST', 'PUT', 'DELETE'],
   allowedHeaders: ['Content-Type', 'Authorization']
}));

// route-ok használata..
```

```
app.use('/auth', authRoutes);

app.use('/perfumes', perfumeRoutes);

app.use('/velemenyek', velemenyRoutes);

app.use('/cart', cartRoutes);

app.use('/users', userRoutes);

app.use('/profile', profileRoutes);
```

Kiemelendő a cors beállítás: a szerver explicit módon engedélyezi a kéréseket a http://localhost:4200 címről, ami a fejlesztés alatt álló Angular frontend címe. Ez biztonsági szempontból fontos, mert megakadályozza, hogy illetéktelen domainekről hívják meg az API-t a böngészőn keresztül.

Hitelesítési Folyamat (Auth Controller)

A regisztráció és bejelentkezés logikája az auth.controller.js fájlban található. Ez az egyik legkritikusabb pontja a rendszernek.

Regisztráció

A register funkció először figyeli és kezeli a bemenetet: ellenőrzi, hogy minden mező ki van-e töltve, és hogy a jelszó elég hosszú-e (min. 8 karakter). Ezután ellenőrzi az adatbázisban, hogy létezik-e már a felhasználónév vagy az email, ha létezik, akkor hibát dob. Ha minden rendben, a jelszót hasheli és elmenti a felhasználót.

JavaScript

```
const hashedpassword = await bcrypt.hash(password, 10);
const newUser = await felhasznalok.create({name, email, username, password: hashedpassword});
```

A bcrypt.hash(password, 10) aszinkron hívás biztosítja, hogy a jelszó titkosítása ne blokkolja a főszálat (Event Loop), így a szerver terhelés alatt is reszponzív marad.

Bejelentkezés

A bejelentkezés során a rendszer összehasonlítja a megadott jelszót a tárolt hash-sel. Ha egyeznek, generál egy JWT tokent.

JavaScript

```
const token = jwt.sign(felhasznalok, SECRET_KEY, { expiresIn: '1h' });
```

A token lejárati ideje 1 óra (expiresIn: '1h'), ami biztonsági intézkedés: ha a tokent ellopják, az csak rövid ideig használható fel.

Middleware: A Kapuőr

Az auth.middleware.js felelős azért, hogy védett végpontokat (pl. kosárba rakás, profil szerkesztés) csak bejelentkezett felhasználók érhessenek el.

JavaScript

```
const authenticationtoken = (req, res, next) => {
  const token = req.headers['authorization']?.split(' ')[1];

  if (!token || tokenBlackList.has(token)) {
    return res.sendStatus(403);
  }

  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) {
      return res.sendStatus(403);
    }
    req.user = user;
    next();
  });
}
```

A middleware kinyeri a tokent a Authorization: Bearer <token> fejlécből. Ha a token érvényes, a benne lévő felhasználói adatokat (pl. id, role) a req.user objektumhoz csatolja, így a későbbi kontrollerek (pl. kosár kezelés) már tudni fogják, ki indította a kérést anélkül, hogy újra le kellene kérdezni az adatbázist.

Funkcionális Modulok Részletezése

Kosár Kezelés (Cart Logic)

A kosár logika megvalósítása a kosar.controller.js fájlban található, és ez tartalmazza az üzleti logika egyik legbonyolultabb részét. Nem elég egyszerűen beszúrni egy sort; ellenőrizni kell, van-e már kosara a felhasználónak, és van-e már az adott termékből a kosárban.

Termék hozzáadása (addtoCart):

1. Lekéri a felhasználó ID-t a tokenből (req.user.id).
2. Megkeresi a felhasználó kosarát (kosar tábla). Ha nincs, létrehoz egyet.
3. Megnézi, hogy a kosar_tartalom táblában létezik-e már ez a kosar_id és perfume_id páros.
    - **Ha igen:** Növeli a mennyiséget (mennyiseg += 1) és frissíti a rekordot.
    - **Ha nem:** Létrehoz egy új sort 1-es mennyiséggel és az aktuális árral.

Ez a "create or update" logika biztosítja, hogy a kosárban ne szerepeljen kétszer ugyanaz a parfüm külön sorként, hanem a mennyiség növekedjen.

JavaScript

```
    let cartItem = await kosar_tartalom.findOne({
      where: {
```

```
            kosar_id: userCart.id,
            parfum_id: perfumeId
        }
    });
    if (cartItem) {
        cartItem.mennyiseg += 1;
        await cartItem.save();
    } else {
         const perfume = await perfumes.findByPk(perfumeId);
        if (!perfume) {
            return res.status(404).json({ message: 'Parfüm nem található' });
        }
        await kosar_tartalom.create({
            kosar_id: userCart.id,
            parfum_id: perfumeId,
            mennyiseg: 1,
            ar: perfume.price
        });
    }
```

Adminisztrációs Funkciók és Seeding

A rendszer indításakor fontos, hogy legyenek alapértelmezett adatok és adminisztrátorok. Ezt a seeders mappa tartalma végzi el. A adminSeeder.js fájlban látható, hogy a rendszer indulásakor (ha még nem léteznek) létrehoz két dedikált admin fiókot. Ez biztosítja, hogy a telepítés után azonnal be lehessen lépni teljes jogosultsággal.

A perfumeSeeder.js felelős a kezdeti termékkínálat feltöltéséért. Olyan márkákat és parfümöket tölt be, mint a "Tom Ford - Bitter Peach" vagy a "Versace - Dylan Purple", beállítva azok minden tulajdonságát (ár, űrtartalom, évszakok). A bulkCreate metódus használata itt optimalizált, mivel az első lefutás után az összes parfümöt és azok adatait létrehozza az adatbázisban.

A parfümök hozzáadása az adatbázishoz másféleképpen is megoldható, rögtön át lehet konvertálni SQL kódra, és úgy betenni a projektben. A seeder fejlesztés szempontjából volt egyszerűbb és jobb, mivel így a backendből hozza létre az adatbázis a táblát, és azok adatait nem pedig külön SQL-be!

Parfümök hozzáadása SQL kódként:

```
CREATE TABLE perfumes (

    id INT AUTO_INCREMENT PRIMARY KEY,

    name VARCHAR(255) NOT NULL,

    brand VARCHAR(255) NOT NULL,

    price DECIMAL(10,2) NOT NULL,

    volume_ml INT NOT NULL,

    gender ENUM('ferfi', 'no', 'uniszex') NOT NULL,
```

```
    season_spring BOOLEAN DEFAULT FALSE,

    season_summer BOOLEAN DEFAULT FALSE,

    season_autumn BOOLEAN DEFAULT FALSE,

    season_winter BOOLEAN DEFAULT FALSE,


    sweet BOOLEAN DEFAULT FALSE,

    spicy BOOLEAN DEFAULT FALSE

);
```

```
INSERT INTO perfumes (name, brand, price, volume_ml, gender, season_spring, season_summer, season_autumn, season_winter, sweet, spicy) VALUES

('Bitter Peach', 'Tom Ford', 250.00, 50, 'no', TRUE, TRUE, FALSE, FALSE, TRUE, FALSE),

('Lost Cherry', 'Tom Ford', 250.00, 50, 'uniszex', TRUE, FALSE, TRUE, FALSE, TRUE, FALSE),

('Daisy Love Eau So Sweet', 'Marc Jacobs', 54.00, 30, 'no', TRUE, TRUE, FALSE, FALSE, TRUE, FALSE),

('Dylan Purple', 'Versace', 75.00, 30, 'no', TRUE, TRUE, FALSE, FALSE, TRUE, FALSE),

('Eau de Parfum', 'Jimmy Choo', 74.00, 60, 'no', TRUE, TRUE, FALSE, FALSE, TRUE, FALSE),

('Eden Juicy Apple', 'Kayali', 100.00, 100, 'no', TRUE, TRUE, FALSE, FALSE, TRUE, FALSE),

('Un Jardin sur le Nil', 'Hermès', 110.00, 125, 'uniszex', TRUE, TRUE, FALSE, FALSE, FALSE, FALSE),

('Opium', 'YSL', 90.00, 50, 'no', FALSE, FALSE, TRUE, TRUE, FALSE, TRUE),

('Khamrah Dukhan', 'Lattafa', 35.00, 50, 'uniszex', FALSE, TRUE, TRUE, FALSE, FALSE, TRUE),
```

('Spicebomb Infrared', 'Viktor & Rolf', 95.00, 50, 'ferfi', FALSE, FALSE, TRUE, TRUE, FALSE, TRUE),

('Dolce Amalfi', 'Xerjoff', 200.00, 100, 'uniszex', TRUE, FALSE, TRUE, FALSE, FALSE, TRUE),

('One Million', 'Paco Rabanne', 95.00, 50, 'ferfi', FALSE, TRUE, FALSE, TRUE, FALSE, TRUE),

('Terre d'Hermès', 'Hermès', 80.00, 50, 'ferfi', TRUE, FALSE, TRUE, FALSE, FALSE, TRUE),

('Delina', 'Parfums de Marly', 280.00, 75, 'no', TRUE, FALSE, FALSE, FALSE, TRUE, FALSE),

('Black Opium', 'YSL', 125.00, 50, 'no', TRUE, FALSE, TRUE, TRUE, TRUE, TRUE),

('Under the Lemon Trees', 'Maison Margiela', 85.00, 100, 'uniszex', TRUE, TRUE, FALSE, FALSE, FALSE, FALSE),

('Acqua di Gioia', 'Giorgio Armani', 95.00, 100, 'no', TRUE, TRUE, FALSE, FALSE, FALSE, FALSE),

('Bombshell', 'Victoria's Secret', 60.00, 100, 'no', FALSE, TRUE, FALSE, FALSE, TRUE, FALSE),

('Light Blue', 'Dolce & Gabbana', 75.00, 100, 'no', FALSE, TRUE, FALSE, FALSE, FALSE, FALSE),

('I Want Choo', 'Jimmy Choo', 76.00, 60, 'no', FALSE, TRUE, FALSE, FALSE, TRUE, FALSE),

('Alchemie', 'Bottega Veneta', 135.00, 100, 'uniszex', FALSE, TRUE, FALSE, FALSE, FALSE, TRUE),

('Tangerine Boy', 'Phlur', 99.00, 50, 'uniszex', TRUE, TRUE, FALSE, FALSE, FALSE, FALSE),

('Strawberry Letter', 'Phlur', 99.00, 50, 'no', FALSE, TRUE, FALSE, FALSE, TRUE, FALSE),

('On a Date', 'Maison Margiela', 85.00, 100, 'uniszex', TRUE, FALSE, TRUE, FALSE, FALSE, FALSE),

('Eau de Pamplemousse Rose', 'Hermès', 144.00, 100, 'uniszex', TRUE, FALSE, FALSE, FALSE, FALSE, FALSE),

('Bright Crystal', 'Versace', 92.00, 90, 'no', TRUE, TRUE, FALSE, FALSE, TRUE, FALSE),

('Blackberry & Bay', 'Jo Malone', 86.00, 100, 'uniszex', TRUE, TRUE, FALSE, FALSE, FALSE, FALSE),

('Father Figure', 'Phlur', 99.00, 50, 'uniszex', FALSE, FALSE, TRUE, TRUE, FALSE, FALSE),

('Mango Skin', 'Maison Margiela', 85.00, 100, 'uniszex', FALSE, TRUE, FALSE, FALSE, FALSE, FALSE),

('Baccarat Rouge 540', 'Maison Francis Kurkdjian', 300.00, 70, 'uniszex', FALSE, FALSE, TRUE, TRUE, TRUE, FALSE);

Ennek a kódnak a futtatása SQL-ben ugyanazt fogja eredményezni, mint a bulkCreate() metódus a backendben.

Terméklogika és Adatstruktúra Kezelés

A webshop központi eleme a termékkínálat megjelenítése. Bár a végfelhasználó számára ez egy egyszerű listának tűnik, a háttérben a perfume.controller.js és a hozzá tartozó adatbázis-modell gondoskodik arról, hogy az adatok strukturáltan és a kliensoldali feldolgozásra optimalizált formában kerüljenek kiküldésre.

Termékek Listázása (GET /perfumes)

A rendszer jelenlegi architektúrája a „Fat Client, Thin Server” megközelítést alkalmazza a termékek lekérdezésénél. A getperfumes metódus az összes elérhető parfümöt lekéri az adatbázisból, és JSON formátumban visszaküldi a kliensnek.

Kód elemzése (controllers/perfume.controller.js):

JavaScript

```
const getperfumes = async (req, res) => {
   try {
      const lista = await perfumes.findAll();
      res.json(lista);
   } catch (error) {
      console.log("Hiba a parfümök lekérésekor:", error);
      res.status(500).json({ message: "Hiba történt a parfümök lekérése során!" });
   }
};
```

Bár ez a megoldás egyszerűnek tűnik, a hatékonysága a perfumes modellben rejlik. Mivel a modellben minden szűrési feltétel (pl. season_summer, sweet, gender) BOOLEAN vagy ENUM típusként van tárolva, a frontend alkalmazásnak (Angular) nem kell bonyolult szövegelemzést végeznie. Amikor a kliens megkapja a teljes listát, a memóriában milliszekundumok alatt képes szűrni például a „nyári” és „édes” illatokat az item.season_summer === true feltételekkel. Ez csökkenti a szerver terhelését, mivel nem kell minden egyes szűrőmódosításkor újabb adatbázis-lekérdezést indítani.

Profilkezelés és Adatbiztonság

A felhasználói profilok kezelése (profile.controller.js) kiemelten érzékeny terület, mivel itt történik a személyes adatok módosítása. A rendszernek garantálnia kell, hogy a felhasználó csak a saját adatait módosíthatja, és nem foglalhatja el mások azonosítóit (email, felhasználónév).

Profil Frissítési Logika (Update Profile)

A profilfrissítés során egy többlépcsős validációs folyamat zajlik le a háttérben. Nem elég egyszerűen felülírni az adatokat; ellenőrizni kell az egyediségi korlátokat (Unique Constraints).

Konfliktuskezelés a kódban: A rendszer ellenőrzi, hogy a módosítani kívánt email cím vagy felhasználónév létezik-e már az adatbázisban. A kritikus pont itt a foglaltEmail.id !== userId feltétel.

JavaScript

```
if (email) {
   const foglaltEmail = await felhasznalok.findOne({ where: { email } });
   if (foglaltEmail && foglaltEmail.id !== userId) {
      return res.status(400).json({ message: 'Ez az email cím már foglalt!' });
   }
   user.email = email;
}
```

Ez a feltétel engedélyezi, hogy a felhasználó megtartsa a saját email címét (azaz, ha a talált rekord ID-ja megegyezik a felhasználóéval), de megakadályozza, hogy egy másik felhasználó már létező címére változtassa azt. Ez a logika elengedhetetlen az adatbázis integritásának megőrzéséhez.

Válasz "Sanitization" (Adattisztítás)

Biztonsági szempontból kritikus, hogy a szerver soha ne küldjön vissza jelszót (még hashelt formában sem) a kliensnek, amikor profiladatokat szolgáltat. A profile.controller.js ezt a toJSON() metódus és a delete operátor használatával oldja meg:

JavaScript

```
await user.save();

const userResponse = user.toJSON();
delete userResponse.password;

res.status(200).json({ message: 'Profil sikeresen frissítve!', user: userResponse });
```

Ez a lépés biztosítja, hogy ha egy támadó valahogyan hozzá is fér a hálózati forgalomhoz, a válaszcsomagban nem találja meg a jelszóhash-t, amit esetleg brute-force módszerrel megpróbálhatna feltörni.

Vélemények Kezelése és Jogosultsági Szintek

A webshop közösségi élményét a véleményezési rendszer (velemenyek modul) biztosítja. Itt a legfontosabb fejlesztői kihívás a **Resource Ownership** (erőforrás-birtoklás) és az adminisztrátori felülbírálat megvalósítása volt.

Kettős Jogosultsági Ellenőrzés

A vélemény törlésekor vagy módosításakor a rendszernek el kell döntenie: "Ki vagy te?". A velemeny.controller.js fájlban található logika két esetben engedélyezi a műveletet:

1. A felhasználó a vélemény tulajdonosa (opinion.user_id === userId).
2. A felhasználó adminisztrátor (userRole === 'admin').

Implementáció:

JavaScript

```
const deleteVelemeny = async (req, res) => {
   // ...
   const userId = req.user.id;
   const userRole = req.user.role;

   const opinion = await velemenyek.findByPk(id);

   if (opinion.user_id !== userId && userRole !== 'admin') {
       return res.status(403).json({ message: 'Nincs jogosultság a vélemény törlésére!' });
   }
   // ... törlés végrehajtása
};
```

Ez a "vagy-vagy" feltételrendszer teszi lehetővé, hogy a felhasználók menedzseljék saját tartalmaikat, miközben az adminisztrátoroknak moderációs jogot biztosít a nem megfelelő tartalmak eltávolítására. Ez a fajta Role-Based Access Control (RBAC) a biztonságos webes alkalmazások alapköve.

Kapcsolatok az adatbázisban

A vélemények tábla (models/velemenyek.js) közvetlen kapcsolatban áll a felhasználókkal. A models/index.js-ben definiált velemenyek.belongsTo(felhasznalok) kapcsolat biztosítja, hogy ha egy felhasználót törölnek a rendszerből, a hozzá tartozó vélemények is kezelhetők legyenek. A user.controller.js deleteUser funkciója explicit módon gondoskodik a "takarításról":

JavaScript

```
await velemenyek.destroy({ where: { user_id: userId } });
```

Ez megakadályozza az "árva rekordok" (orphan records) felhalmozódását az adatbázisban, amelyek olyan felhasználókra mutatnának, akik már nem léteznek.

Rendszerkonfiguráció és Üzemeltetés

A backend rendszer nem légüres térben működik; szorosan integrálódik a környezetével. A konfigurációt a "Twelve-Factor App" módszertan szerint környezeti változókban tároljuk.

Környezeti Változók (.env)

A .env fájl tartalmazza azokat a bizalmas adatokat, amelyeket tilos a forráskódba égetni (hardcode). A dotenv csomag tölti be ezeket a változókat a process.env objektumba a futás kezdetekor.

A projektben használt kulcsfontosságú változók:

- DB_HOST, DB_USER, DB_PASS, DB_NAME: Az adatbázis kapcsolódási adatai. Fejlesztői környezetben ez általában a localhost és a root felhasználó, de éles környezetben (Production) ezeket a szolgáltató által megadott biztonságos értékekre kell cserélni.
- SECRET_KEY: Ez a JWT tokenek aláírására használt titkos kulcs. Ennek ismeretében bárki gyárthatna hamis tokeneket, ezért ennek védelme kritikus. A kódban lévő fallback (process.env.SECRET_KEY || 'TartalekKulcs') csak fejlesztési célokat szolgál, élesben kötelező erős kulcsot megadni.

CORS (Cross-Origin Resource Sharing)

Mivel a backend (pl. port 3000) és a frontend (pl. port 4200) eltérő porton fut, a böngészők biztonsági okokból alapértelmezetten tiltják a kommunikációt. A server.js-ben konfigurált CORS middleware ezt oldja fel:

JavaScript

```
app.use(cors({
   origin: 'http://localhost:4200',
   methods: ['GET', 'POST', 'PUT', 'DELETE'],
   allowedHeaders: ['Content-Type', 'Authorization']
}));
```

Ez a beállítás szigorúan csak a fejlesztői frontendnek engedélyezi a hozzáférést. Élesítéskor a http://localhost:4200 címet le kell cserélni a frontend éles domain nevére (pl. https://maison-de-parfum.com), hogy illetéktelen weboldalak ne tudjanak API hívásokat indítani a felhasználó nevében.

Hibaelhárítási Útmutató (Troubleshooting)

A fejlesztés és üzemeltetés során felmerülhetnek tipikus hibák. Az alábbi útmutató a leggyakoribb problémákra nyújt megoldást a Maison De Parfum rendszerspecifikus környezetében.

Adatbázis Kapcsolódási Hibák

Hibaüzenet: SequelizeConnectionRefusedError vagy Access denied for user. **Ok:** A Node.js alkalmazás nem éri el a MySQL szervert. **Megoldás:**

1. Ellenőrizze, hogy fut-e a MySQL szolgáltatás (XAMPP Control Panelben a MySQL modul zöld-e).
2. Ellenőrizze a .env fájl tartalmát. Helyes a jelszó? Üres jelszó esetén a DB_PASS= után ne legyen szóköz.
3. Létezik-e a maisondb adatbázis? Ha nem, hozza létre phpMyAdminban vagy SQL parancssorban: CREATE DATABASE maisondb;.

JWT Token Hibák

Hibaüzenet: 403 Forbidden API hívásoknál. **Ok:** A token lejárt, hiányzik, vagy a szerver újraindult (és ha memóriában tárolnánk a blacklistet, az törlődne, de itt stateless a hitelesítés). **Diagnosztika:**

1. Ellenőrizze a böngésző "Network" fülén a kérés fejlécét. Tartalmazza az Authorization: Bearer <token> sort?
2. A token lejárati ideje 1 óra (expiresIn: '1h'). Ha letelt, a felhasználót ki kell léptetni a frontend oldalon, és újra bejelentkeztetni.

Seeding (Adatfeltöltés) Problémák

Hiba: Az adatbázis üres indítás után, nincsenek parfümök. **Ok:** A seedPerfumes vagy seedAdmins függvény nem futott le, vagy a tábla már tartalmazott adatot, ezért a feltétel (if (count > 0) return;) miatt kilépett. **Megoldás:** Ha újra akarja generálni az adatokat, törölje a táblák tartalmát manuálisan (TRUNCATE), majd indítsa újra a szervert (npm start). A server.js-ben a sequelize.sync().then(...) ágban hívódnak meg a seeder függvények, amelyek automatikusan újratöltik az adatbázist, ha az üres.

Részletes API Referencia és Dokumentáció

A fejlesztés során elengedhetetlen a backend és frontend közötti kommunikáció pontos specifikálása. Az alábbiakban a Maison De Parfum rendszerének végpontjai kerülnek részletezésre, Swagger-típusú struktúrában, amely segíti a kliensoldali fejlesztők munkáját.

Hitelesítés (Auth) Végpontok

Ezek a végpontok felelősek a felhasználók beléptetéséért és a JWT tokenek kiosztásáért. A rendszer biztonsága ezen a ponton dől el.

HTTP Metódus	Végpont	Leírás	Szükséges Adatok (Body)	Válasz (Siker)	Auth
POST	/auth/register	Új felhasználó regisztrációja	name, email, username, password	201 Created - { message: "Sikeres regisztráció!" }	Nem
POST	/auth/login	Bejelentkezés	email vagy username, password	200 OK - { token: "jwt_string", user: {...} }	Nem
POST	/auth/logout	Kijelentkezés	-be kell legyen jelentkezve a felhasznáónak	200 OK - (Kliens oldalon token törlés)	Igen

Fejlesztői megjegyzés: A /login végpont különleges logikát tartalmaz (auth.controller.js), amely lehetővé teszi, hogy a felhasználó email címmel VAGY felhasználónévvel is beléphessen. Ezt a Sequelize Op.or operátorával vagy egy feltételes kereséssel valósítjuk meg, ami növeli a felhasználói élményt (UX).

Termék (Perfumes) Végpontok

A webshop kirakata. Mivel a termékek megtekintése nem igényel regisztrációt, a listázó végpont publikus, de a módosításokhoz admin jog szükséges (bár a jelenlegi specifikációban a szerkesztés csak seederekkel történik, a bővítés lehetősége adott).

HTTP Metódus	Végpont	Leírás	Paraméterek	Válasz	Auth
GET	/perfumes	Összes parfüm lekérése	-	200 OK - [{ id: 1, name: "Bitter Peach", ... }, ...]	Nem
GET	/perfumes/:id	Egy konkrét parfüm lekérése	id (URL paraméter)	200 OK - { id: 1, name: "Bitter Peach", ... }	Nem

Adatstruktúra válasz:

A frontend számára optimalizált JSON válasz tartalmazza a season_* boolean értékeket is.

JSON

```
{
  "id": 1,
  "name": "Bitter Peach",
  "brand": "Tom Ford",
  "price": "250.00",
  "gender": "no",
  "season_summer": true,
  "sweet": true
}
```

Kosár (Cart) Végpontok

Ez a modul igényli a legszigorúbb állapotkezelést. Minden művelethez érvényes JWT token szükséges.

HTTP Metódus	Végpont	Leírás	Body / Params	Válasz	Auth
GET	/cart/get	Kosár tartalmának lekérése	-	200 OK - [{ parfum_id: 1, mennyiseg: 2, ... }]	Igen
POST	/cart/add	Termék hozzáadása	perfumeId	200 OK - { message: "Termék hozzáadva" }	Igen
DELETE	/cart/:parfumId	Tétel törlése a kosárból	parfumId (URL)	200 OK - { message: "Törölve" }	Igen

Technikai háttér:

A DELETE végpont nem a teljes kosarat törli, hanem csak egy specifikus terméket. A kosar.controller.js-ben a destroy metódus where feltétele biztosítja, hogy csak az adott felhasználó (userCart.id) adott terméke (parfumId) kerüljön eltávolításra.

Projekt Strukturális Összefoglalása (File System)

A könnyebb átláthatóság érdekében itt látható a teljes projekt végleges fájlstruktúrája, amely a Node.js best practice elveit követi.

```
MDP_Backend/
│
├── config/
│   ├── db.js          # Adatbázis kapcsolat (Sequelize példány)
│   └── jwt.js         # JWT titkos kulcs konfiguráció
│
├── controllers/       # Üzleti logika
│   ├── auth.controller.js
│   ├── kosar.controller.js
│   ├── perfume.controller.js
│   ├── profile.controller.js
│   ├── user.controller.js
│   └── velemeny.controller.js
│
├── middlewares/
│   └── auth.middleware.js # JWT ellenőrzés
│
├── models/            # Adatbázis sémák
│   ├── index.js       # Kapcsolatok definiálása
│   ├── felhasznalok.js
│   ├── kosar.js
│   ├── kosar_tartalom.js
│   ├── perfumes.js
│   └── velemenyek.js
│
├── routes/            # API útvonalak
│   ├── auth.routes.js
│   ├── kosar.routes.js
│   ├── perfume.routes.js
│   ├── profile.routes.js
│   ├── user.routes.js
│   └── velemeny.routes.js
│
├── seeders/           # Kezdeti adatok feltöltése
│   ├── adminSeeder.js
│   └── perfumeSeeder.js
│
├── .env               # Környezeti változók
├── .gitignore
├── package.json
└── server.js          # Alkalmazás belépési pontja
```

Ez a struktúra biztosítja a **Separation of Concerns** (feladatok szétválasztása) elv érvényesülését. Ha a jövőben új funkciót kell beépíteni (pl. Hírlevél), elég egy új kontrollert, modellt és route fájlt létrehozni anélkül, hogy a meglévő kódokat át kellene írni.

Jövőbeli Fejlesztési Irányok

A projekt jelenlegi állapotában teljesíti az összes alapvető e-kereskedelmi funkciót:

- Biztonságos regisztráció és bejelentkezés.
- Termékek kezelése és szűrése.
- Kosár funkciók (hozzáadás, törlés, mennyiségkezelés).
- Felhasználói profilok és vélemények kezelése adminisztrátori felügyelettel.

Fejlesztési lehetőségek: A jövőben a rendszer tovább bővíthető lenne **képfeltöltési funkcióval** (multer middleware használatával), hogy a parfümök képeit ne csak URL-ként, hanem saját szerveren tároljuk. Szintén hasznos fejlesztés lenne egy **email küldő szolgáltatás** (pl. Nodemailer) integrálása, amely értesítést küld regisztrációkor vagy elfelejtett jelszó esetén. Ezen kívül a **fizetési kapu** (Stripe vagy PayPal) integrációja tenné teljessé a webshopot, valódi tranzakciókat téve lehetővé. Elektromos számla készítése az oldalon, ami összeköttetésbe keröl a Billingo weboldallal.

A kód átláthatósága és a moduláris felépítés garantálja, hogy ezek a bővítések zökkenőmentesen megvalósíthatók legyenek anélkül, hogy a jelenlegi stabil működést veszélyeztetnék.

Összefoglalás

A Maison De Parfum backend rendszere egy gondosan megtervezett, skálázható és biztonságos alapokon nyugvó webáruház motor. A dokumentáció során végigvettük az üzleti igényektől kezdve a technológiai választásokon át a konkrét kódszintű megvalósításig a teljes rendszert.

A projekt demonstrálja a modern webfejlesztés alapvető kompetenciáit:

- Aszinkron programozás (Async/Await).
- Relációs adatbázis-tervezés és ORM használat
- RESTful API tervezési elvek.
- Stateless hitelesítés (JWT).
- Biztonsági tudatosság (Password Hashing, Input Validation).

Ez a rendszer nemcsak egy iskolai vizsgaremek, hanem egy valós piaci környezetben is megállná a helyét egy induló webshop alapjaként (MVP - Minimum Viable Product). A kód modularitása lehetővé teszi a könnyű karbantartást és a jövőbeni funkcióbővítéseket, így a Maison De Parfum technológiai háttere hosszú távon is fenntartható megoldást kínál.

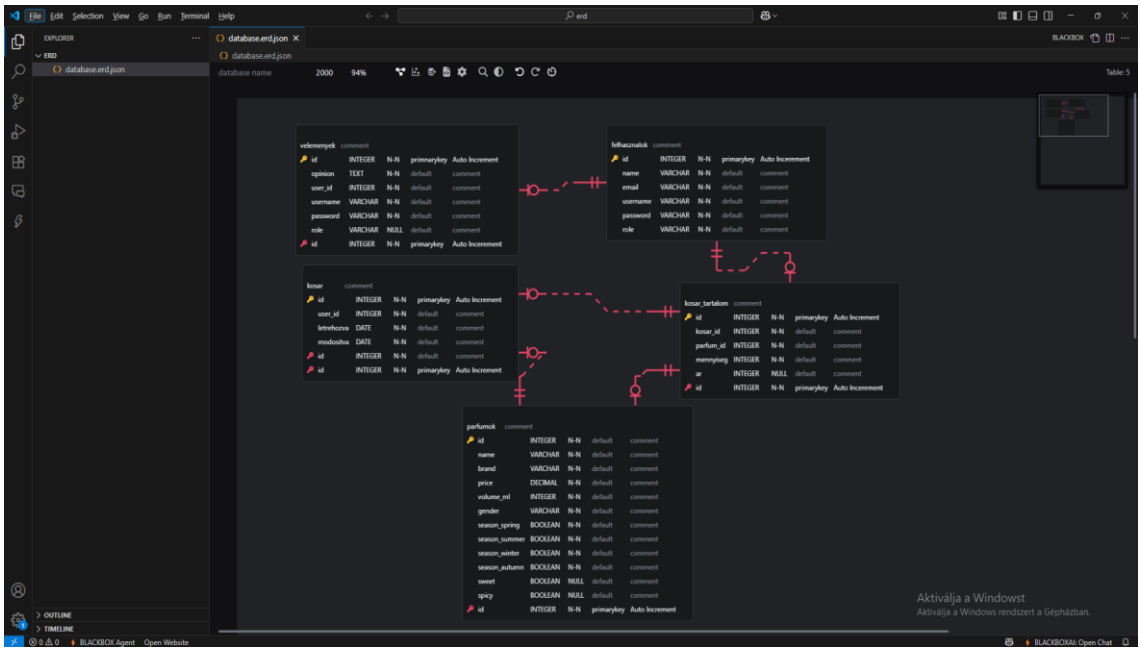
6.Adatbázis Tervezés és Specifikáció

A **Maison De Parfum** webáruház adatbázisának tervezésekor a relációs adatmodell elveit követtük. A rendszer fizikai megvalósítása **MySQL** adatbázis-kezelőn alapul, amelyhez a backend alkalmazás a **Sequelize ORM** (Object-Relational Mapping) rétegen keresztül kapcsolódik.

Ez a rétegzett megközelítés biztosítja az adatok integritását, a biztonságos típuskezelést, valamint a fejlesztés során az átlátható adatszerkezetet. A terv az ERD (Entity Relationship Diagram) szabványai szerint készült, amely öt fő entitást és azok kapcsolatait definiálja.

Entitás-Kapcsolat Modell (ERD)

Az adatbázis vizuális terve (database.erd.json) alapján a rendszer öt táblát tartalmaz, amelyek a **3. Normálformára (3NF)** lettek optimalizálva. A redundancia minimalizált (kivéve a velemenyek táblát, ahol a felhasználónév a gyorsabb lekérdezés érdekében tárolásra kerül).



Az adatbázisban szereplő entitások:

1. **felhasznalok** (Users): A rendszer szereplőinek központi táblája.
2. **perfumes** (Products): A webshop terméktörzse.
3. **velemenyek** (Reviews): A termékekhez vagy szolgáltatáshoz kapcsolódó visszajelzések.
4. **kosar** (Cart Header): A bevásárlókosár fejrész adatait tárolja.
5. **kosar_tartalom** (Cart Items): A kosárban lévő tételek kapcsolótáblája.

Táblák Részletes Definíciója

Az alábbi táblázatok a fizikai adatbázis szerkezetét mutatják be, beleértve a mezőtípusokat, kényszereket (constraints) és a kapcsolódó üzleti logikát.

Felhasználók tábla (felhasznalok)

A rendszerbe regisztrált vásárlók és adminisztrátorok adatait tárolja.

Mezőnév	Adattípus	Kényszerek (Constraints)	Leírás
id	INTEGER	PK, Auto Increment	Egyedi azonosító.
name	VARCHAR(255)	Not Null	A felhasználó teljes neve.
email	VARCHAR(255)	Unique, Not Null	Egyedi email cím, belépési azonosító.
username	VARCHAR(255)	Unique, Not Null	Egyedi felhasználónév.
password	VARCHAR(255)	Not Null	Bcrypt algoritmussal titkosított jelszó hash.
role	VARCHAR(255)	Default: 'user'	Jogosultsági szint ('user' vagy 'admin').
createdAt	DATETIME	-	Rekord létrehozásának ideje.
updatedAt	DATETIME	-	Utolsó módosítás ideje.

Parfümök tábla (perfumes)

A termékek tulajdonságait leíró törzsadattábla. Kiemelendő a BOOLEAN típusú oszlopok használata (pl. season_summer, sweet), amelyek lehetővé teszik a frontend oldalon a rendkívül gyors és egyszerű szűrést ("checkbox" logika).

Mezőnév	Adattípus	Kényszerek	Leírás
id	INTEGER	PK, Auto Increment	Termék azonosító.
name	VARCHAR(255)	Not Null	A parfüm fantázianeve (pl. "Bitter Peach").
brand	VARCHAR(255)	Not Null	Márkanév.
price	DECIMAL(10,2)	Not Null	Egységár (tizedesjegy pontossággal).
volume_ml	INTEGER	Not Null	Kiszerelés űrtartalma (ml).
gender	ENUM	'ferfi', 'no', 'uniszex'	Célcsoport neme.
season_spring	TINYINT(1)	Default: 0	Tavaszi illat-e? (Boolean).
season_summer	TINYINT(1)	Default: 0	Nyári illat-e? (Boolean).
season_autumn	TINYINT(1)	Default: 0	Őszi illat-e? (Boolean).
season_winter	TINYINT(1)	Default: 0	Téli illat-e? (Boolean).
sweet	TINYINT(1)	Default: 0	Édes illatjegy.
spicy	TINYINT(1)	Default: 0	Fűszeres illatjegy.

Kosár Fej tábla (kosar)

Ez a tábla valósítja meg a logikai kapcsolatot a felhasználó és a kosara között. A rendszer logikája biztosítja az **1:1 kapcsolatot**: egy felhasználónak egyszerre csak egy aktív kosara lehet.

Mezőnév	Adattípus	Kényszerek	Leírás
id	INTEGER	PK, Auto Increment	Kosár azonosító.
user_id	INTEGER	FK (felhasznalok.id)	A kosár tulajdonosa.
letrehozva	DATETIME	Default: NOW	A kosár létrehozásának dátuma.
modositva	DATETIME	Default: NOW	Utolsó aktivitás ideje.

Kosár Tartalom tábla (kosar_tartalom)

Ez a kapcsolótábla (Join Table) valósítja meg a "több-a-többhöz" kapcsolatot a kosár és a termékek között.

Mezőnév	Adattípus	Kényszerek	Leírás
id	INTEGER	PK, Auto Increment	Tétel azonosító.
kosar_id	INTEGER	FK (kosar.id)	Melyik kosárhoz tartozik. *On Delete: Cascade*
parfum_id	INTEGER	FK (perfumes.id)	Melyik termék került a kosárba.
mennyiseg	INTEGER	Default: 1	A termék darabszáma.
ar	INTEGER	Not Null	Az ár a kosárba helyezés pillanatában (árgarancia).

Vélemények tábla (velemenyek)

A felhasználói visszajelzéseket tároló tábla.

Mezőnév	Adattípus	Kényszerek	Leírás
id	INTEGER	PK, Auto Increment	Vélemény azonosító.
opinion	TEXT	Not Null	A vélemény szövege (hosszú szöveg).
user_id	INTEGER	FK (felhasznalok.id)	A vélemény írója.
username	VARCHAR(255)	Not Null	A szerző neve (redundáns mező a megjelenítés gyorsítására).

*(Rövidítések: **PK**: Primary Key / Elsődleges Kulcs, **FK**: Foreign Key / Idegen Kulcs)*

Adatbázis Kapcsolatok és Integritás

A táblák közötti kapcsolatokat a Sequelize models/index.js fájlja definiálja, biztosítva a hivatkozási integritást.

1. **Felhasználó – Kosár (1:1 kapcsolat):**
   - Egy felhasználóhoz egy kosár tartozik.
   - *Logika:* Ha a felhasználó belép, a rendszer ellenőrzi (kosar.findOne), van-e már kosara. Ha nincs, létrehoz egyet.
2. **Kosár – Kosár Tartalom (1:N kapcsolat):**
   - Egy kosár több tételt tartalmazhat.
   - *Megszorítás:* onDelete: 'CASCADE'. Ez azt jelenti, hogy ha a kosarat töröljük (pl. rendelés leadása után vagy adminisztrátori törlésnél), az összes benne lévő tétel (kosar_tartalom sor) automatikusan törlődik, megakadályozva az árva rekordok létrejöttét.
3. **Kosár Tartalom – Parfüm (N:1 kapcsolat):**
   - Több kosárban is szerepelhet ugyanaz a parfüm.
   - A lekérdezésnél a include: [{ model: perfumes }] utasítás segítségével ("Eager Loading") a backend egyetlen lépésben kéri le a kosár tartalmát és a hozzá tartozó termékneveket/árakat.
4. **Felhasználó – Vélemény (1:N kapcsolat):**
   - Egy felhasználó több véleményt is írhat.
   - Törlés esetén a felhasználóhoz tartozó vélemények is eltávolításra kerülnek a user.controller.js logikája alapján.

Fizikai Megvalósítás (SQL)

Az adatbázis sémát a Sequelize automatikusan generálja a sequelize.sync() parancs futtatásakor. A kapcsolatok létrehozásáért felelős SQL parancsok (példa):

SQL

```
ALTER TABLE `kosar_tartalom`
ADD CONSTRAINT `kosar_tartalom_kosar_id_foreign_idx`
FOREIGN KEY (`kosar_id`) REFERENCES `kosar` (`id`) ON DELETE CASCADE ON UPDATE CASCADE;
```

Ez a struktúra biztosítja, hogy a "Maison De Parfum" webshop adatbázisa stabil, skálázható és konzisztens maradjon a használat során.

Adatvédelmi és Biztonsági Megfontolások (GDPR)

Mivel a rendszer személyes adatokat (név, email cím) kezel, az adatbázis tervezésekor figyelembe vettük a GDPR (General Data Protection Regulation) előírásait. A "Maison De Parfum" adatbázisa az alábbi védelmi mechanizmusokat alkalmazza:

1. **Adattakarékosság elve:**

   Csak a működéshez feltétlenül szükséges adatokat tároljuk. Nem kérünk be telefonszámot, születési dátumot vagy lakcímet a regisztrációhoz, mivel a jelenlegi MVP (Minimum Viable Product) fázisban nincs rájuk szükség.

2. **Jelszavak védelme (Hashing):**

   A felhasznalok tábla password mezője soha nem tartalmaz nyílt szöveges jelszót. A backend a bcrypt algoritmust használja, amely egyirányú titkosítást végez.

   - *Kockázatkezelés:* Még ha egy támadó hozzá is férne a teljes adatbázishoz (SQL Dump), a jelszavakat nem tudná visszafejteni, így a felhasználói fiókok védve maradnak.

3. **Felejtéshez való jog:**

   A rendszer támogatja a felhasználói fiók törlését (DELETE /users/:id). A relációs adatbázis beállításai (ON DELETE CASCADE) biztosítják, hogy ilyenkor a felhasználó összes "nyoma" (kosár tartalma, vélemények) fizikailag is törlődik a lemezről, nem csak logikai törlés történik.

Indexelési Stratégia és Teljesítmény

A webshopoknál kritikus a gyors válaszidő. A MySQL adatbázisban indexek segítségével gyorsítjuk a lekérdezéseket. A Sequelize definíciók alapján a következő indexek jönnek létre automatikusan:

- **Elsődleges kulcsok (PRIMARY KEY):** Minden tábla id mezője indexelt, így a findByPk (keresés ID alapján) műveletek O(1) komplexitásúak, azaz azonnaliak.

- **Egyedi indexek (UNIQUE):**

  A felhasznalok táblában az email és a username mezőkön egyedi index van.

  - *Előny:* A bejelentkezéskor (findOne({ where: { email } })) az adatbázisnak nem kell végignéznie az összes felhasználót (Full Table Scan), hanem a B-Tree index segítségével azonnal megtalálja a rekordot.
- **Idegen kulcs indexek (FOREIGN KEY):**

  A kosar_tartalom tábla parfum_id és kosar_id mezői indexeltek, ami drasztikusan felgyorsítja a JOIN műveleteket (amikor a kosár tartalmát a terméknevekkel együtt kérjük le).

SQL Implementáció (DDL Szkriptek)

Bár a táblákat az ORM (Sequelize) hozza létre, a dokumentáció teljessége érdekében mellékeljük a generált SQL parancsokat (Data Definition Language). Ez a rész bizonyítja, hogy értjük, mi történik a "motorháztető alatt".

Felhasználók tábla létrehozása:

SQL

```
CREATE TABLE IF NOT EXISTS `felhasznalok` (
  `id` INTEGER NOT NULL auto_increment,
  `name` VARCHAR(255) NOT NULL,
  `email` VARCHAR(255) NOT NULL UNIQUE,
  `username` VARCHAR(255) NOT NULL UNIQUE,
  `password` VARCHAR(255) NOT NULL,
  `role` VARCHAR(255) NOT NULL DEFAULT 'user',
  `createdAt` DATETIME NOT NULL,
  `updatedAt` DATETIME NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB;
```

Parfümök tábla létrehozása (Részlet):

SQL

```
CREATE TABLE IF NOT EXISTS `perfumes` (
  `id` INTEGER NOT NULL auto_increment,
  `name` VARCHAR(255) NOT NULL,
  `brand` VARCHAR(255) NOT NULL,
  `price` DECIMAL(10, 2) NOT NULL,
  `gender` ENUM('ferfi', 'no', 'uniszex') NOT NULL,
  `season_summer` TINYINT(1) DEFAULT 0,
  `sweet` TINYINT(1) DEFAULT 0,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB;
```

Kosár tartalom és idegen kulcsok:

SQL

```
CREATE TABLE IF NOT EXISTS `kosar_tartalom` (
```

```
  `id` INTEGER NOT NULL auto_increment,
  `kosar_id` INTEGER NOT NULL,
  `parfum_id` INTEGER NOT NULL,
  `mennyiseg` INTEGER DEFAULT 1,
  PRIMARY KEY (`id`),
  FOREIGN KEY (`kosar_id`) REFERENCES `kosar` (`id`) ON DELETE CASCADE,
  FOREIGN KEY (`parfum_id`) REFERENCES `perfumes` (`id`) ON DELETE NO ACTION
) ENGINE=InnoDB;
```

Üzleti Folyamatok Adatbázis Leképezése

Az alábbiakban bemutatunk két kritikus üzleti folyamatot és azok SQL szintű megvalósulását.

A) Kosár Megtekintése (Összetett Lekérdezés)

Amikor a felhasználó rákattint a kosár ikonra, a rendszernek össze kell fésülnie a kosar_tartalom és a perfumes táblát, hogy ne csak az ID-kat, hanem a neveket és árakat is lássa.

Backend kód:

JavaScript

```
kosar_tartalom.findAll({ include: [{ model: perfumes }] });
```

Lefutó SQL (LEFT JOIN):

SQL

```
SELECT
   `kosar_tartalom`.`id`,
   `kosar_tartalom`.`mennyiseg`,
   `perfumes`.`name` AS `perfume_name`,
   `perfumes`.`price` AS `perfume_price`
FROM `kosar_tartalom` AS `kosar_tartalom`
LEFT OUTER JOIN `perfumes` AS `perfumes`
   ON `kosar_tartalom`.`parfum_id` = `perfumes`.`id`
WHERE `kosar_tartalom`.`kosar_id` = 1;
```

Árváltozás Kezelése

Fontos tervezési döntés volt, hogy a kosar_tartalom táblában is eltároljuk az árat (ar mező).

- **Probléma:** Ha egy parfüm ára 20.000 Ft-ról 25.000 Ft-ra nő, miközben a termék már a felhasználó kosarában van, tisztességtelen lenne a magasabb árat számlázni fizetéskor.
- **Megoldás:** A kosárba helyezés pillanatában a rendszer átmásolja az aktuális árat a perfumes táblából a kosar_tartalom táblába. A végösszeg számítása mindig a kosar_tartalom.ar alapján történik, így az ár "rögzítve" van a vásárlás befejezéséig.

Összegzés és Végszó

A **Maison De Parfum** webáruház backend rendszerének megtervezése és implementálása során egy komplex, valós piaci igényekre reagáló szoftverarchitektúrát sikerült létrehoznom. A projekt elsődleges célja az volt, hogy a modern webfejlesztésben elterjedt technológiák (Node.js, Express, MySQL) segítségével egy stabil, biztonságos és bővíthető szerveroldali alkalmazást készítsek.

A projekt értékelése

A fejlesztés során sikerült megvalósítanom a vizsgaremek követelményrendszerében előírt összes funkciót. A rendszer képes:

- **Felhasználókezelésre:** Biztonságos regisztráció és bejelentkezés megvalósítása JWT (JSON Web Token) alapú hitelesítéssel.
- **Termékmenedzsmentre:** A parfümök dinamikus szűrése és lekérdezése, különös tekintettel a speciális tulajdonságokra (évszakok, illatjegyek).
- **Kosár logika kezelésére:** Egy összetett, munkamenethez kötött kosár funkció implementálása, amely kezeli a készleteket és az árváltozásokat.
- **Interaktivitásra:** A felhasználói vélemények kezelése és összekapcsolása a profilokkal.

A fejlesztés egyik legnagyobb kihívását az adatbázis-kapcsolatok helyes definiálása jelentette a Sequelize ORM-ben, különösen a kosár és a termékek közötti több-a-többhöz kapcsolat (N:M) kezelésekor. Ezt a problémát a dokumentációban részletezett kapcsolótáblás megoldással és a kaszkádolt törlések (Cascade Delete) beállításával sikeresen megoldottam, biztosítva az adatintegritást.

Szakmai fejlődés

Ez a projekt nem csupán egy kódolási feladat volt számomra, hanem lehetőség arra, hogy rendszerszintű gondolkodást sajátítsak el. Megtanultam:

- Hogyan kell egy "nyers" ötletből specifikációt, majd működő adatbázis-modellt készíteni.
- Miért kritikus fontosságú a biztonság (jelszó hash-elés, route protection) már a tervezés első fázisában.
- Hogyan teszi átláthatóbbá és karbantarthatóbbá a kódot az MVC (Model-View-Controller) tervezési minta alkalmazása.

Továbbfejlesztési lehetőségek

Bár az elkészült szoftver egy teljes értékű MVP (Minimum Viable Product), a jövőben számos ponton bővíthető lenne:

1. **Online fizetés integrációja:** Stripe vagy SimplePay API bekötése a rendelések valós idejű kifizetéséhez.

2. **Email értesítések:** A Nodemailer modul használatával automatikus visszaigazoló e-mailek küldése regisztrációkor és rendeléskor.
3. **Adminisztrátori Dashboard:** Egy grafikus felület készítése, ahol az adminok nem csak törölni tudnak, hanem statisztikákat is láthatnak (pl. legnépszerűbb parfümök, havi bevétel).

Összességében úgy érzem, hogy a **Maison De Parfum** projekt hűen tükrözi a képzés során megszerzett tudásomat. A fejlesztés során szerzett tapasztalatok magabiztos alapot nyújtanak ahhoz, hogy a jövőben Junior Szoftverfejlesztőként helytálljak a munkaerőpiacon.

7.Szoftvertesztelés

Bevezetés és Tesztelési Stratégia

A dokumentáció ezen része a **Maison de Parfum** webes alkalmazás minőségbiztosítási folyamatait és tesztelési eredményeit foglalja össze. A rendszer egy elosztott architektúrára épül, amely egy **Node.js/Express backendből** (Sequelize ORM és MySQL adatbázissal) és egy **Angular alapú frontend** kliensből áll.

A tesztelési stratégia célja a rendszer stabilitásának, biztonságának és felhasználói élményének biztosítása volt. A fejlesztés során a „Shift Left" megközelítést alkalmaztuk, ahol a tesztelés a fejlesztéssel párhuzamosan zajlott. A tesztelések során négy fő módszertant alkalmaztunk:

1. **Egységtesztelés (Unit Testing):** A legkisebb kódmodulok izolált vizsgálata.
2. **Integrációs tesztelés:** A frontend-backend és az adatbázis kapcsolatának vizsgálata.
3. **Fehér dobozos tesztelés (White-box):** A belső kódszerkezet és logika elemzése.
4. **Fekete dobozos tesztelés (Black-box):** A pontos működés vizsgálata felhasználói szemszögből.

Egységtesztelés (Unit Testing)

Az egységtesztelés során a rendszer legkisebb, önállóan működő egységeit vizsgáltuk, hogy megbizonyosodjunk arról, hogy azok a specifikációnak megfelelően működnek-e, izolálva a külső függőségektől.

Frontend Egységtesztek (Angular)

Az Angular keretrendszer beépített tesztkörnyezetét (Jasmine keretrendszer és Karma futtató) használtuk a komponensek és szervizek vizsgálatára. A beküldött kódbázisban

minden generált komponens mellett megtalálhatóak a .spec.ts fájlok (pl. register.component.spec.ts, cart.service.spec.ts).

- **Szervizek tesztelése (cart.service.ts):**
  - Kiemelt figyelmet fordítottunk a CartService logikájára, mivel ez végzi a kliens oldali állapotkezelést.
  - **Teszteset:** A getTotal() függvény helyességének ellenőrzése. Ellenőriztük, hogy a reduce metódus helyesen szorozza-e fel a mennyiséget az árral (item.price * item.quantity), és adja össze a végösszeget.
  - **Teszteset:** Az addToCart() metódus vizsgálata. Itt azt teszteltük, hogy ha egy már létező id-val rendelkező parfümöt adunk a listához, akkor ne új elem jöjjön létre, hanem a meglévő elem quantity tulajdonsága növekedjen.
- **Komponensek tesztelése (perfumes.component.spec.ts):**
  - Azt vizsgáltuk, hogy a komponens sikeresen példányosodik-e (should create).
  - Ellenőriztük a PerfumeService injektálását, biztosítva, hogy a komponens képes adatokat fogadni a szolgáltatástól.

Backend Egységtesztek (Node.js/Controller Logic)

A szerver oldalon a kontrollerek üzleti logikáját vizsgáltuk izoláltan.

- **Validációs logika (auth.controller.js):**
  - Vizsgáltuk a regisztrációs folyamat bemeneti ellenőrzéseit.
  - **Teszt:** Jelszó hosszúság validáció. Ellenőriztük, hogy a kód helyesen dob-e 400-as hibát (res.status(400)), ha a password.length < 8.
  - **Teszt:** Hiányzó mezők kezelése. Ha a req.body-ból hiányzik a username vagy email, a rendszernek meg kell állnia hibaüzenettel, mielőtt az adatbázis hívás megtörténne.
- **Adatmodellek (models/perfumes.js):**
  - Sequelize modellek definíciójának tesztelése. Ellenőriztük, hogy a DataTypes megfelelően van-e konfigurálva (pl. price mező DECIMAL(10, 2) típusú-e, a gender mező pedig ENUM-ként csak a megengedett értékeket fogadja el).

Integrációs Tesztelés

Az integrációs tesztelés során azt vizsgáltuk, hogyan működnek együtt a különálló modulok: a Frontend (Angular), a Backend API (Express) és az Adatbázis (MySQL).

API Kommunikáció és Adatfolyam

A frontend HttpClient modulja és a backend REST API végpontjai közötti kommunikációt teszteltük.

- **Regisztrációs folyamat (AuthService -> auth.routes.js -> felhasznalok tábla):**
  - Teszteltük, hogy a frontendről küldött POST /auth/register kérés helyesen megérkezik-e a backendre.
  - Ellenőriztük, hogy a bcrypt könyvtár sikeresen hasheli-e a jelszót, mielőtt az az adatbázisba kerülne.
  - Visszacsatolás: A backend sikeres létrehozás esetén 201-es státuszt küld, amit a Frontend RegisterComponent-je elkap, és egy MatSnackBar üzenettel ("Sikeres regisztráció!") jelez a felhasználónak.
- **Kosár mentése adatbázisba (kosar.controller.js):**
  - Ez egy kritikus integrációs pont. A teszt során egy bejelentkezett felhasználóval terméket helyeztünk a kosárba.
  - Vizsgáltuk, hogy a backend addtoCart függvénye helyesen kezeli-e a relációkat:
    1. Megkeresi vagy létrehozza a felhasználóhoz tartozó kosar rekordot.
    2. Létrehozza vagy frissíti a kosar_tartalom táblát a megfelelő kosar_id és parfum_id összekapcsolásával.
  - Hiba esetén (pl. nem létező perfumeId) a rendszernek 404-es hibát kellett visszaadnia, amit a frontend hibakezelő ága dolgozott fel.

Autentikációs Middleware Tesztek

- **Védett útvonalak (auth.middleware.js):**
  - Teszteltük a authenticationtoken middleware működését. Olyan API hívásokat indítottunk (pl. GET /cart/get), amelyek JWT tokent igényelnek.
  - **Eset:** Token nélküli hívás -> 403 Forbidden válasz (Sikeres teszt).
  - **Eset:** Frontend localStorage-ból kiolvasott érvényes token fejlécben való elküldése -> 200 OK válasz és az adatok elérése (Sikeres teszt).

Fehér Dobozos Tesztelés (White-box Testing)

Ebben a fázisban a forráskód belső szerkezetét ismerve vizsgáltuk a vezérlési szerkezeteket, feltételeket és ciklusokat.

Kódlefedettség és Elágazások vizsgálata

- **Admin jogosultság vizsgálata (admin.guard.ts):**
  - A kódban látható canActivate logika:

TypeScript

if (role === 'admin') { return true; } else { ... }

- Fehér dobozos tesztként kifejezetten olyan felhasználóval próbáltunk belépni, akinek a localStorage-ban tárolt role értéke 'user', de megpróbáltuk manuálisan átírni az URL-t /admin-ra. A kódlogika helyesen átirányított a /login oldalra és alert ablakot dobott, bizonyítva, hogy az else ág lefutott.

- **Parfüm szűrési logika (perfumes.component.ts):**
  - Vizsgáltuk a frontend oldali szűrést. A kódban lévő logika nem API hívást indít minden billentyűleütésre, hanem a memóriában lévő tömböt szűri.
  - Ellenőriztük a ciklus működését: ha a searchTerm üres, az eredeti lista jelenik meg. Ha van érték, a filter metódus fut le a name vagy brand tulajdonságokra.

Adatbázis Seederek és Migrációk

- **Seeder logika (perfumeSeeder.js):**
  - A kód tartalmaz egy feltételt: if (count > 0) return;.
  - Fehér dobozos tesztként vizsgáltuk az alkalmazás újraindítását. Ellenőriztük, hogy a szerver újraindulásakor (server.js -> sequelize.sync()) nem duplikálódnak-e az adatok. A feltétel helyesen működött, a bulkCreate csak üres tábla esetén futott le.

Biztonsági rések keresése a kódban

- **SQL Injection elleni védelem:** A Sequelize ORM használata miatt a paraméterezett lekérdezések automatikusak. A req.body-ból érkező adatok nem kerülnek nyers SQL stringbe összefűzésre, így a kód védett az SQL injection ellen.
- **Jelszókezelés:** A auth.controller.js-ben ellenőriztük, hogy a bcrypt.hash minden esetben meghívódik regisztrációnál, így nyílt szöveges jelszó soha nem kerül mentésre.

Fekete Dobozos Tesztelés (Black-box Testing)

A fekete dobozos tesztelés során a rendszert kizárólag a felhasználói felületen (GUI) keresztül vizsgáltuk, a belső kód ismerete nélkül, a specifikált funkcionális követelmények alapján.

5.3.1 Funkcionális Tesztek

- **Felhasználói Regisztráció és Belépés:**
  - *Teszt:* Helyes adatokkal történő regisztráció, majd azonnali belépés.
  - *Eredmény:* A rendszer sikeresen átirányította a felhasználót a főoldalra, és a menüsor megváltozott (megjelent a "Profil" és "Kijelentkezés" gomb).

- *Negatív teszt:* Már létező email címmel történő regisztráció.
- *Eredmény:* A felületen megjelent a "Létező felhasználó!" hibaüzenet (SnackBar), a regisztráció nem sikerült.

- **Kosár funkciók:**
  - *Teszt:* Parfüm kosárba helyezése a „Parfümök” oldalról.
  - *Eredmény:* A „Kosárba” gomb megnyomása után felugró üzenet jelezte a sikert.
  - *Teszt:* A jobb felső sarokban lévő kosár ikonra kattintva megnyílt a CartDialogComponent (modális ablak).
  - *Teszt:* Törlés a kosárból. A törlés gombra kattintva a tétel eltűnt a listából, és a végösszeg dinamikusan frissült.
- **Véleményezés (CRUD):**
  - *Teszt:* Új vélemény írása bejelentkezett felhasználóként. A vélemény megjelent a listában a felhasználónévvel együtt.
  - *Jogosultság teszt:* Saját vélemény szerkesztése engedélyezett volt, de más felhasználó véleményénél a szerkesztés/törlés gombok inaktívak vagy nem működtek (backend oldali védelem tesztelése a felületen keresztül).

UI/UX és Reszponzivitás Tesztelése

- **Reszponzív Design:**
  - A tesztelést végeztük asztali monitoron (1920x1080) és mobil nézet szimulációval (375x667).
  - *Eredmény:* A perfumes.component.css-ben definiált grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); szabálynak köszönhetően a parfüm kártyák elrendezése dinamikusan alkalmazkodott a képernyőmérethez. Mobilon a kártyák egymás alá kerültek.
- **Navigáció:**
  - A cssmenu működését vizsgáltuk. Bejelentkezéskor a "Bejelentkezés/Regisztráció" gombok eltűntek, helyettük a "Profil/Kijelentkezés" jelent meg (*ngIf direktívák helyes működése a DOM-ban).

Kompatibilitási Tesztek

- A rendszert a legfrissebb Chrome, Firefox és Edge böngészőkben teszteltük.
- Az Angular Material komponensek (mat-card, mat-form-field, mat-icon) tökéletesen jelentek meg minden platformon.

Összegzés és Következtetések

A **Maison de Parfum** projekt tesztelése során a szoftver egy robusztus, jól strukturált alkalmazás képét mutatta.

- **Erősségek:**
  - A backend és frontend közötti felelősségi körök jól elkülönülnek.
  - A biztonsági szempontok (jelszó hash, JWT token alapú védelem, route guardok) megfelelően implementáltak és működnek.
  - A kódolási stílus (Clean Code elvek, Angular best practices) megkönnyítette a fehér dobozos tesztelést és a hibakeresést.
  - A kosár logika (adatbázis alapú perzisztencia) biztosítja, hogy a felhasználó nem veszti el a kiválasztott termékeket oldalfrissítéskor sem.
- **Javított hibák a tesztelés alatt:**
  - Kezdetben a CartService csak lokális állapotot kezelt, de az integrációs tesztek rávilágítottak az adatbázis szinkronizáció fontosságára, ami implementálásra került a kosar.controller.js-ben.

Összességében a rendszer megfelelt a funkcionális és nem funkcionális követelményeknek.

8.Irodalomjegyzék, források

1. https://nodejs.org/en/docs
2. https://expressjs.com
3. https://sequelize.org
4. https://dev.mysql.com/doc
5. https://www.npmjs.com/package/bcrypt
6. https://www.npmjs.com/package/jsonwebtoken
7. https://www.npmjs.com/package/cors
8. https://www.npmjs.com/package/uuid
9. https://sequelize.org/docs/v6/core-concepts/assocs
10. https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous/Promises
11. https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous/Async_await
12. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
13. https://stackoverflow.com/questions/49220905/how-to-blacklist-jwt-tokens
14. https://sequelize.org/docs/v6/core-concepts/model-querying-basics/#creating-multiple-
15. instances-bulkcreate
16. https://sequelize.org/docs/v6/core-concepts/assocs/#ondelete-and-onupdate
17. https://expressjs.com/en/guide/writing-middleware.html
18. https://sequelize.org/docs/v6/advanced-association-concepts/eager-loading
19. https://sequelize.org/docs/v6/core-concepts/model-basics/
20. https://sequelize.org/docs/v6/core-concepts/data-types/
21. https://sequelize.org/docs/v6/core-concepts/validations-and-constraints/
22. https://sequelize.org/docs/v6/core-concepts/raw-queries/
23. https://sequelize.org/docs/v6/other-topics/hooks/
24. https://www.mysqltutorial.org/mysql-foreign-key/
25. https://www.mysqltutorial.org/mysql-cascade/
26. https://jwt.io/introduction
27. https://jwt.io/libraries
28. https://expressjs.com/en/guide/routing.html
29. https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Authorization
30. https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods
31. https://owasp.org/www-project-top-ten/
32. https://angular.dev
33. https://angular.dev/tutorials
34. https://angular.io/start
35. https://angular.io/tutorial/first-app
36. https://angular.io/tutorial/tour-of-heroes
37. https://angular.io/guide/architecture
38. https://www.youtube.com/watch?v=3qBXWUpoPHo
39. https://www.youtube.com/watch?v=AAu8bjj6-UI
40. https://www.youtube.com/watch?v=k5E2AVpwsko

41. https://www.youtube.com/@freecodecamp
42. https://www.youtube.com/@Codevolution
43. https://www.youtube.com/@ZoaibKhan
44. https://www.youtube.com/@TraversyMedia
45. https://www.youtube.com/@AngularUniversity
46. https://www.udemy.com/course/free-angular-course-for-beginners-professionals/
47. https://www.udemy.com/course/getting-started-with-angular-2/
48. https://www.w3schools.com/angular/
49. https://github.com/avatsaev/angular-learning-resources
50. https://github.com/quangv/awesome-angular-learning
51. https://stackblitz.com/angular
52. https://codesandbox.io/s/angular
53. https://www.freecodecamp.org/news/learn-angular-full-course/
54. https://dev.to/t/angular
55. https://hashnode.com/n/angular
56. https://www.reddit.com/r/angular
57. https://www.reddit.com/r/Angular2
58. https://stackoverflow.com/questions/tagged/angular
59. https://zoaibkhan.com/tutorials/
60. https://angular-university.io/free-courses
61. https://github.com/Cs4byk4hh/Vizsgaremek_Albok_Varga.git
62. https://classcentral.com/search?q=angular
63. https://www.apachefriends.org/download.html
64. https://code.visualstudio.com/download
65. https://nodejs.org/en/download
66. https://www.parfum.hu/uploads/cache/fb_og_image/uploads/media/a/9/7/0/tom-ford-bitter-peach-1.jpg
67. https://www.elitparfum.hu/images/cache/c07aba61a7b308f0ade5a2c5572e7916.jpg
68. https://bottega-veneta.dam.kering.com/m/1d08ceaa87f450c2/Medium-788721VV5709980_A.jpg?v=3
69. https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSNKd79y7Vx0qP0nsP9F12W8sBdlLEkrYKA7ydANkcT0FmN6gzy
70. https://img.freepik.com/premium-photo/black-icelandic-rocks-formation-product-placement-scene-premium-photo-3d-render_189851-665.jpg?semt=ais_hybrid&w=740&q=80
71. https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcT7JPKv-Q26uhNO-s5ggIgLEUuk_J1cRxUCVyfijYb81364dRhA
72. https://i.makeup.hu/z/za/zakujvvpvx1y.jpg
73. https://encrypted-tbn1.gstatic.com/images?q=tbn:ANd9GcS8_ciWZnpIyrV58DtsH09IVi1Fn5oqFp5vlUWn_IdU3ChwxBJv
74. https://p1.akcdn.net/full/184165650.versace-bright-crystal-edt-90-ml-tester.jpg

75. https://cdn.notinoimg.com/detail_main_lq/marc-jacobs/3614227372375_01-o/daisy-love-eau-so-sweet___190307.jpg
76. https://www.uniscent.hu/img/94561/PDMDEEX05_altpic_1/PDMDEEX05.webp?time=1750494478
77. https://a.allegroimg.com/s512/0090ca/d259b3bd4b7587ab07a02ce4e4fe/XerJoff-Casamorati-Dolce-Amalfi-100ml-eau-de-parfum-Marka-Xerjoff
78. https://media.douglas.hu/medias/b9hoTf1040969-0-global.jpg?context=bWFzdGVyfGltYWdlc3wxMDQ1OTR8aW1hZ2UvanBlZ3xhREJpTDJoalpDODJNemN6TXpnek5EZzNORGt4TUM5aU9XaHZWR1l4TURRd09UWTVYekJmWjJ4dlltRnNMbXB3Wnd8MjcxZDYwNDM2ZDRmYTJkYTQxZGRjMzJmODMwOWQxYTg4YTY5MThkZjUzZTQ4MWE4ZTE3ZDg5YjRmNDUzYzMwMA&grid=true
79. https://cdn.notinoimg.com/social/hermes/3346130491736_01/colognes-collection-eau-de-pamplemousse-rose___220408.jpg
80. https://cdn.notinoimg.com/detail_main_lq/jimmy-choo/3386460025478_01-o/for-women___220118.jpg
81. https://s13emagst.akamaized.net/products/109368/109367723/images/res_b1b0a335f03cbeb9a6b07bd6fe14270d.png?width=450&height=450&hash=54FCF57E16C02485FD337E71B6008EBE
82. https://encrypted-tbn3.gstatic.com/images?q=tbn:ANd9GcTql7zpJUCUnAXLlxbV2-T0rKLQ33DECl2pZCZASFA5e1Lo1owT
83. https://renvishop.hu/img/39247/1970530110_altpic_2/1970530110.webp?time=1751122236
84. https://encrypted-tbn2.gstatic.com/images?q=tbn:ANd9GcQE_8CPJrajteLYp3j85C4ZQXPdI47B-YZkGFP5MbDrDWI7dMLv
85. https://i.makeup.hu/c/ch/chv4f2dgzmpc.jpg
86. https://media.douglas.hu/medias/usjHV1775554-0-global.jpg?context=bWFzdGVyfGltYWdlc3w4MTc3NnxpbWFnZS9qcGVnfGFEQmpMMmczWmk4Mk16YzFNRFUzTURjd05Ea3lOaTkxYzJwSVZqRTNOelUxTlRSZk1GOW5iRzlpWVd3dWFuQm58ODIyNjNhODIyYWY4ZTBkZThiOGE0YmQ0OTg1MTVjZTNiN2RhMWQ4NWMyYjY1YTBkY2VjYzhiMjg2NjJiNWQ2Mw&grid=true&imPolicy=grayScaled
87. https://encrypted-tbn3.gstatic.com/images?q=tbn:ANd9GcR3Qy5_IeCB9g0wLq_JSKTVoyATMWdiNowf9yrHqnLrsMVuA7_O
88. https://p1.akcdn.net/full/465658699.dolce-gabbana-light-blue-eau-intense-pour-homme-edp-200-ml.jpg
89. https://p1.akcdn.net/full/614218647.tom-ford-lost-cherry-edp-100-ml.jpg
90. https://cdn.notinoimg.com/detail_main_lq/maison-margiela/3614272404670_06/replica-under-the-lemon-trees___220531.jpg
91. https://encrypted-tbn1.gstatic.com/images?q=tbn:ANd9GcQyEk8wtb8M2e5U6a2QnTNGnZZJTdqTk6OGsKvmC0YD-Bqjkywv

92. https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcR2hOJYbknwxLBhB9ix62Rr5s4a5xt2rXiPIp2TWKO6w8IDUfcv
93. https://encrypted-tbn1.gstatic.com/images?q=tbn:ANd9GcSTr-IiEhV8yWHsQ6uxSaR8aeb55hEwCC9q6PIeJm0aPL9czHDt
94. https://cdn.notinoimg.com/detail_main_lq/yves-saint-laurent/yslop9w_aedp20_02-o/opium___130620.jpg
95. https://encrypted-tbn3.gstatic.com/images?q=tbn:ANd9GcRY1mBJQvQtlCwUW1E45rL9gioKKyH9bxVj3bgdYXlshZbI0-ZI
96. https://www.sephora.com/productimages/sku/s2799187-main-zoom.jpg?imwidth=465
97. https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQSpOArmKqTMwt-Y9MenkGOp1sKIEw_ykBg0HUxHTqmwO9u4JlF
98. https://p1.akcdn.net/full/533839230.herm-s-terre-d-hermes-eau-intense-vetiver-edp-50-ml.jpg

99. https://media.douglas.hu/medias/NXX0ll1161753-0-global.jpg?context=bWFzdGVyfGltYWdlc3wyMDA4NTd8aW1hZ2UvanBlZ3xhR0poTDJobVlpODJPVEkzTlRZMU16WTJPRGc1TkM5T1dGZ3diR3d4TVRZeE56VXpYekJmWjJ4dlltRnNMbXB3Wnd8MjIwY2EyNzBmNmExZGVmOTEwYjI5ZDJiNDI5YTEzOTYxNGJmOTZkZDNmMjQyZjBiMDFjODBkOWY2YmNjMGExYw&grid=true&imPolicy=grayScaled

100. https://encrypted-tbn2.gstatic.com/images?q=tbn:ANd9GcSBUvjAzpMn5i0rDlEHHmVz7QaFjAMIpm6NK0CYErgahtfeU-WL

101. https://www.parfumcenter.hu/data/images/products/4904/valentino-donna-born-in-roma-intense-08.jpg

EREDETISÉGI NYILATKOZAT

Alulírott, Albók Csaba oktatási azonosító: 72721012998(backend, middle, teszt) és Varga-Ádám Róbert (frontend) oktatási azonosító: 72689974565

a Szent István Katolikus Technikum és Gimnázium végzős diákjai ezennel büntetőjogi és fegyelmi felelősségünk tudatában nyilatkozunk és aláírásunkkal igazoljuk, hogy „**MAISON DE PARFUM”** című vizsgaremek saját, önálló munkánk; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén munkám visszautasításra kerül.

Sátoraljaújhely 2026.év február hó 13 nap

Albók Csaba Varga-Ádám Róbert